VERTICAL AND HORIZONTAL EXPANSION OF FUZZING TECHNIQUES

by

DYLAN J. WOLFF
(B.S., Boston College; M.Sc., ETH Zürich)

A THESIS SUBMITTED FOR THE DEGREE OF

DOCTOR OF PHILOSOPHY

in

COMPUTER SCIENCE

in the

GRADUATE DIVISION

of the

NATIONAL UNIVERSITY OF SINGAPORE

2026

Advisor:
Professor Abhik Roychoudhury

Examiners:
Associate Professor Trevor Carlson
Assistant Professor Manuel Rigger

Declaration

I hereby declare that this thesis is my original work.
I have duly acknowledged all the sources of information which have been used in the thesis.

This thesis has also not been submitted for any degree in any university previously.

Use of Generative AI I acknowledge the use of generative artificial intelligence (AI) tools in the preparation of this thesis, solely for language refinement, structuring and proofreading purposes. All AI-generated outputs have been critically reviewed, edited, and verified by me. I confirm that the intellectual content, analysis, and conclusions presented are entirely my own, and I take full responsibility for the work submitted.

Dylan J. Wolff

27 April 2026

Contents

Abstract **v**

List of Figures **vii**

List of Tables **x**

Acknowledgments **1**

1 Introduction **2**

2 Background **7**

2.1 Dynamic Analysis and Fuzzing 7

2.1.1 A General Fuzzing Algorithm 8

2.1.2 Fuzzer Taxonomy 9

2.2 Concurrency 15

2.2.1 Definition of Concurrency Bugs 15

2.2.2 Controlled Concurrency Testing 17

3 Motivating User Study **20**

3.1 Survey Methodology 21

3.2 Survey Results 24

3.2.1 Workflows and Use Cases (RQ1) 24

3.2.2 Fuzzer Setup and Inputs (RQ2) 26

3.2.3 Fuzzer Outputs and Artifacts (RQ3) 27

3.2.4 Fuzzer Evaluations (RQ4) 28

3.2.5 Summary 30

3.3 Threats to Validity 30

3.4 Discussion 31

I Vertical Expansion to Developer Workflows 32

4 Change-Based Fuzzing 33

4.1 Evaluation of Fuzzers in a Commit Fuzzing Scenario 34
4.1.1 Experiment Setup 34
4.1.2 Effectiveness of Existing Fuzzers (RQ1) 38
4.1.3 Impact of Additional Variables on Fuzzer Effectiveness (RQ2) 40
4.2 Threats to Validity 42
4.3 Discussion 44

5 A Fuzzing-Integrated Cyber Reasoning System 46

5.1 A Cyber Reasoning System 47
5.2 Opportunities for LLMs in Software Analysis 52
5.2.1 Augmenting Individual Analyses with Developer Intent . . . 52
5.2.2 Generalizability and Flexibility of LLMs 54
5.2.3 Opportunities for Agentic AI 55
5.3 Challenges for LLMs in CRSs 56
5.4 Threats to Validity 57
5.5 Discussion 57

II Horizontal Expansion to Concurrent Systems 59

6 Structured Grey-box Concurrency Fuzzing 60

6.1 Overview 63
6.2 Core Approach behind `RFF` 65
6.3 `RFF` Architecture and Implementation 72
6.3.1 Schedule Fuzzer 75
6.4 Evaluation 76
6.4.1 Experimental Setup 77
6.4.2 Comparison with the State-of-the-Art (RQ1) 80
6.4.3 Contribution of the Abstract Schedule (RQ2) 81
6.4.4 Exploration of the Schedule Space (RQ3) 82
6.4.5 Reads-from Testing via Q-Learning (RQ4) 83

6.5 Threats To Validity . . . 84
6.6 Discussion . . . 85
6.7 Mean Number of Schedules to 1st Bug . . . 86

7 Directed Concurrency Fuzzing 87
7.1 Motivation and Taxonomy . . . 88
7.2 Approach . . . 91
7.2.1 Tackling **(C1)** With Unstructured Concurrency Fuzzing . . . 93
7.2.2 Concurrency-aware Program Dependency Graph . . . 94
7.2.3 Addressing **(C2)** With Directed Fuzzing . . . 96
7.2.4 Mitigating **(C3)** With Directed Mutation . . . 97
7.2.5 Opportunistic Prefix Reduction . . . 98
7.3 Implementation . . . 98
7.4 Evaluation . . . 100
7.4.1 Experimental Setup . . . 101
7.4.2 RQ1: Bug Finding Ability . . . 104
7.4.3 RQ2: Patch Testing . . . 106
7.4.4 RQ3: Static Analysis Effectiveness . . . 107
7.5 Discussion . . . 109

8 Concurrency Fuzzing in Kernel Code 111
8.1 Design of SECT . . . 114
8.1.1 SECT-Scheduler for Concurrency Fuzzing . . . 115
8.1.2 Scheduling Point Injection . . . 117
8.1.3 Input Fuzzing Loop . . . 118
8.2 Implementation . . . 119
8.3 Evaluation . . . 120
8.3.1 Experiment Setup . . . 120
8.3.2 Code Coverage and Performance Overhead . . . 122
8.3.3 Scheduling Effectiveness . . . 123
8.4 Real-world Bug Finding . . . 126
8.4.1 Case Study: Use-after-free in `sl_sync` . . . 126
8.4.2 Case Study: Partial Fix in JFS . . . 127

8.5 Limitations 128
8.6 Discussion 129

III Related Work and Conclusion 130

9 Related Work 131
9.1 Fuzzing and Cyber Reasoning Systems 131
9.2 Concurrency 134

10 Conclusion and Future Work 140
10.1 Overview 140
10.2 Perspectives 142
10.3 Future Directions 143
10.4 Concluding Remarks 145

Bibliography 146

Publications during PhD Study 175

Abstract

Vertical and Horizontal Expansion of Fuzzing Techniques

by

Dylan J. Wolff

Doctor of Philosophy in Computer Science

National University of Singapore

Fuzzing —a biased random exploration of software behavior— has become one of the most successful automated techniques for discovering vulnerabilities in large, real-world systems. Yet despite its remarkable efficacy, fuzzing remains a point technology. Today's fuzzers excel only on stateless, sequential programs and are largely isolated from the broader software engineering workflows in which security and reliability failures are discovered, diagnosed, and repaired. Indeed, the use of fuzzing is currently an "after-thought", testing a fully developed software project instead of integrating proactively into software development processes. As software systems grow increasingly large, concurrent and interdependent, we need techniques that both scale vertically into richer software workflows and horizontally across richer program behaviors. This thesis investigates how fuzzing techniques can be expanded holistically to answer the challenges presented by our vast and growing software infrastructure.

First, this thesis examines how fuzzing can be embedded more naturally within the software development and maintenance practices. Through an empirical study, it identifies how fuzzing can be shifted earlier in the development lifecycle and integrated into automated CI/CD workflows. It also demonstrates how fuzzing can form the foundation of end-to-end automated security systems encompassing testing, crash triage, debugging, and automated repair. It presents an example of such an integrated system, and examines how recent advances in Large Language Models can best be leveraged to improve on such systems in the future.

Second, it develops new systems and algorithms that extend fuzzing into the domain of controlled concurrency testing (CCT), enabling exploration of the vast, stateful, and nondeterministic behaviors of modern multithreaded software. These

contributions include new CCT platforms for user-space and kernel-space targets, efficient mechanisms for schedule control, and novel randomized algorithms that provide semantic feedback for exploring deep concurrency bugs. Evaluated across both C/C++ user applications and the Linux kernel, these techniques uncover elusive concurrency defects that evade existing tools.

Together, these contributions present a unified view of fuzzing as a general-purpose behavioral search methodology: one that can explore concurrent executions, integrate with modern development practices, and support autonomous vulnerability discovery and repair. This work suggests a path toward holistic, automated software protection infrastructures capable of handling the complexity of future software systems written by both humans and generative AI tools.

List of Figures

2.1 A general fuzzing system. The fuzzer ① creates a new input, ② runs the input on the SUT, ③ records results of the execution, and ④ saves the input if it deems the results interesting (e.g. a crash was observed). 9

2.2 A code snippet (left) and its corresponding state-transition diagram for grey-box fuzzing (example from [27]). Transition probabilities are derived from a mutation of a single ASCII character: e.g. 2^{-2} probability of picking a particular character to change and 2^{-8} probability of picking the correct byte. 12

2.3 A program illustrating a data-race free order violation concurrency bug (left). Two serializations show schedules in which the bug does not (right-middle) and does (right) occur. 16

3.1 Participant's years of experience in their current role. 22

3.2 Participant's familiarity with fuzzing/testing tools. 23

3.3 Participants' opinions about the most important testing properties. . . 26

3.4 Participants' willingness to configure and setup the fuzzer. 27

3.5 Cumulative presentation of the expected fuzzing campaign lengths for the different workflow types. 29

4.1 **(Left)** Initial Corpus Coverage Normalized to Maximum Observed Coverage per Program and **(Right)** Initial Corpus Size (Number of Test Cases) 39

4.2 Probability that the regression bug **is not found** by each fuzzer over time (lower is better). **(top)** using static corpus; **(middle)** using saturated corpus; **(bottom)** using the saturated corpus *excluding* build times . . 45

5.1 A Cyber-Reasoning System Workflow; Example CRS tools listed within each component 47
5.2 Example of An Incomplete SemGrep Analysis 53
5.3 Input Structure and Constraints for the Jenkins Exemplar Bug from the AIxCC Semi-final Competition 53
5.4 Prompting Template and Templated Values 55
5.5 Python Constraints Passed to an LLM in the Template of Figure 5.4 for the *Translated* Jenkins Exemplar 56

6.1 SCTBench subject `reorder_100` (repurposed) 63
6.2 State-Transition for scheduling reads-from constraints. En and Lw represent the set of enabled events and last write event on the same location as `w`. Solid arrows indicate actions that facilitate the constraint. Dashed arrows indicate actions that inhibit the constraint. Label enable(op) indicates that op is ready to be executed. Label exec(op) indicates that the action executes op. Label overwrite(w) indicates executing another write $\mathtt{w}'$ on same variable as `w`. 70
6.3 Overall system architecture of RFF. The two main components are (1) the instrumented *Program Under Test* (PUT), and (2) the *schedule fuzzer*. The workflow takes in an uninstrumented program (`prog.exe`) and a test *input*, and outputs a set of observed racey schedules (if any). The PUT is executed with mutant schedules, with observed schedules as feedback. 75
6.4 Total Bugs Discovered After Log(# Schedules) Across All Trials (higher is better)[5] 80
6.5 Log-scale frequency of reads-from sequences observed by POS (top) and Sect (bottom) in 10000 executions of the `SafeStack` program 83

7.1 Motivating example concurrent program. Highlighted in red are instructions contributing to a noisy *state* space ($\propto 2^{\mathtt{M}}$). Blue regions introduce noise in the *schedule* space ($\propto$ N). Preemption depth is $\propto$ D. 89

7.2 Comparison of Restricted Mean Survival Time (RMST) across different scheduling algorithms. **Lower** RMST indicates faster bug discovery, truncated lines indicate the bug was *not* found in any trial within 5000 schedules. Each curve shows the average RMST for the assertion in Figure 7.1 as depth (D), state noise (M) and/or schedule space noise (N) increase. 90
7.3 Simplified Program Dependency Graph (PDG) for the program in Figure 7.1 up to line 9. Dashed edges represent ① control dependencies, solid edges represent ② intra-thread dataflow, and dotted edges represent ③ inter-thread reads-from relations. Each node represents a single load (L) or store (S) instruction of a variable in the program. The assertion targeted is the $L(d)$ on the Main Thread. 95
7.4 Cumulative unique RFVs observed at changed locations in patch-testing of embedded database systems. 106
7.5 Distribution of distances in PDG for blackscholes (left) and raytrace2 (right); true positives in orange, false positives in blue. Medians shown with dashed lines. 109

8.1 Overview of SECT (components of SECT in blue). 114
8.2 The safety check for additional scheduling points injected by SECT instrumentation. 119
8.3 Code coverage achieved by Syzkaller, SegFuzz and SECT (N=10). . . . 122
8.4 Probability that the bug is not found after a number of schedules have been explored (lower is better) (N=20). 124
8.5 Kaplan-Meier survival plot for a newly confirmed use-after-free bug found by SECT in `sl_sync` (N=20). 127
8.6 A patch for a partial fix for bug in `jfs` filesystem highlighted in blue. The additional changes needed for a complete fix are highlighted in yellow. 128

List of Tables

4.1 Probability of Bug Discovery for Commit-level Fuzzing of Open Source Programs with Time Limit *including build times* of 5 minutes (P_{5m}) and 1 hour (P_{1h}). Results shown for using a saturated (-sat.) or static (-static) corpus. 35

7.1 Bug-Finding results on RaceBench [191] subjects. Each row shows the Mean ± Standard Deviation and total unique ('Uniq') bugs found out of 100 across N=10 trials. A † indicates the target was instrumented with sampling instrumentation to increase throughput for all tools. **Bold** indicates the statistically highest mean number of bugs found at $p < 0.05$ [143]. 102

7.2 Number of bug discoveries in selected subjects from SCTBench [199] and ConVul [35] (N=30 trials). 104

7.3 Estimated statistics for the PDG on RaceBench programs. **Bolded** effect size indicates statistical difference between true and false positive distances at $p < 0.05$ [143]. 109

8.1 State-of-the-art kernel concurrency fuzzers. 112

8.2 Average fuzzing throughput and concurrent testcase-latency over 48 hours (N=10). 122

8.3 Known concurrency bugs found by SegFuzz, Syzkaller, SECT (N=20). Note that SECT_RW is default SECT. 124

8.4 Kernel discovered by SECT 126

Acknowledgments

Firstly, I'd like to thank all of my co-authors and research collaborators. Shi Zheng for the many shared iterations (and frustrations) on RFF. Zhao Huan for bringing his unique perspective to each discussion and joining me in the realm of concurrency testing. Xing Yi Han, for her openness to feedback and willingness to tackle hard problems. Mehtab Zafar for his eagerness to learn and tireless commitment to every project. Martin Mirchev for his fearlessness and zeal. Jiacheng Xu for his dedication. Zhengxiong Luo for his quiet tenacity. Ridwan Shariffdeen and Yannic Nöller, for their support and leadership. I'd also like to thank all of my fellow Cyber Reasoning System collaborators (too many to list).

I'm so happy to have worked alongside so many incredible researchers in the TSS group. Thanks to Yuntong Zhang for the many, many speculative chats over the years. I still can't believe we managed to go five years without working on anything directly together. And thanks to Ruijie Meng, Sungmin Kang, Haoxin Tu, Haifeng Ruan, Ali El-Husseini, Jiawei Wang, Andreea Costea, Yahui Song and the rest of the group for sharing their thoughts (and occasionally lunches) with me.

Many mentors have also been instrumental to my progress as a researcher and to this thesis. Thanks to members of my thesis committee, Manuel Rigger and Trevor Carlson, for their constructive feedback. To Marcel Böhme, for guiding me through the crunch and then slow iterations on that first submission. Cristian Cadar, for his advice and enthusiasm. Umang Mathur, for helping me find my footing in concurrency testing and changing my perspective as a researcher.

And of course, I am thankful to my advisor Abhik Roychoudhury, who always made himself available for interesting discussions. I may not have always followed his advice (often to my detriment and his frustration), but I always valued it. His unique perspective on research is one that has greatly influenced me; I think I am just now starting to see the forest from the trees.

Most of all, I'm grateful to my wife, Daryl, without whom I would never have made it this far. I owe everything to her unwavering support.

Chapter 1

Introduction

Software systems control numerous key aspects of society such as government agencies, medical services, utilities, national defense infrastructure and beyond. A core challenge in ensuring these systems are reliable and secure, is their diversity and scale. Modern software is composed of components written by different people, organizations, and, increasingly, generative AI tools. The resulting systems are composed of many programming styles, languages, deployment environments, and dependencies. Furthermore, despite the ubiquity of parallelism and concurrency, reasoning about the exponentially large space of interleavings in such programs remains a challenging task today. And reasoning about the correctness of our software systems is *critical*. For example, a recent major AWS outage caused by a latent race condition in a DNS server [215] had cascading effects across industrial sectors [217] and cost estimated *billions* of US dollars in losses [85]. This overwhelming complexity can not only lead to additional bugs and vulnerabilities being introduced, but also increase the difficulty of analyzing and reasoning about these systems.

Indeed, as generative AI accelerates the pace of software development and the quantity of code produced continues to grow, our software infrastructure is becoming so large that it simply *cannot* be manually audited. Highly automated reasoning and analysis tools are therefore *essential* to ensure that these systems do not collapse under the weight of their own complexity. Software is becoming so diverse and convoluted that building effective automated analysis tools also presents acute challenges. As a result, even the very tools developed to mitigate security issues in our software infrastructure are not exempt from the perils of growing complexity.

This was made abundantly clear by the recent CrowdStrike incident[1], where the security system itself had a bug that caused worldwide outages across countless industrial sectors.

Various automated reasoning techniques for software programs have been the subject of extensive research for decades [20, 33]. For example, software model checking is an automated exhaustive search of the state space of an abstraction of the program [31]. However, exhaustive approaches such as model checking are limited by the state space explosion problem [45], which makes them impractical for large-scale systems containing intractably large state spaces. Additionally, many automated approaches like model checking use *symbolic* reasoning to explore program behaviors. However, this symbolic reasoning requires *significant* modeling to capture environmental interactions and semantics of today's programs. As this modeling is a manual process, symbolic reasoning is typically *not* a fully automated solution in practice, despite its promise. Even when full models are available, solving symbolic constraints is generally NP-hard or even undecidable for many common theories, again limiting the scalability of these symbolic techniques.

Recently, fuzzing [158] has emerged as a fully automated analysis technique that can be readily applied to a wide range of software systems. Rather than relying on an exhaustive search and symbolic reasoning, fuzzers conduct a stochastic search over *concrete* program executions. By *biasing* the randomized search, as in *grey-box fuzzing*, fuzzers can often explore deep program behaviors *without* heavy-weight analysis or modeling. As a result, fuzzing can scale to extremely large systems, and is widely used in a broad array of real-world software projects [204].

While fuzzing has demonstrated its effectiveness in certain domains, such as single-threaded parsers, many other domains still remain out-of reach for fuzzers. Multi-threaded and concurrent systems are especially problematic; not only is exploration of the scheduling space generally out-of-scope for most fuzzers, but also non-determinism arising from the OS scheduler can degrade the effectiveness of fuzzing and lead to bugs which cannot be easily reproduced. Fuzzing is also typically limited to a post-development, maintenance phase of the software development lifecycle. Even in this context, fuzzers are often limited by the workflows in which

[1] https://www.crowdstrike.com/falcon-content-update-remediation-and-guidance-hub/

they are deployed. A fuzzer-detected vulnerability may only be triaged and repaired by an expert in downstream, dependent software 90-150 days after it is reported due to a shortage of cyber-security professionals [91]. During this period, the vulnerability has often been widely published, but other software dependent on older or forked versions of the component in question may still be vulnerable. Thus, to ensure that our software is truly secure, a holistic approach is needed that can bring the automation of fuzzing through all stages of the development lifecycle.

In this thesis, we propose an expansion of randomized search techniques, called fuzzing. We motivate our work through a user-study of software practitioners, in which we ask them to assess what would be needed to drive broader adoption of automated reasoning via fuzzing. Here we identify two key directions which we explore in this thesis. First, we examine how fuzzing can be embedded into the development process to ensure that fuzzing is both automated itself, and part of a more automated end-to-end workflow. In particular, we empirically assess current research and industry fuzzing tools in the context of *commit fuzzing*, where fuzzers are automatically triggered on every change to a codebase. We also demonstrate a more forward looking approach, in which fuzzers are integrated into a *Cyber Reasoning System* (CRS), which can automatically detect, triage and repair vulnerabilities without human intervention. Second, we investigate how a fuzzing-inspired approach of biased random search can be leveraged beyond sequential software. We first apply a fuzzing-inspired biased random search algorithm to search the scheduling space of these systems using a novel schedule structure based on the reads-from relation. Next, we examine how a directed approach can gain the benefits of grey-box schedule fuzzing without being susceptible to state and schedule-space noise. Lastly, we investigate novel approaches for *controlling* the schedules of concurrent systems, including the Linux kernel itself. Overall, through these two complementary directions –vertical and horizontal– we show how fuzzing can be used to improve the security and reliability of software systems, not just today, but also in the future.

Concretely, this thesis makes the following contributions:

Conceptual

– We develop the notion of structured grey-box fuzzing as a novel algorithmic

framework for concurrency testing, including a new schedule representation

– We motivate and propose directedness in the context of concurrency testing and illustrate how a stateful, directed approach addresses fundamental challenges in concurrency testing

Empirical

– We conduct a user study motivating expansion of fuzzing into new workflows and domains

– We evaluate the capabilities of existing fuzzers in one such new domain, regression fuzzing

Technical

– We construct several controlled scheduling tools for both kernel and user-space code which can be used for concurrency fuzzing

– We instantiate and make openly available a full-fledged Cyber Reasoning System, which bridges multiple workflows with fuzzing to autonomously both find and fix vulnerabilities in software systems

Thesis Organization We begin with the requisite background material on fuzzing and concurrent systems in Chapter 2. Next we discuss our motivating user-survey in Chapter 3, in which we queried software professionals about their experiences with fuzzing and their expectations for future fuzzing tools. Then we move on to the first key direction identified in the user-study, Part I, in which we explore the *vertical* expansion of fuzzing into more deeply integrated software workflows. We begin by examining how fuzzers can be more tightly integrated into CI/CD and local software development workflows in Chapter 4. Then, in Chapter 5 we build and analyze a fully autonomous Cyber Reasoning System (CRS) that leverages fuzzing as part of a end-to-end vulnerability discovery and remediation process. In the second part of this thesis, we look at a *horizontal* expansion of fuzzing into the new domain of concurrency testing in Part II. Chapter 6 introduces a novel

algorithmic framework for concurrency fuzzing. Next, we explore how directedness can be added to schedule fuzzers to improve their effectiveness in Chapter 7. We then provide an overview of how schedules can be controlled in concurrent systems to enable fuzzing for kernel-space applications in Chapter 8. After a discussion of related work in Chapter 9, we conclude with a summary of our key contributions, a look towards the future of autonomous program analysis, and some perspectives in Chapter 10.

Chapter 2

Background

2.1 Dynamic Analysis and Fuzzing

Dynamic analysis techniques examine one or more program executions to detect bugs. By only focusing on a finite, often bounded, number of program executions, dynamic analyses are nearly always incomplete, and can miss some software bugs on unseen program executions. However, such analyses are typically sound in that any reported bugs are known to be true positives because they have been observed in a concrete execution trace. In particular automated randomized testing, or *fuzzing* has become a popular dynamic analysis tool. While software testing has been a technique used by engineers for assurance even earlier, the term "fuzzing" or "fuzz testing" was coined by Barton Miller in 1990 [158]. By his initial definition, fuzzing consisted of feeding purely randomized inputs to programs to test their reliability. In their first publication on fuzzing, Miller, Fredriksen and So found that, surprisingly, more than 25% of UNIX utility programs would eventually crash if fed random inputs [158]. Since its inception, the term fuzzing has grown to encompass many automated testing techniques, typically those utilizing randomness to generate inputs. These range from Miller's initial *black-box* formulation of fuzzing to dynamic symbolic execution, often referred to as *white-box* fuzzing, which leverages logical constraints derived from detailed knowledge of system under test to generate test cases. *Grey-box* fuzzers fall in-between these two extremes, using lightweight instrumentation to bias a randomized search over the space of program inputs.

In recent years, fuzzing has been increasingly embraced by the open source

community, industry and academia. Google has found over 36,000 bugs in more than 550 open source projects using OSS-Fuzz [204]. Beyond only OSS-Fuzz, there are over 5000 open source projects on GitHub alone which are related to fuzzing, 1000 of which have at least 5 stars [3]. In industry, fuzzing has seen adoption from major tech companies such as Cisco [211], Amazon [232], Google [204] and Microsoft [39]. In fact, fuzzing has been so successful internally at Microsoft that it is now a required part of their development lifecycle for all untrusted interfaces [86].

2.1.1 A General Fuzzing Algorithm

We begin with an outline of typical a grey-box, mutational fuzzing algorithm, since this is the most prominent fuzzing variant and the variant most relevant to this thesis. More precisely, a classic fuzzing algorithmic framework for sequential programs which take a single input (test-case), process that input, and possibly produce an output before terminating. We present a sketch of our fuzzing algorithm in Algorithm 1. A fuzzer takes as inputs the system under test, as well as an optionally an initial corpus (line 1). Once the fuzzer has preprocessed the system under test, it proceeds to the main fuzzing loop (line 2). For mutational fuzzers, program inputs are derived by applying random perturbations called *mutation* to existing inputs (line 5). The input to be mutated in this step is sampled proportionally to a fuzzer-specific fitness function pickNextAndAssignEnergy, with more favorable inputs being mutated more frequently (often called a *power schedule*). It then executes the system under test on the input and records the result of the execution, both in terms of output as well as any other behavior that may be captured by instrumentation such as code coverage (line 6-8). The output for a fuzzing campaign is a set of witness inputs for each bug and/or interesting behavior found by the fuzzer (line 10).

Figure 2.1 shows a similar general fuzzing system graphically. Here step ① corresponds to the test case creation in Algorithm 1 (line 5). Step ② shows the execution of the target program (Algorithm 1, line 6, left-hand side). The ③rd step shows the result(s) of the execution being recorded by the fuzzer (Algorithm 1, line 6-7). Finally, based on the fuzzer's decision on whether or not an input is interesting, step ④ accumulates inputs (Algorithm 1, line 7).

Algorithm 1: Grey-box Fuzzing

```
Input: Initial corpus of inputs I_init
I ← I_init, I_fail ← ∅
repeat
    (i, η_i) ← PickNextAndAssignEnergy(I)
    for n ∈ { 1, ..., η_i } do
        i_mut ← mutateInput(i, I)
        if i_mut crashes then
            S_fail ← I_fail ∪ { i_mut }
        if isInteresting(i_mut, I) then
            I ← I ∪ { σ_mut }
until timeout
return I_fail
```

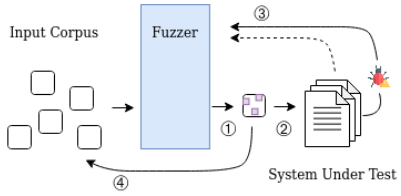


Figure 2.1: A general fuzzing system. The fuzzer ① creates a new input, ② runs the input on the SUT, ③ records results of the execution, and ④ saves the input if it deems the results interesting (e.g. a crash was observed).

2.1.2 Fuzzer Taxonomy

Fuzzers are often broadly categorized by their input creation strategy (step ①, Figure 2.1), their level of insight into the target program (e.g. step ③, Figure 2.1), and the set of behaviors that they target in that program (Section 2.1.2.3). In the following section, we give an overview of various classes of fuzzers used by practitioners and researchers today, as well as the strengths and weaknesses of each technique.

2.1.2.1 Program Insight

One of the two primary axes on which fuzzers are categorized is their knowledge of the system under test. This knowledge is also correlated with the speed at which the approach can process new inputs; more insight into the program typically leads to lower testing throughput due to additional time spent gathering and analyzing information about the program.

Black-Box fuzzing assumes no access to the target program, treating it as a "black-box". In essence, this means that the IsInteresting function of Algorithm 1 only takes into account program outputs to assess a given test case and that there is no additional instrumentation to provide feedback from an execution. Early approaches to fuzzing, such as Miller's began as black-box testing tools [84] [1] [8].

Despite their simplicity, black-box fuzzers have been effective in finding vulnerabilities in software where instrumentation is difficult [47] or impossible [147]. For example, there are many instances when part or all of an application are opaque because of intellectual property restrictions or network partitions for vendor software, in which case network-based black-box fuzzers such as Restler [13] can still be used to find bugs.

White-Box fuzzing [72] leverages detailed knowledge of the program under test via a *dynamic symbolic execution* engine to generate tests. Dynamic Symbolic Execution (DSE) tracks the symbolic values of program variables for a given execution path, as well as the path constraints themselves for that execution in terms of the input variables [110]. To generate new testcases, DSE approaches use an SMT solver to find a satisfying assignment to the program inputs for a new set of path constraints. DSE can be broadly divided into two techniques [34]: *Concolic Testing* [200][188] and *Execution Generated Testing* (EGT) [33] [207]. Concolic executors gather path constraints and symbolic values as the system under test is executed concretely, generating new inputs by solving a formula with one of the gathered path constraints negated. In contrast, EGT symbolic executors work incrementally, forking for each branch during symbolic execution. Additionally, EGT only selectively concretizes values when they are needed for external calls or to generate a final input for a completed path, rather than following a concrete execution path.

Despite being a powerful technique, dynamic symbolic execution brings with it several disadvantages and challenges. While advances in Conflict Driven Clause Learning-style satisfiability solvers have made many problems tractable in practice [50] [17], the Satisfiability Modulo Theories (SMT) formulas generated by DSE are either NP-hard or undecidable to solve in the worst case, depending on the theories involved. For some constraints, and even some whole theories like floating-point numbers and arrays, SMT solvers still struggle to produce satisfying assignments quickly or at all. Additionally, interaction with the program environment (OS, network etc.) is opaque to an SMT solver. While some solvers like KLEE [33] have extensive models for components of the program environment, it is not typically feasible to comprehensively model all external interactions for real world programs. Lastly, exploration of the exponentially large space of paths is a problem common

to all testing techniques, but it is particularly acute in the context of symbolic execution. This is because constraints are explicitly mutated to explore each possible program path, rather than a coarse approximation of program behavior such as code coverage. With such a large search space, it is easy to spend excessive time exploring behaviourally similar or even identical paths, and several research efforts have been directed towards mitigating this issue [28] [93].

Grey-Box fuzzing is a middle ground between white and black-box fuzzing. In grey box fuzzing, the testing tool uses a light-weight analysis or feedback mechanism, typically achieved through program instrumentation. In essence, this means that the IsInteresting function in Algorithm 1 processes this feedback after each execution, in addition to looking at the outputs. The most common kind of feedback for grey-box fuzzers is a measure of code coverage. Notably fuzzers do not use path coverage to avoid retaining too many inputs which cover behaviourally similar executions, as is an aforementioned issue with symbolic execution. This approach allows grey-box fuzzers to mostly maintain the throughput of black-box fuzzing while still having some insight into the program's behavior. Grey-box fuzzing was popularized by AFL [9] and later LibFuzzer [127], and many newer grey-box fuzzers [25] [27] [59] are extensions of these two implementations. Grey-box fuzzers have garnered much attention because they make very few assumptions about the system under test, often only requiring an additional compiler pass for instrumentation without manual intervention. They are also very effective, outperforming symbolic execution engines on real-world software in many cases [67].

```
1 foo(char* s) {
2  if (s[0] == 'b') {
3   if (s[1] == 'a') {
4    if (s[2] == 'd') {
5     if (s[3] == '!') {
6         abort();
7 }}}}}
```

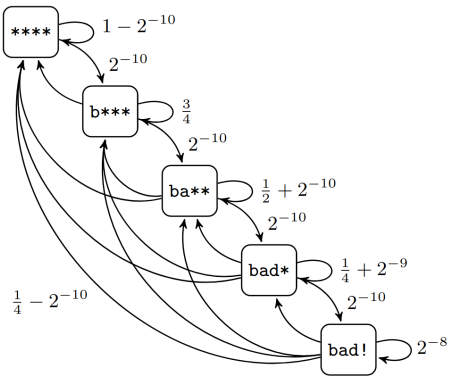


Figure 2.2: A code snippet (left) and its corresponding state-transition diagram for grey-box fuzzing (example from [27]). Transition probabilities are derived from a mutation of a single ASCII character: e.g. 2^{-2} probability of picking a particular character to change and 2^{-8} probability of picking the correct byte.

Figure 2.2 illustrates the advantages of a coverage guided fuzzer over a black-box approach. It shows a program with five execution paths given by nested conditional checks on an input string. Coverage guidance allows a fuzzer to "remember" visited states by saving coverage-improving inputs. In this example, once the grey-box fuzzer has covered the first branch (i.e. `b***`), it can base future test-case creation off of that successful input. Thus at each step, it can prioritize fuzzing a particular state in the diagram on the right of Figure 2.2 to incrementally solve the next condition with roughly the probability of solving that constraint in isolation (discounting the probability of regressing to a prior state). This means that, optimally, a grey-box fuzzer could solve these conditionals with probability approximate to the sum of the individual probabilities ($P \approx 4 * 2^{-10}$). AFLFast, for instance, prioritizes rarely reached states to focus on solving more deeply nested conditionals [27]. In contrast, a black-box fuzzer must solve all of the constraints simultaneously to find the crash, essentially going directly from the top left state in Figure 2.2 to the bottom right state ($P \approx 2^{-10^4} = 2^{-40}$). This limitation makes even simple nested conditions difficult for black-box fuzzers.

2.1.2.2 Input Creation

The other primary axis on which fuzzers are categorized is in how they generate new test cases. These differences manifest in alternative implementations for input creation during the fuzzing process (line 4 in Algorithm 1).

Structured-Input or *Generational* fuzzers create new inputs from scratch, often utilizing a specification or grammar for a particular input format. Structured-input fuzzing is often used in black-box fuzzing, with information about the input format providing some direction within an otherwise opaque search space, as in PeachFuzz [2]. Zest [178] and Nautilus [12] both combine generative fuzzing with grey-box, coverage guided feedback mechanisms, however. More recently, generational fuzzing has seen popularity in "property-based testing" frameworks such as Hypothesis [225] or QuickCheck [190]. Because these property-testing tools operate as unit tests at the source-code level, it can be easy for developers to create (or even automatically derive) generators for complex data-types using the source language and libraries of the system under test. While generational fuzzing has the advantage of more carefully crafted inputs, this can lead to it missing bugs in error handling code for parsing malformed inputs. Additionally, the manual overhead of creating input generators or even an input language specification can be quite large. While machine parseable context-free grammars may be available for some programming languages and textual data formats, these may not encode even simple input invariants such as length constraints on individual fields [212]. Furthermore, many other data formats, especially binary formats, may not have machine parseable or even well-defined input specification at all; the commonly used WebP image format, for example, has a natural language specification which is 57 pages long [1]. In Chapter 6, we explore an analogous approach to structure the space of program schedules, rather than program inputs.

Mutational fuzzing produces test cases by mutating existing inputs to a program, typically starting from an initial corpus of inputs. For mutational fuzzers, this process is typically is split into two parts. First, the fuzzer must schedule an input for mutation from its corpus of interesting testcases. For many fuzzers, such as AFL, a test case is selected for mutations multiple times consecutively, with the number of selections being determined by a *power schedule*. By modifying the power schedule, fuzzers such as AFLFast [27] can prioritize exploring the local input space near a

[1] `https://developers.google.com/speed/webp`

known good test-case via mutations. The second component of input creation for mutational fuzzers is to mutate the selected testcase. MOpt [140], for example, uses particle swarm optimization to choose from a suite of mutation operators during a fuzzing campaign.

Popularized by AFL [9], mutational fuzzing has been remarkably successful, allowing practitioners to effectively fuzz programs which take structured inputs without deep knowledge or accurate specifications needed to comprehensively generate such inputs from scratch. Furthermore, because the inputs are mutated and not generated from a strictly correct grammar, mutational fuzzing is able to discover many bugs in the input parsing code of the program itself, often for mostly correct inputs. However, this means that the initial seed corpus can have a strong effect on mutational fuzzer performance [113] [82].

2.1.2.3 Directed Fuzzing

Directed fuzzing is a class of grey-box fuzzing in which the fuzzer prioritizes exploring behaviors of specific code regions of interest [26]. These areas of interest can vary based on the testing requirements, and could be provided by a human auditor, static analysis tool, or other source. This approach is in contrast to general purpose fuzzers, which are designed to explore *any* new (hopefully buggy) behaviors in the program, regardless of location. Directed fuzzers can be set to target the behavior of changed lines of code, similar to commit and regression fuzzing (defined below). *Commit fuzzing* targets behaviors in locations affected by individual commits. When a new commit is made, inputs are generated specifically to exercise the program locations impacted by the code changes introduced in that commit [243]. *Regression fuzzing* is a specialized process aimed at targeting code that has been recently and frequently modified. It focuses on directing the input generation towards these frequently changed code regions [246]. Unlike commit fuzzing, which targets specific program locations affected by individual commits, regression fuzzing focuses on code regions that are frequently modified across multiple commits. Both commit and regression fuzzing are instances of *change-based fuzzing*, and typically leverage directed fuzzing techniques to efficiently explore the impact of code changes. Directed fuzzing is of particular relevance to Chapter 7 and Chapter 4.

2.2 Concurrency

Concurrent programs are programs that contain multiple threads which can interleave their executions and interact with each other at runtime. Note that we use threads as a general term for unit of concurrency throughout this thesis, but that this unit of concurrency may not correspond precisely to operating system threads. For example, asynchronous Rust programs leverage stackless coroutines to achieve concurrency, but can be analyzed in the same manner as threads. Similarly, distributed systems exhibit concurrency in different processes on different machines, but can be modeled as multiple isolated threads. In this thesis, we are primarily concerned with *shared memory* programs, in which different threads operated on shared synchronization primitives like mutexes within the same address space. *Message passing* is an alternative paradigm for concurrent programs. While this paradigm is not explored explicitly, message passing primitives like channels can be modeled and implemented using lower-level shared-memory primitive types such as batch semaphores [208].

In a concurrent program, each thread executes a sequence of *operations*, often referred to as *events*. These events map to individual instructions as the program executes. For concurrent programs, it is sufficient to consider only *visible* operations, those which can be observed by other threads, to capture the behavior of the program. Examples of visible operations could be acquiring a mutex or writing to a memory location which is shared between threads. Generally, when we refer to operations in this thesis in the context of concurrency, we mean visible operations.

2.2.1 Definition of Concurrency Bugs

We define a concurrency bug as an undesirable behavior that occurs *only* as a consequence of multiple threads interacting. We can categorize concurrency bugs into deadlocks, livelocks, data races and functional bugs. For deadlock bugs, all threads are *blocked* on one or more resources that are unavailable, often because those resources are held by other blocking threads. In the case of livelocks, at least one thread is not blocked, but the overall system still cannot advance. This definition of progress can be difficult to identify and define as it may be program specific. Data races occur when two threads access the same memory location without proper

synchronization. In the context of most programming languages, data races are bugs *regardless* of individual program semantics. For example, in the C and C++ programming languages primarily studied in this thesis, data races are undefined behavior. This categorization allows the compiler to make optimizations that are only correct in the *absence* of data races. However, without explicit crashes or observable side effects, it can be difficult to convince developers that data races alone are worth addressing [106, 163]. Other functional correctness issues come in many different forms, with two of the most common patterns being atomicity violations and order violations. Atomicity violations refer to an (incorrect) assumption that a sequence of operations is executed atomically. Similarly, order violations refer to an (incorrect) assumption that operations take place in a particular order. In both cases, which operations are expected to be atomic or ordered is a characteristic of the program's intended semantics.

Of these bug patterns, data-race and deadlock detection have been the subject of extensive study [61, 56, 148, 193]. However, *functional* bugs such as atomicity violations require a more general analysis approach because they are program specific. In this thesis, we go *beyond* data-race detection, detecting arbitrary assertion failures and memory safety violations that arise from functional concurrency issues in addition to expanding coverage of existing data-race and deadlock detection techniques.

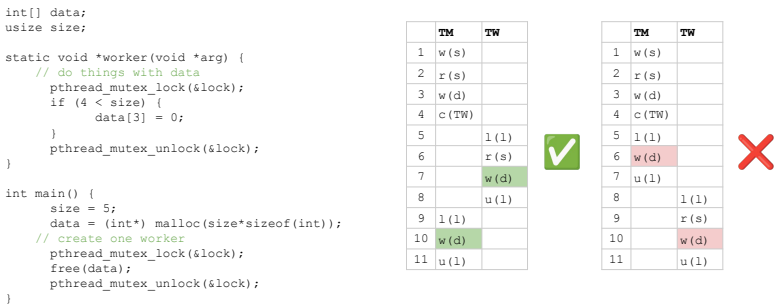


Figure 2.3: A program illustrating a data-race free order violation concurrency bug (left). Two serializations show schedules in which the bug does not (right-middle) and does (right) occur.

Figure 2.3 shows an example of a concurrency bug that we might target using approaches presented in this thesis. If the `free` of data on the main thread happens before the write to data on the worker thread, the program will perform a Use-After-Free. This bug is an example of a memory safety issue *caused* by an order violation; contradicting the assumption about the ordering of these two events causes a global invariant to be broken. Notably, this bug is not detectable by traditional race detectors such as TSAN [202], which only report data-race violations.

2.2.2 Controlled Concurrency Testing

A common approach adopted in practice by software engineers to detect concurrency bugs such as those in Figure 2.3 is *stress testing* software against exceptional workloads. However, stress testing relies *entirely* on the operating system scheduler to produce different interleavings of operations, and these schedulers are fundamentally *not designed* to explore extreme behaviors. Also, by virtue of depending on the operating system scheduler, stress testing is highly *non-deterministic*, making debugging any issues detected difficult.

This thesis expands on the related strategy of *Controlled Concurrency Testing* (CCT) [208, 52, 122]. CCT tools typically transform a parallel, preemptively scheduled program into a fully *serialized* program which is then *cooperatively* scheduled. These tools search the *interleaving* space of events in the serialized program for a *fixed workload* via various scheduling algorithms. While leveraging similar workloads as those used in stress testing, CCT is deterministic, enabling easier debugging and reproducibility once bugs are found. Additionally, because CCT algorithms are *designed* for the exploration of the scheduling space, they can much more effectively explore concurrent behaviors than the operating system scheduling algorithm.

CCT tools must *instrument* the system under test to inject cooperative scheduling points. To ensure that all possible concurrent behaviors can be observed, these scheduling points must be injected *between each visible operation*. Additionally, the CCT framework must enforce the invariant that only one thread is allowed to execute a time. It must also track which threads are blocked, to ensure that the thread selected to run can indeed progress until the next cooperative scheduling point has been reached. We discuss how CCT systems can be implemented at different layers

of the software stack in Chapter 8.

The schedules shown in Figure 2.3 (right) are examples of serializations that would be generated by CCT tools. The first schedule executes all events on the worker once it has been created (#4), before switching back to the main thread (#9). The second schedule executes all events on the main thread before switching to the worker (#8). Each schedule is a *total ordering* of all events, which is constructed incrementally as the program executes by a CCT algorithm.

Operationally, a scheduling algorithm is called at each step in the execution, and selects a thread to run next from the set of runnable threads. For example, at step #5 in Figure 2.3 (right), the algorithm may choose between executing either of the two threads, which may eventually lead to either of the two schedules shown. By utilizing exhaustive search over the schedule space, often leveraging partial-order and other state reduction techniques [118, 189, 161], CCT tools, frequently called *model-checkers*, can verify smaller concurrent programs and data-structures. However, even with reduced state spaces, these approaches still suffer from state explosion [45]. As a result, when it comes to larger concurrent applications, randomized scheduling algorithms, which sample interleavings from the space of all feasible orderings, have been found to generally scale better [199]. These randomized algorithms have seen increased attention in recent years for applying analogous reduction techniques to this stochastic approach to interleaving search [32, 245, 241]. As these randomized CCT algorithms are a form of random search over the scheduling space, as traditional fuzzing techniques are a randomized search over the input space, we refer to them as *concurrency fuzzing* in this thesis.

While the schedules generated by CCT tools are total orders of *all* events in the program, many concurrent events (such as memory accesses to disjoint memory locations) are *commutative* and can be reordered without affecting the program's behavior. As a result, *partial order* methods [70] are frequently used to capture the relations between only a *subset* of events in the full execution trace. Common partial orders include those based on happens-before [148, 61], reads-from, and Mazurkiewicz traces [151]. In our example Figure 2.3, we might say that the buggy schedule for Figure 2.3 is solely captured by the happens-before relation: $\texttt{HB}(\texttt{free}_{main}, \texttt{store}_{worker})$. Both modern exhaustive and modern randomized scheduling algorithms leverage partial order methods to reduce the

search space.

Randomized scheduling algorithms have been extensively studied for CCT, but these algorithms have several shortcomings. First, the algorithms typically are stateless, in that they do not maintain any information across different executions of the program. This prevents them from learning from past executions and adapting their scheduling strategy accordingly. Second, these algorithms often leverage little, if any, information about the semantics of the events being scheduled. The well-known PCT [32] algorithm, for example, uses only an estimate of the total number of events in the program to make scheduling decisions. In terms of fuzzing, this is akin to a black-box approach – no feedback or other analysis of the program's behavior at runtime is used to guide the search. One of the explicit goals of this thesis is to address these shortcomings. We do so by developing randomized algorithms which (1) are semantically aware and (2) leverage adaptive bias a la fuzzing techniques (Chapter 6).

Chapter 3

Motivating User Study

To mitigate the impact of critical bugs reaching production, one approach that has seen increasing attention [228] is to push validation activities towards earlier phases of development, sometimes referred to as a "shift left" [210]. Unit testing [49], for example, has become emblematic of this effort, with developers manually writing tests for smaller units of functionality along with or even before [18] writing the application code itself. Increasingly, these unit tests are run as part of Continuous Integration and Continuous Deployment (CI/CD) workflows to ensure that code that is committed or deployed has always passed these checks. Still, in our surveyed users, many found that such tests can be onerous to write (31/44 participants) and difficult to maintain as software evolves (34/44). Moreover, unit testing (for testing functionality) has been widely employed by practitioners for roughly two decades [196, 157], but security and reliability issues continue to plague the software industry today. Static analysis [90] has also been leveraged to detect bugs earlier in development cycles [44, 53, 102]. However, static analyses typically generate large numbers of false positives, resulting in trust issues among the developer community [185].

Recently, fuzzing [158] has emerged as another effective technique for finding security vulnerabilities and other classes of critical bugs [126, 144]. Unlike static analysis, it has higher precision, providing a reproducing test case for each bug found. Unlike unit testing, it is highly automated, generating test cases without human intervention. However, while much research has been dedicated to improve the bug detection capabilities of fuzzers [154, 153, 183], even expert users face many challenges in using existing fuzzers [167]. Furthermore, little or no research has been conducted on user needs for fuzzers in left-shifted software development workflows

such as automated CI/CD pipelines.

In this chapter, we investigate what would be needed to make fuzzers useful at earlier stages of development for a software practitioner who is not necessarily a fuzzing expert. What are the gaps between these software professionals' expectations and existing fuzzing tools or workflows? What changes would drive broader adoption of fuzzing at early stages in the development process? How do these changes impact the way researchers design and evaluate fuzzers? We examine industry practitioner perspectives on development workflows that utilize fuzzing to provide early assurance during development. We hope these user insights will open new research directions in which fuzzing can be a pathway for shifting automated validation to the left in software development.

Concretely, we first examine user expectations in fuzzing through a survey, aiming to answer the following research questions:

RQ1 What workflows and use cases do practitioners have for fuzzing?

RQ2 What kinds of inputs and setup will users undertake for fuzzers?

RQ3 What kinds of outputs and artifacts do users expect from fuzzers?

RQ4 How do users evaluate the effectiveness of fuzzing tools?

We surveyed 44 industry practitioners with 1-10 years of experience and asked how they envision a fuzzer's role in the development process. Most of our survey participants are highly technical developers who are not necessarily fuzzing experts, with half of them having more than five years of experience in software development.

3.1 Survey Methodology

To address the RQ1-4, we designed an online survey, which we distributed in March 2023 via invitations to contacts from global companies, using the "snowball" method [112]. The participants did not receive any compensation; however, for each participant, we donated $5 SGD to Doctors Without Borders/Médecins Sans Frontières (MSF). Note that before the study, we received the corresponding approval from our organization's Institutional Review Board (IRB).

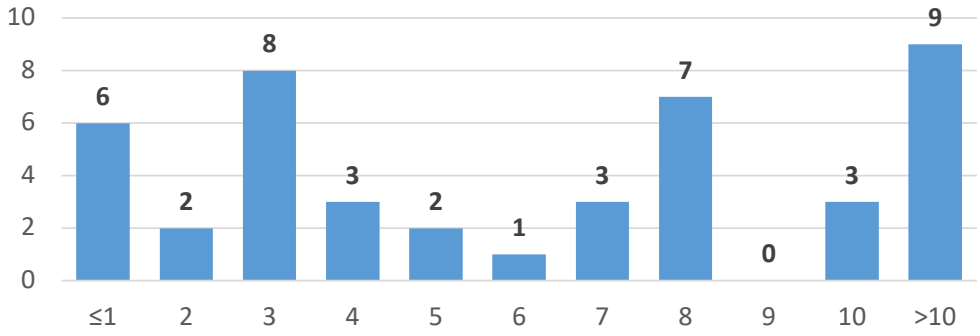


Figure 3.1: Participant's years of experience in their current role.

Survey Overview In total, we asked 47 questions about the *goals* and *usage scenarios* of fuzzing, as well as the relevant *inputs*, *setups*, *interactions* and *outputs*. Prior to the survey questions, all participants were presented with a brief textual definition and description of fuzzing, as well as a link to a short 4-minute video introduction to fuzzing produced by the Communications of the Association for Computing Machinery (CACM) [73]. The survey includes open-ended questions like "If you use fuzzers regularly, please describe your step-by-step workflow(s) when using these tools." and close-ended questions like "At what scale would you conduct fuzzing campaigns?" with Multiple Choice, a 5-point Likert scale, or ranking options. The survey was created and deployed with Microsoft Forms.

Participants In total, we received 51 survey responses from over eleven countries in a wide range of industries, e.g., technology (11), software services and consulting (7), general software development (5), and software security (4). Of the 51 total participants, seven identified themselves as academics or students. To accurately capture the opinions and experiences from *software practitioners*, rather than those of academics, we omit these seven responses from our analysis. Of the remaining participants, 61% (27/44) identified themselves as software developers. 78% (21/27) of these software developers have not used a fuzzer in their professional work setup. Other participants identified themselves as technical leads (6), managers (2) or in other roles like security architect, security engineer, or technical consultant. Figure 3.1 and 3.2 show the practical experience and familiarity with fuzzing of our participants. Across all participants, 52% (23/44) have not used a fuzzer yet. From the remaining participants, only 27% (12/44) indicated that they are expert users

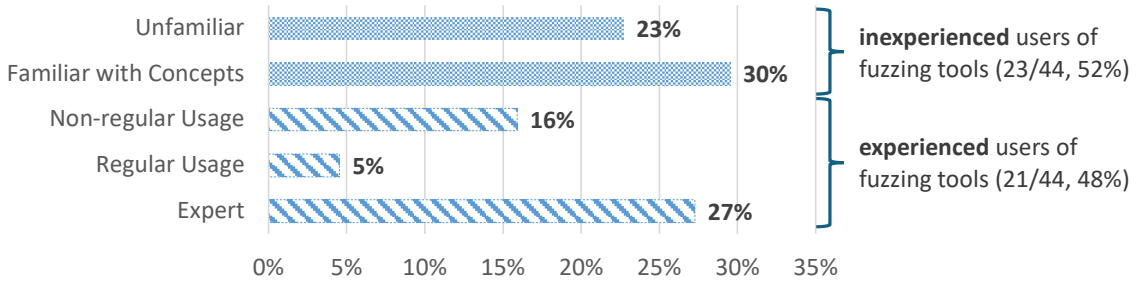


Figure 3.2: Participant's familiarity with fuzzing/testing tools.

of automated software testing tools. However, 77% (34/44) of the participants are at least familiar with the concept of fuzz testing. Most of our participants are not experts in fuzzing, but typically had a large amount of technical experience (7.2 years mean, 6.5 years median). While only 12 participants considered themselves fuzzing experts, 34/44 were at least familiar with fuzzing concepts. The 10 remaining participants who are not at all familiar with fuzzers have a mean of 5.3 years technical experience. Therefore, we believe we have reached an interesting set of participants who can provide insights into expectations and current obstacles for the integration of fuzzing in broader software development workflows. We used the data shown in Figure 3.2 to separate the participants into two groups: experienced (21/44, 48%) who reported at least non-regular usage of fuzzers and inexperienced (23/44, 52%) practitioners. In our discussion, we highlight differences between these two groups, if they exist. For the remainder of this chapter, when we say "experienced" or "inexperienced" developers, we refer to their experience with fuzzing/testing tools.

Analysis For all questions with a 5-point Likert scale, we analyzed the distribution of negative/disagreement (1 and 2), neutral (3), and positive/agreement (4 and 5) responses. For the ranking questions, we analyzed which answers have been ranked highest. For the Multiple Choice questions, we analyzed which choices were selected most; further, we analyzed the open-ended "Other" choices and mapped them to the existing options or treated them as new ones if necessary. For all other open-ended questions, we performed a qualitative content analysis coding [197] to summarize the themes and opinions. The first iteration of the analysis and coding was done by one author, followed by the other authors' review. The following sections summarize and discuss the most mentioned responses indicating their frequency in brackets. To

compare the two groups of experienced and inexperienced developers in the context of fuzzing, we use the chi-square test of independence [180] ($\alpha = 0.01$). The chi-square test of independence tests whether the two categorical variables are related. For example, a) the experience of developers in using fuzzing and b) their preferred (local/CI/on-demand) fuzzing campaign length. In particular, we test whether there is any statistically significant difference in their general preferences. We also test the significance of the obtained trends/majorities with the Binomial Test [54] ($\alpha = 0.05$), e.g., is an observed majority of preferred usage actually significant in relation to the total number of participants. We present the corresponding P values. Our research artifact includes all data, statistics, and codes:

`https://doi.org/10.6084/m9.figshare.24769719.v1`

3.2 Survey Results

3.2.1 Workflows and Use Cases (RQ1)

The first section of our survey focused on high-level goals and workflows, to understand *how developers envision using fuzzing* in their day-to-day development process.

Testing Goals To determine which kinds of bugs and test-oracles fuzzing researchers and practitioners should focus on, we asked our participants *"What properties are important to test your software for?"* (Figure 3.3). Classes of bugs commonly targeted by existing fuzzers and sanitizers, such as memory safety bugs, are deemed important by a strong majority of experienced and inexperienced developers. Functional correctness, however, was the most important property identified by nearly all of our participants (39/44, 89%, $P < .001$). Participants mentioned memory safety as the second most important property, followed by concurrency safety. Many other non-functional properties, such as execution speed, memory consumption and side-channel vulnerabilities were also deemed important by more than one-third of our participants. Contrary to the goals of our participants, in general, fuzzer and sanitizer support for finding non-crashing bugs is often limited, if

it exists at all. The commonly used LLVM sanitizers, for example, can not identify liveness, performance, or side-channel vulnerabilities at all, and only find a subset of concurrency bugs (data-races and deadlocks) at the cost of extremely high memory and execution time overhead (often over 10x).

Many diverse fuzz-testing oracles, especially for ***functional correctness and concurrency****, are considered important by developers; better support for detecting these bug types may boost adoption of fuzzing tools.*

Concrete Workflows CI/CD has emerged as a key process in modern software development. We asked our participants whether they would include fuzzing in their CI/CD system and also *"How frequently would you run a fuzzer as part of your build integration or deployment system?"* to understand how fuzzing can be used in this context. We found that a large majority of our participants (33/44, 75%, $P < .001$) would add fuzzing to their CI/CD pipelines, predominantly as either per commit or in nightly administered fuzzing runs. We could not determine any significant difference between the responses of experienced and inexperienced participants for this topic.

We also queried about integrating fuzzing into the participants' local development workflows. Perhaps surprisingly, most participants (28/44, 64%, $P < .05$) would include fuzzing in their local development workflow. These practitioners would mostly run a fuzzer only as a local pre-commit or pre-merge step (19/28, 68%, $P < .05$), rather than more frequently (e.g., on each file-edit). Some experienced practitioners indicated that they would also periodically run a fuzzer multiple times before committing to the CI pipeline, while *no* inexperienced developers indicated they would do so (experienced: 5/21, 24%; inexperienced: 0/23, 0%; $P = .013 > .01$).

A majority of developers want to be able to run fuzzers as part of their local development workflow.

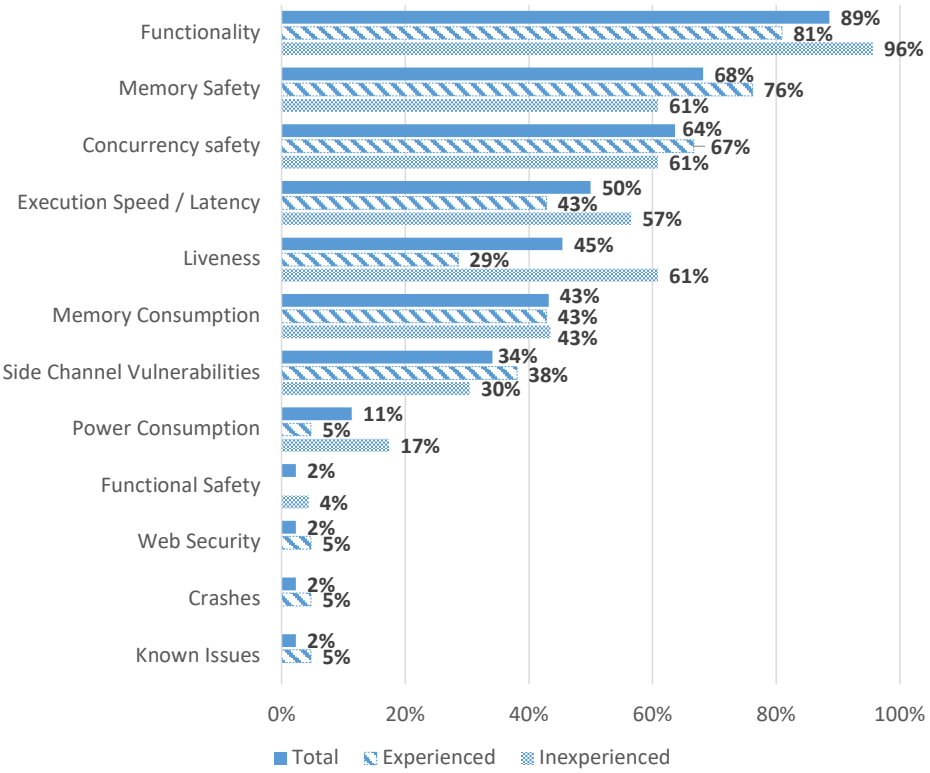


Figure 3.3: Participants' opinions about the most important testing properties.

3.2.2 Fuzzer Setup and Inputs (RQ2)

The next section of our survey investigated developers' tolerance for manual steps to start using a fuzzer, to better establish which steps should be automated or otherwise avoided in fuzzing workflows.

Fuzzer Setup To understand users' tolerances to various common set-up tasks for fuzzing, we asked our participants *"What level of setup are you willing to do (if any) to configure a fuzzer on a given target?"* (Figure 3.4). While a small number of participants would only use a nearly completely automated, "push-button" fuzzer, most (38/44, $P < .001$) would be willing to do some amount of manual configuration. For the various tasks associated with the program under test –modifying the build-process, writing a fuzzing harness, providing initial test cases, or specifying an input grammar– a similar proportion of roughly 40% of users are willing to undertake them. Far fewer would consider modifying the fuzzer to conform to a particular target program. We note that this dichotomy may contrast with the fuzzing research

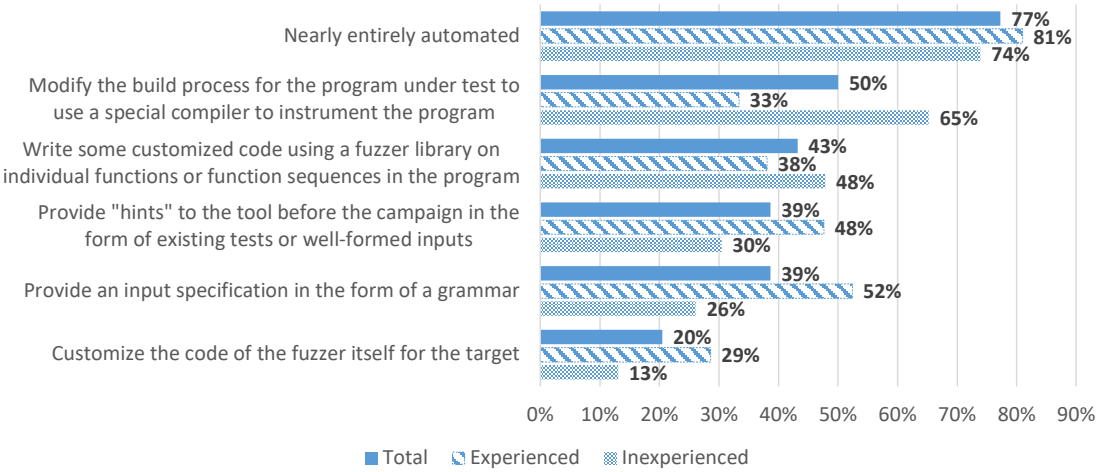


Figure 3.4: Participants' willingness to configure and setup the fuzzer.

community, who are likely more familiar with the fuzzing tools than the programs being tested. While, in general, we could not determine a statistically significant difference between the experienced and inexperienced participants for this topic, we still would like to highlight two aspects: Firstly, while most inexperienced practitioners (15/23, 65%, $P < .05$) would be willing to modify the build process, only a few experienced practitioners (7/21, 33%) would do the same (difference across groups: $P = .035 > \alpha = .01$). Secondly, most inexperienced participants (17/23, 74%, $P < .01$) would not be willing to provide any input specification; however, the experienced participants were more divided on this question, with 11/21 being positive and 10/21 being negative about providing an input specification (difference across groups: $P = .074 > .alpha = 01$).

Developers desire highly automated workflows for fuzzing.

3.2.3 Fuzzer Outputs and Artifacts (RQ3)

This section of our survey examined developer expectations for the outputs of a fuzzing campaign, which can help tool builders understand which output artifacts are most useful for their users.

Usefulness of Bug Reports After showing participants examples of fuzzer-generated bug reports, we asked *"How useful do you think each feature of a fuzzer-*

generated bug report would be?". Unsurprisingly, most participants (36/44, 82%, $P<.001$) mention it is very useful to have the identified assertion violation or crash message, including a stack trace. A large majority also indicated that suggested fault and fix locations (32/44, 73%, $P<.001$) or a proposed patch (31/44, 70%, $P<.01$) for the bug would be beneficial. Interestingly, the responses from experienced and inexperienced practitioners were mainly consistent, indicating the relevance of the obtained feedback about fuzzer-generated bug reports for general software development.

Additional artifacts such as suggested fix locations or proposed patches are more desirable than a reproducing test case for automated bug reports.

Outputs Participants mentioned the integration of fuzzers into their existing development environments, pipelines, and tools. For example, automated issue reporting with tools like Sonarqube and Jira. Others mentioned ideas to further improve the information in bug reports by adding the execution trace with the state values of the reproducing test case, the relevant bytes in the crashing input that differentiate (w.r.t. the execution behavior) from other similar but non-crashing inputs, runtime profiling (memory usage and timing information) of each function that is executed along with the reproducing test case, and any additional information the system under test produced as part of the execution (logs, error messages, etc.).

3.2.4 Fuzzer Evaluations (RQ4)

Developer expectations also provide important to establish evaluation criteria for fuzzing research, which is of great interest to the fuzzing research community [23, 126].

Evaluation Configurations Generally, fuzzer evaluation configurations should mirror real-world use cases, such as those identified in Section 3.2.1. We queried participants about the expected fuzz campaign lengths for three fuzzing workflows: on-demand audits, CI/CD fuzzing, and fuzzing in the local development cycle. Figure 3.5 shows their aggregate responses. While we observed that experienced practitioners could be willing to wait a bit longer, our analysis could not determine

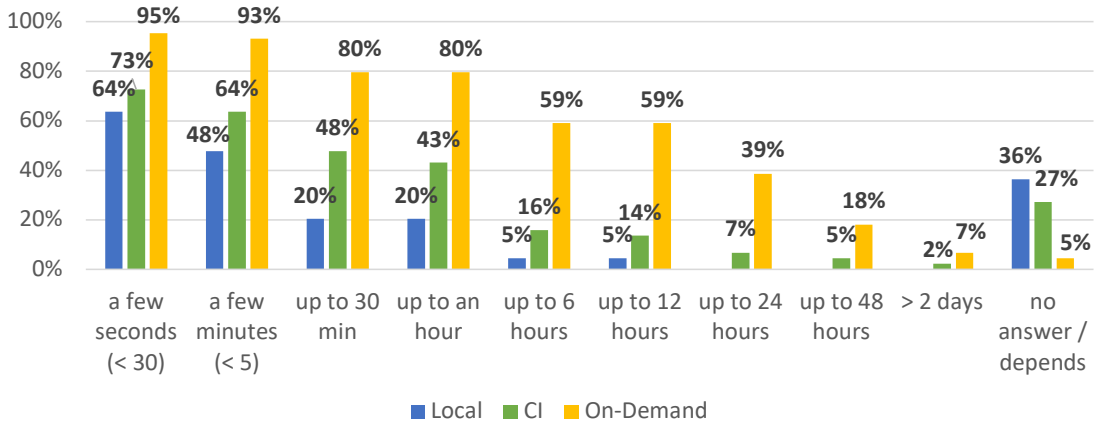


Figure 3.5: Cumulative presentation of the expected fuzzing campaign lengths for the different workflow types.

a significant difference between the two groups. Therefore Figure 3.5 shows the responses over all 44 participants. For an on-demand fuzzing campaign, 39% (17/44) of all participants would wait up to 24 hours, with a sharp drop to only 18% of participants willing to wait 48 hours or more. There are similarly sharp declines from 1 hour to 6 hours, and between 12 and 24 hours. Still, the general recommendation of 24 hours for fuzzing campaigns appears to be in-line with developer expectation for on-demand fuzzing campaigns.

In contrast, participants prefer shorter time bounds for CI fuzzing. Only a few of the participants (7/44, 16%) were willing to run a CI fuzzing job for more than one hour. During local development, even fewer of the participants would wait longer than an hour for a fuzzing run, while most would not wait more than five minutes.

For the strongly desired local and CI/CD fuzzing, the expected timeouts are substantially shorter than the usual evaluation timeouts suggested in the fuzzing literature [113]. This inspired us to perform follow-up experimentation to assess current state-of-the-art fuzzers under these new expectations (see Section 4.1).

To meet developer expectations, evaluations of fuzzers should timeout at:

- *5 minutes, for local development*
- *1 hour, for CI/CD fuzzing*
- *24 hours, for on-demand audits*

3.2.5 Summary

We summarize our findings from this user survey in the table below:

4.1: Workflows and Use Cases	
Types of Bugs	Diverse fuzz-testing oracles are desired, especially for functional correctness and concurrency bugs
Testing Goals	Regression-test generation, bug discovery, and directed testing are equally important
Workflows	Local Development and CI/CD workflows are desired for fuzzing
Obstacles	UI/UX is the greatest obstacle to adoption of fuzzing
4.2: Setup and Inputs	
Setup	Developers are somewhat willing to tolerate setup costs for fuzzing
4.3: Outputs and Artifacts	
Report Artifacts	Fault localization and proposed patches are more important than a reproducing test case
Other Outputs	Developers mentioned integrations with issue-tracking platforms as desired
4.4: Evaluations	
Evaluation Configurations	We identify empirical thresholds for fuzzing cutoff times that meet developer expectations

3.3 Threats to Validity

Construct Experimenter bias is always a threat to the validity of personal opinion surveys. We attempted to phrase all of our questions in neutral language, but it is possible that aspects of our survey instrument may have biased the results. To avoid observer bias, we designed the survey to collect no personally identifying information, giving our participants a trusted platform.

Internal To ensure the clarity of our questions and the form's structure, we performed small-scale test runs with researchers who were not involved in the study. However, some of our questions are not phrased completely neutrally, and this could have affected participants responses.

The overarching goal of our study was to assess the expectations of software professionals for a usable fuzzing workflow; to ensure our conclusions reflect this, we blocked our data on fuzzing experience and discarded responses from academic researchers. We choose to include responses from those unfamiliar with fuzzing because we are explicitly interested in how these users see themselves using a fuzzing tool. While there is a risk that this population are not able to accurately answer questions, our target population is nonetheless highly technical (Section 3.1) and we provided a general summary of fuzzing concepts at the beginning of our survey form to ensure our participants had the requisite background information.

Conclusion To reliably interpret all responses, particularly the open-text responses, we applied qualitative analysis coding with an agreement of at least two authors on all codes. For questions measuring agreement, we used the chi-square test of independence [180] and the binomial test [54], both well-established statistical tests in the realm of user surveys [83, 78, 95, 222, 128, 165].

External A potential threat to external validity is the limited sample size and scope of participants in our survey. By collecting responses using the snowball method (see [112]) from 44 participants across 23 companies and 11 countries, which we see as a sufficiently diverse set of participants, but we cannot guarantee the generalization of our results. We believe that we were able to reach an important sample cohort because they mostly identify as highly technical software developers who have only limited experience with fuzzing while being familiar with the concept (Section 3.1).

3.4 Discussion

Our user-survey in this chapter highlights the importance of integrating fuzzers into new and existing developer workflows. From this survey, we found that developers want fuzzers to trigger automatically on local commits and pull requests (Section 3.2.1), nearly fully automated to apply (Section 3.2.2), and provide output that provides rich debugging artifacts as well as integrating with existing tooling (Section 3.2.3). We also saw that developers are interested in finding functional correctness and concurrency bugs with fuzzers and that lack of support for additional kinds of programs and bugs is a significant barrier to adoption (Section 3.2.1). Fuzz testing has conventionally been used for finding security vulnerabilities in well-established, sequential software. The role of fuzz testing during software development has been relatively less examined. In this thesis, we refer to this as a *vertical expansion* through layers of the software development process. Similarly, the role of fuzz testing in domains beyond sequential inputs is likewise under-explored. We refer to this as a *horizontal expansion* through domains of software. The remained of this thesis is dedicated to expanding these two dimensions of fuzz testing.

Part I

Vertical Expansion to Developer Workflows

Chapter 4

Change-Based Fuzzing

One of the strongest results of our survey in Chapter 3 is that developers are interested in incorporating fuzzing into their local development workflows and CI/CD pipelines. Broadly, we categorize both of these workflows as *change-based* fuzzing, where fuzzers are invoked automatically to test changes to a codebase as it is being developed. By integrating fuzzers continuously into the development process, developers can catch bugs earlier in the development cycle, before they reach production. However, this usage of fuzzers contrasts sharply with how they are typically evaluated and deployed in practice. Instead of longer often 24-hour campaigns, developers want change-based fuzzing workflows to *quickly* identify bugs within a matter of minutes. Instead of broadly exploring all behavior of a system, change-based fuzzing is concerned only with behaviors arising from the code which was changed. This chapter aims to understand how different state-of-the-art fuzzers fare in change-based workflows.

Specifically, from the results of our survey in Chapter 3, we formulate and answer two additional questions to examine the feasibility of one highly desired development workflow with existing fuzzing tools:

RQ1 How effective are existing fuzzers at testing code-changes in a software project?

RQ2 How do additional variables affect fuzzer effectiveness in this scenario?

For these questions, we analyze the current capabilities of state-of-the-art fuzzers' in continuous testing. We compare the efficacy of these tools with respect to developer expectations and desired workflows. In total, we evaluate two general

purpose fuzzers (AFL[1] and AFL++ by [59]), two commit/regression fuzzers (AFLChurn by [246], and CIDFuzz by [243]) and two directed fuzzers (AFLGo by [26] and SelectFuzz by [138]). Our results indicate that state-of-the-art fuzzers can find roughly 50%-60% of the regression bugs in our data set within a time period desired for local development or CI/CD workflows, respectively. Additionally, we find that the context of the fuzzing campaign: how long a fuzzer takes to instrument and analyze the code of a target program, and the initial test cases given to the fuzzer both have a *significant* impact on fuzzer effectiveness in this scenario. These results show that, while existing tools can already provide a substantial benefit in this context, more research is needed to find all bugs within the shorter time frames desired by developers.

4.1 Evaluation of Fuzzers in a Commit Fuzzing Scenario

As continuous local and CI/CD fuzzing were both strongly desired workflows for our survey participants, we conducted an additional quantitative analysis to assess the capabilities of fuzzing tools in this context.

4.1.1 Experiment Setup

Fuzzers For our evaluation, after a literature review in total we select six state-of-the-art fuzzers that met our criteria representing three different avenues explored in the literature, namely commit fuzzing (CIDFuzz by [243]), regression fuzzing (AFLChurn by [246]), directed fuzzing (AFLGo by [26] and SelectFuzz by [138]), and general fuzzing (AFL and AFL++ by [59]). Our selection criteria for directed, commit and regression fuzzers was that they (1) were compatible and able to run in Fuzzbench [156] infrastructure (2) that the code for the fuzzer was openly available and (3) that it was compatible with both C and C++ programs. We included all such fuzzers we identified that met this criteria. We selected AFL and AFL++ as the baseline comparisons for our evaluation as state-of-the-art general-purpose fuzzers because all fuzzers in our evaluation are based on the AFL fuzzing algorithm and

[1] `https://github.com/google/AFL`

	Program →	yara	libtiff	proj4	zstd	libxml2	file	haproxy	libvpx	usrsctp	openvswitch
	Issue # →	48329	58729	49256	57086	57469	59438	52049	48816	47712	47112
Fuzzer ↓											
AFL	t_{build} (s)	18.65	42.50	110.36	107.21	22.12	28.35	84.19	7.48	12.36	31.36
	P_{5m}-sat.	0.00	1.00	0.00	0.00	0.00	1.00	0.95	0.00	1.00	0.60
	P_{1h}-sat.	0.00	1.00	0.00	0.25	0.00	1.00	1.00	0.00	1.00	1.00
	P_{5m}-static	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.00	1.00	0.05
	P_{1h}-static	0.00	1.00	0.00	0.05	0.00	1.00	1.00	0.00	1.00	0.55
AFLchurn	t_{build} (s)	36.86	58.72	119.25	142.16	53.50	73.73	1168.09	12.03	-	150.93
	P_{5m}-sat.	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.00	-	0.90
	P_{1h}-sat.	0.00	1.00	0.00	0.20	0.00	1.00	1.00	0.00	-	1.00
	P_{5m}-static	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.00	-	0.05
	P_{1h}-static	0.00	1.00	0.00	0.00	0.00	1.00	1.00	0.00	-	0.40
AFLgo	t_{build} (s)	80.97	127.14	487.35	-	73.07	57.35	198.41	34.98	-	108.66
	P_{5m}-sat.	0.00	1.00	0.00	-	0.00	1.00	0.85	0.00	-	0.90
	P_{1h}-sat.	0.00	1.00	0.00	-	0.00	1.00	1.00	0.00	-	1.00
	P_{5m}-static	0.00	1.00	0.00	-	0.00	1.00	0.00	0.00	-	0.00
	P_{1h}-static	0.00	1.00	0.00	-	0.00	1.00	1.00	0.00	-	0.45
AFL++	t_{build} (s)	101.20	104.93	241.39	216.15	53.33	65.95	194.88	15.46	30.67	70.17
	P_{5m}-sat.	0.00	1.00	0.00	0.00	0.00	1.00	0.25	0.00	1.00	0.40
	P_{1h}-sat.	0.00	1.00	0.00	0.05	0.00	1.00	1.00	0.00	1.00	1.00
	P_{5m}-static	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.00	1.00	0.05
	P_{1h}-static	0.00	1.00	0.00	0.05	0.00	1.00	1.00	0.00	1.00	0.35
CIDfuzz	t_{build} (s)	-	-	-	-	923.58	80.65	-	-	-	-
	P_{5m}-sat.	-	-	-	-	0.00	1.00	-	-	-	-
	P_{1h}-sat.	-	-	-	-	0.00	1.00	-	-	-	-
	P_{5m}-static	-	-	-	-	0.00	1.00	-	-	-	-
	P_{1h}-static	-	-	-	-	0.00	1.00	-	-	-	-
Selectfuzz	t_{build} (s)	107.06	952.77	2670.94	-	133.40	67.30	219.56	102.30	-	207.92
	P_{5m}-sat.	0.00	0.00	0.00	-	0.00	1.00	0.10	0.00	-	0.55
	P_{1h}-sat.	0.00	1.00	0.00	-	0.00	1.00	1.00	0.00	-	1.00
	P_{5m}-static	0.00	0.00	0.00	-	0.00	1.00	0.00	0.00	-	0.05
	P_{1h}-static	0.00	1.00	0.00	-	0.00	1.00	1.00	0.00	-	0.30

Table 4.1: Probability of Bug Discovery for Commit-level Fuzzing of Open Source Programs with Time Limit *including build times* of 5 minutes (P_{5m}) and 1 hour (P_{1h}). Results shown for using a saturated (-sat.) or static (-static) corpus.

implementation. We compare against AFLGo and SelectFuzz, which attempt to generate test inputs that reach target locations in a program. For the program locations, we use the changed lines in the buggy commit as targets. Finally, we choose AFLChurn and CIDFuzz as regression fuzzers specifically designed to detect bugs in a commit of a software program. We configured each fuzzer to use default parameters in our benchmarking setup according to its respective documentation or openly available example configurations in a best-effort manner. Based on our survey results in Sections 3.2.1 and 3.2.2, fuzzer usability is the number one obstacle for software professionals to use fuzzers and the majority of developers are not willing to extensively customize fuzzer configurations to their systems; thus we believe this setup reflects how practitioners would utilize these tools in their own workflows.

Subject Programs We randomly selected 10 bugs from OSS-Fuzz [204] that are (1) labeled as reproducible, (2) were able to be built and run in the FuzzBench infrastructure, and (3) where we were manually able to reproduce the bug and identify the bug-introducing commit. We sampled bugs starting from the front page of the bug tracker[2] at the time of experimentation, up to one regression bug per program that met criteria (1), (2), and (3), moving to the subsequent page if no bugs met our criteria. We sampled up to 10 bugs, with no more than one bug per project. As a result, all our subjects are C/C++ programs from open source projects integrated with the OSS-Fuzz infrastructure.

Fuzzing Setup For our evaluation setup, we seek to emulate frequent, cumulative fuzzing on a given target in response to code changes, as in a CI/CD pipeline or in a local development workflow. Thus, we create a fuzzer-generated corpus on the program under test *before* the bug-introducing commit which we call the *saturated* corpus. This saturated corpus emulates a fuzzer-created test suite built up over many iterations of successive, change-based fuzzing campaigns during the lifetime of a project. To generate the saturated corpus, we fuzzed the pre-bug commit version of each program continuously for two weeks each with AFL and AFL++. We then aggregated all test cases saved by the two-week campaigns and used `afl-cmin`[3] to minimize them with respect to edge coverage. `afl-cmin` is a utility program that greedily constructs a subset of test cases to achieve the total coverage of all test cases. In our evaluation of fuzzers, we use this minimized corpus as the initial seeds for each program.

As an additional baseline, we also evaluated each fuzzer with a corpus comprised *only* of test cases provided in the project source repository. For each project, we used the OSS-fuzz configuration and project documentation to locate these test cases, taking only the test cases which were available prior to the bug introducing commit. We call this the *static* corpus.

Data Availability Our research artifact includes initial corpora, data, software (with pinned versions), and documentation necessary to reproduce our experiments:

[2] `https://issues.oss-fuzz.com/issues`
[3] `https://github.com/google/AFL/blob/master/afl-cmin`

https://doi.org/10.6084/m9.figshare.24769719.v1

Hardware All experiments were conducted using the standard benchmarking service FuzzBench [156] via Docker containers without disabling Spectre mitigations on a Intel(R) Xeon(R) Gold 6258R CPU @ 2.70GHz (Model 85) with 112 logical cores and 188 GB of RAM.

Evaluation Metrics We assess both the efficacy and efficiency of the fuzzers in finding regression errors. We manually analyzed the crash signatures recorded by Fuzzbench and determined if they corresponded to the vulnerability in question. Three of the authors independently checked the crash signatures to determine whether they corresponded to the vulnerability in question.

To analyze the results we use *survival analysis* because our bug-finding trials are *right-censored* data. Specifically, we use the Kaplan-Meier [105] method. For some additional claims of statistical significance, we use Wald's t-test [223] on the bootstrapped Restricted Mean Survival Time (RMST) of the fitted Kaplan-Meier models. The RMST captures the area under the curve of a survival plot, thus a *higher* RMST indicates that a fuzzer is *less effective* because the bug "survives" longer.

Results Table Description We collect results for each of the fuzzers over 20 repetitions and analyzed the efficacy of finding the target regression error. Table 4.1 summarizes the efficacy of each fuzzer for timeouts of 5 and 60 minutes (see Section 3.2.4) and the build time for each project. For each sub-column, P_{5m} and P_{1h} represent the probability of the fuzzer detecting the target regression errors in twenty trials for time duration of 5 minutes and 60 minutes, respectively. P_x-static represents these probabilities when using the static corpus, rather than the saturated corpus (P_x-sat.). Sub-column t_{build} depicts the average time in seconds needed for building the subject for each fuzzer over 20 repetitions. Note that t_{build} –which captures the time taken for each fuzzer's instrumentation and/or other initial program analyses– is *included* in the computation of P_{5m} and P_{1h}. In other words, a trial is considered successful if the fuzzer can both build, instrument and analyze

the project *and* find the bug within the target timeframe.

4.1.2 Effectiveness of Existing Fuzzers (RQ1)

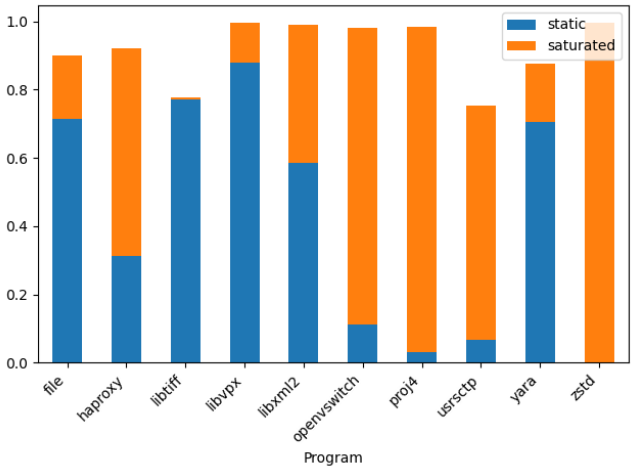
Of the ten bugs in our evaluation, four bugs –in `libvpx`, `proj4`, `yara` and `libxml2`– were not found by any of our evaluated fuzzers in the one-hour time limit.[4] However, out of the remaining six programs, we find that the static corpora *or* a saturated corpus of test cases generated by fuzzing *earlier* versions of the program was able to trigger the bug in three programs: `file`, `usrsctp` and `libtiff`. These bugs were triggered by existing inputs without *any* additional fuzzing on the bug-introducing commit itself.

With respect to our identified thresholds of *5 minutes* for local fuzzing and *1 hour* for CI/CD fuzzing, we see mixed results when using a saturated corpus. Table 4.1 shows the discovery rates for each fuzzer on each bug at each duration. Here, we observe that two of the bugs are discovered at least once within the first five minutes. Mostly, the probability of finding these bugs within 5 minutes is relatively high as well (e.g., geomean of 0.75 for AFL). At an hour, prospects improve somewhat for finding the bug in `zstd`, but still, no fuzzer can find the bug in the majority of trials. For the other programs in which bugs can be found, increasing the fuzzing campaign up to one hour nearly ensures the bug will be discovered.

Existing fuzzers can already have a substantial impact if integrated into change-based developer workflows, finding half of all bugs at least once within expected time frames for local development (5/10) and CI/CD fuzzing (6/10). Future research in will be needed to find the remaining bugs within these shorter time limits more consistently.

Comparison of Fuzzers We show the [105] fitted survival plot for all programs where there are differences in effectiveness across fuzzers in Figure 4.2**.(middle)**. The survival plots show the probability that a bug has *not* been found by the corresponding fuzzer over time. Across these programs, we don't see any trend in

[4] In preliminary testing, some fuzzers were able to find the bugs in `libxml2` and `yara` within one hour with very low probability using the saturated corpus, but this did not occur in our final evaluation with 20 trials.



Program	# Test Cases	
	Saturated	Static
file	386	74
grok	442	36
haproxy	87	40
libtiff	940	7367
libvpx	155	161
libxml2	377	40
openvswitch	2249	24
proj4	1555	133
usrsctp	25	24
yara	121	7
zstd	782	24223

Figure 4.1: **(Left)** Initial Corpus Coverage Normalized to Maximum Observed Coverage per Program and **(Right)** Initial Corpus Size (Number of Test Cases)

terms of fuzzer superiority; no one fuzzer appears to be significantly better or worse than the others at finding this regression bug. We do observe that AFL consistently beats AFL++, despite the latter being a re-implementation of the former with additional features ($RMST_{5m}$: 226.85 $AFL{+}{+} \rightarrow$ 208.34 AFL, $p < 0.001$ and $RMST_{1h}$: 2099.02 $AFL{+}{+} \rightarrow$ 2026.42 AFL, $p < 0.001$). This is unusual, as AFL++ is generally meant to be an improved implementation of AFL, but not unheard of; AFL also beats AFL++ occasionally in public benchmarking results[5]. We could not determine clearly that directed or commit/regression fuzzers perform better in this context relative to the general-purpose fuzzers. This result is surprising, as directed and commit/regression fuzzers are optimized for reaching particular code target locations quickly. Unfortunately, all of the directed and regression fuzzers are incompatible with one or more of the programs in our evaluation set, especially in the case of CIDFuzz.[6] However, despite CIDFuzz compatibility issues, it does attain the statistically significant highest *code* coverage for the two programs it can successfully fuzz.

[5] `https://www.fuzzbench.com/reports/2021-06-02/index.html#mbedtls_fuzz_dtlsclient-summary`

[6] CIDFuzz's LLVM instrumentation pass causes segmentation faults on most programs. The build process for `zstd` is complex, and would need to be heavily modified to incorporate most AFLGo-based fuzzers' instrumentation steps. Similarly, `usrsctp` has a library versioning conflict with several AFLGo-based fuzzers, which results in a linker error on instrumentation.

4.1.3 Impact of Additional Variables on Fuzzer Effectiveness (RQ2)

Comparison of Corpora For this experiment we used two different starting corpora. One of which is saturated in that it is distilled from the historical corpus of OSS-Fuzz runs and thus is expected to be high coverage. In contrast, the static corpus consisted of existing test cases extracted from each target program's repository. Figure 4.1**.(left)** shows the LLVM edge coverage of each corpus, normalized to the maximum coverage observed during fuzzing.[7] Here we can see that the saturated corpus is indeed, very high coverage in all cases. However, the static corpus coverage varies widely, with some examples less than 10% of the maximum observed coverage and other examples having nearly the same coverage as the saturated corpus. Figure 4.1**.(right)**, we can see the distribution of corpus sizes across programs. Here again, we see a wide variance across projects, with some projects like `libvpx` having more test cases than the saturated corpus, despite having lower initial coverage.

Looking at Figure 4.2**.(top)** and **.(middle)**, we can compare the bug-finding efficiency of our fuzzers when using each corpora. Here we see that fuzzers using either corpus are able to find all three bugs in at least some cases, but, generally, the bugs are found more quickly and reliably with the saturated corpus. For example, on `openvswitch`, all fuzzers find the bug within one hour when using the saturated corpus, but none of the fuzzers can find the bug in the majority of trials when the static corpus is used instead. Similarly, both AFL and AFLGo are able to find the bug in `haproxy` within 5 minutes in the majority of trials with the saturated corpus, but *never* find the bug in less than 5 minutes using the saturated corpus. Across all benchmarks, there is a significant decrease in effectiveness when the static corpus is used instead of the saturated corpus ($RMST_{5m}$: 239.13 *static* $\rightarrow$ 206.80 *saturated*, $p < 0.001$ and $RMST_{1h}$: 2324.71 *static* $\rightarrow$ 1940.65 *saturated*, $p < 0.001$).

[7] Note that for `zstd` the LLVM coverage calculator failed to generate a coverage report, likely due to its excessively large size. Thus we are missing the initial coverage for the static corpus of this target program.

A high coverage corpus generated from cumulative fuzzing is generally more effective at finding new bugs that the static test cases included within source repositories for OSS-Fuzz. Practitioners should consider using an updated, cumulative corpus for CI/CD fuzzing. Future research in test case selection for local development and CI/CD fuzzing, such as [37], could improve fuzzer efficacy.

Build Times An often overlooked aspect in research papers is the *build time* overhead of a given tool. This number includes the time it takes a fuzzer to analyze and instrument a target program *before* it starts fuzzing. In the context of 24-hour fuzzing campaigns, this additional instrumentation and analysis time may be negligible. However, for local development or CI/CD fuzzing, with the total campaign length being only a few minutes to an hour, the additional build times can be substantial. Indeed, looking at the columns t_{build} in Table 4.1, we can see a large difference between fuzzers. Selectfuzz takes *ten times* the build time of AFL on average! Even AFL++ or AFLGo takes 80 to 90 seconds longer than AFL to instrument programs on average; in the context of a local development fuzz campaign, this is equivalent to more than 20% of the total time budget.

We can see the impact on fuzzer effectiveness for our identified timing thresholds of 5 minutes and 1 hour in Figure 4.2. Comparing the **(middle)** and **(bottom)** rows, we can see the difference in survival plots for the saturated corpus are included and excluded, respectively. Here we can see a significant visual difference for some fuzzers: AFLChurn goes from being among the worst fuzzers on the `haproxy` benchmark program with build times included **(middle)**, to being among the best if we ignore build times **(bottom)**. Indeed overall, we see a statistically significant drop in effectiveness across all fuzzers and benchmarks ($RMST_{5m}$: 234.31 $excluded \rightarrow$ 200.12 $included, p < 0.001$ and $RMST_{1h}$: 2154.33 $included \rightarrow$ 2076.55 $excluded, p < 0.001$).

Build-time overhead introduced by fuzzers can have a significant impact in shorter campaigns; regression fuzzer developers should emphasize approaches which do not drastically increase build times for real-world software projects.

4.2 Threats to Validity

Construct To ensure construct validity, we directly measure our criteria for fuzzer effectiveness –ability to find bugs associated with a code change– rather than using a proxy metric, such as code coverage. Because crashes from different bugs can look similar, three of the authors examined the crash signatures for each crash detected during fuzzing to determine whether they corresponded to the targeted bug.

Internal To avoid selection bias in our choice of benchmarks, we chose them randomly from real-world bugs in the OSS-Fuzz [204] bug tracker. More specifically, we sampled randomly from the front page of the OSS-Fuzz bug tracker at the date of experimentation, filtering for confirmed and reproducible bugs. It is possible that there is a temporal bias, as our bugs all come from a similar timeframe, but we sampled only up to one bug per project to minimize this effect.

We attempted to avoid selection bias in our choice of fuzzers by using all openly available directed, commit and regression fuzzers that we were able to make compatible with the Fuzzbench benchmark suite. To identify these fuzzers, we searched for these keywords (along with fuzzing/fuzzer) in research databases such as Google Scholar and DBLP. We used all fuzzers we identified that met our criteria. We chose AFL and AFL++ as baselines to reduce the risk of cross-implementation differences in fuzzers, rather than fundamental algorithmic differences (e.g. directed vs. undirected fuzzing); all of the evaluated fuzzers' implementations are modifications or extensions of the AFL fuzzer.

Another possible threat to internal threat to validity is three of the bugs were originally discovered by AFL, which might give AFL an advantage in our experiment. However, for two of these three bugs (`yara` and `file`) we see no difference in fuzzer effectiveness. For the last bug found by AFL, `haproxy`, there are differences in performance, but all fuzzers except for one (Selectfuzz) find the bug in 5 minutes excluding build times. Additionally, because we evaluated *only* fuzzers based on the AFL fuzzer implementation, we believe this effect to be minimal. We leave a larger follow up study with additional bugs found by both AFL and LibFuzzer which could isolate this effect to future work.

Conclusion In general, we believe our experimental setup is in accordance with the latest standards for fuzzer evaluations [113, 109].

To avoid issues with misinterpreting our results, we use statistical techniques that are well-established across multiple fields of research. As bug-finding benchmarks are *right-censored* data, we use survival analysis [105]. To determine statistical significance between fuzzers on individual benchmarks for a single configuration, we use the confidence intervals given by our survival analysis [103], which can be seen in Figure 4.2. When comparing across benchmarks and fuzzers, we use the bootstrapped Restricted Mean Survival Time [194] and Wald's t-test [223]. For bootstrapping, we use 100 iterations of case-resampling the individual trials of each experiment. For other claims of significance, we compare build times with Wald's t-test and code coverage for CIDFuzz with Fuzzbench's built in Mann-Whitney U test [142, 227]. To validate model assumptions, we check for normality visually using a QQ plot before applying Wald's t-test.

Notably, we choose not to use two common analyses for censored data –the log-rank test and Cox-regression– because our data appears to violate the proportional hazards assumption (Figure 4.2).

External We follow the recommendation of [113] that ≥ 10 benchmark programs should be used for fuzzer evaluations. As with any empirical study, more subject programs and fuzzers would improve the generalizability of our results. However, verifying each bug's reproducibility, the introducing commit, integrating OSS-Fuzz benchmark programs, and generating a large initial corpus all take significant manual effort. During the course of this experiment, we attempted to set up a total of 25 OSS-fuzz programs using Fuzzbench's integration with OSS-Fuzz. However, of these programs and bugs, most could not be used: seven resulted in build errors that we were not able to debug, three had errors related to fuzzer instrumentation passes, three we could not reproduce reported bugs at the commits indicated in the bug report, and the remaining two had issues with several dependent libraries which also needed to be built as specific commits to reproduce past bugs. We believe that this study is still valuable at its current size, especially when viewed with complementary works such as [246, 243, 109, 114] and we ensured that each of our configurations had sufficient trials ($n = 20$) to generalize for the bugs and subject programs we

evaluated.

4.3 Discussion

In this chapter, we examined one actionable vertical expansion of fuzzing into existing developer workflows. Several obstacles remain, namely finding more challenging bugs within allotted time-limits and mitigating the cost of pre-analysis used by many specialized directed, commit and regression fuzzers. However, we found that existing fuzzers *can already* be leveraged with minimal effort to be effective in identifying bugs in code-changes in both CI/CD and local development contexts. Looking ahead, we believe these tighter integrations will be critical to scale fuzzing to the needs modern software development. Once integrated, the next question will be how to triage and fix bugs reported by fuzzers, which we explore in the following chapter.

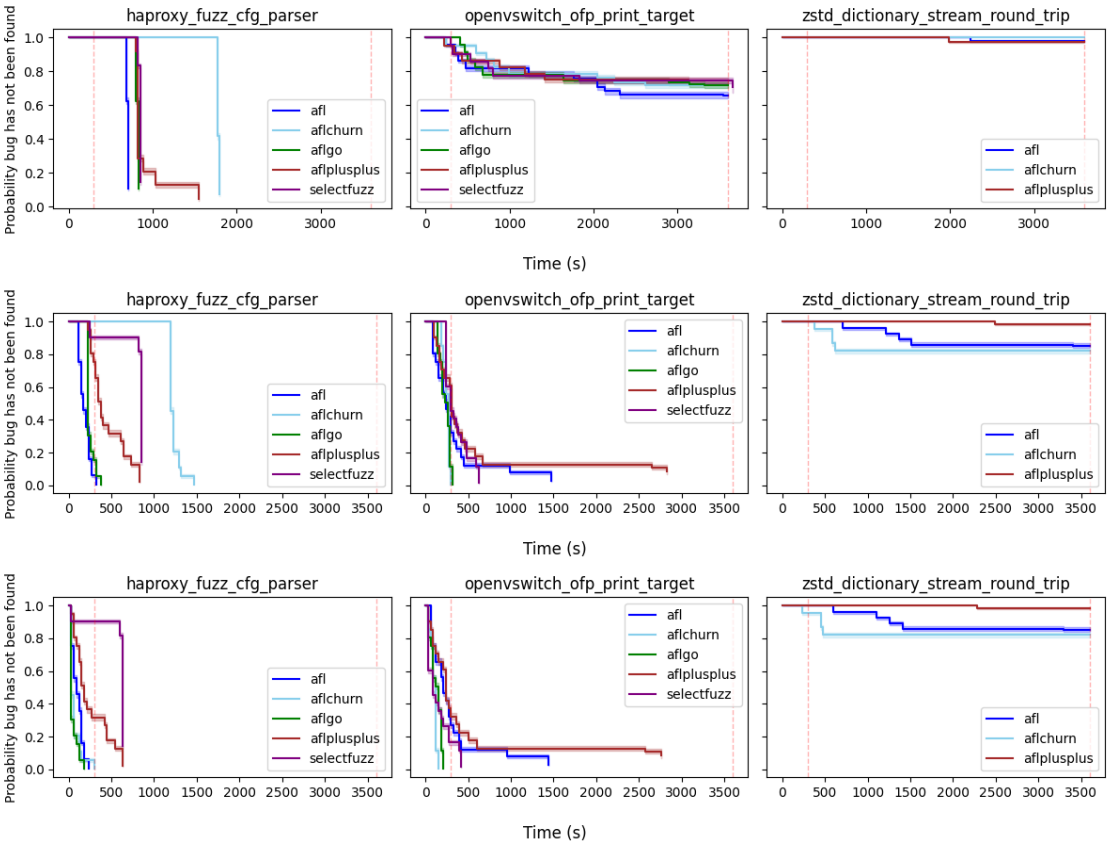


Figure 4.2: Probability that the regression bug **is not found** by each fuzzer over time (lower is better). **(top)** using static corpus; **(middle)** using saturated corpus; **(bottom)** using the saturated corpus *excluding* build times

Chapter 5

A Fuzzing-Integrated Cyber Reasoning System

Thus far, we have explored how fuzzing can be expanded into existing change-based developer workflows. However, the demands of modern software infrastructure and the accelerating pace of its development call for a more radical change to these workflows. Indeed, because of their high level of automation, fuzzers are often limited by the human processes surrounding their deployment [91]. Thus, to ensure that our software is truly secure and reliable, a holistic approach is needed that can automatically progress through *all* stages of the cyber-security and reliability pipeline. How can we discover *and* remediate vulnerabilities in complex software, prior to them being encountered in the field? What opportunities do recent advances in Large Language Models (LLMs) present for achieving this goal?

In this chapter, we articulate broadly how LLMs can contribute to building so-called Cyber-Reasoning Systems (CRSs) which attempt to address these issues fully *autonomously*. We first outline the general structure of a consolidated Cyber-Reasoning System (CRS) which leverages fuzzing, along with examples of the role that LLMs can play in various components of this system. Subsequently, we summarize the high-level technical challenges and opportunities in this emerging research area. This chapter is inspired by the recently completed DARPA AI CyberChallenge[1] from which we take the term Cyber-Reasoning System, and partly also by the recent public interest in autonomous software engineering, where programs are fixed and improved automatically [101] using LLM agents such as AutoCodeRover [244, 195].

[1] `https://aicyberchallenge.com`

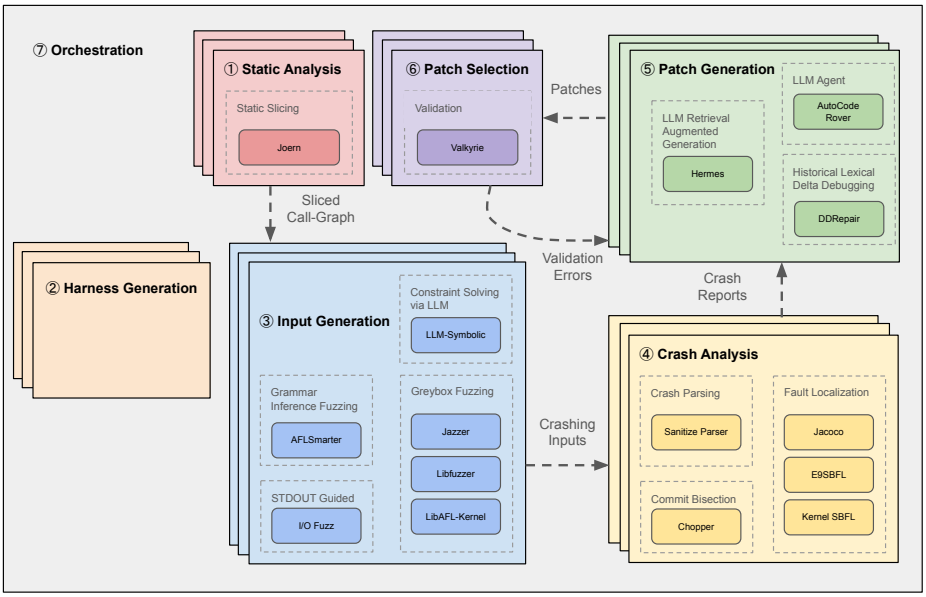


Figure 5.1: A Cyber-Reasoning System Workflow; Example CRS tools listed within each component

5.1 A Cyber Reasoning System

A *Cyber Reasoning System* (CRS) is a software system which can both detect and repair software vulnerabilities *autonomously* in a given System Under Test (SUT). Ideally, a CRS supports a wide range of real-world software systems, including those written in different programming languages and/or containing millions of lines of code. We note that substantial prior research has been conducted on various subgoals of a CRS [79, 141, 230]. Thus the core challenge of building a CRS lies in overall *flexibility*, *scale*, and the careful *combination* of disparate techniques into a coherent, effective system. Despite decades of research and industry interest, building such a CRS using only traditional program analysis tools has proven to be impractical to the point of impossibility. In particular, there are acute challenges that must be overcome. For example that the success probabilities of each "stage" in the remediation pipeline are multiplied, leading to a significantly lower success rate of the whole process as compared to each individual step. Similarly, the *robustness* of each constituent tool is higher than that of a naive composition. If a fuzzer works "out-of-the-box" on 90% of programs and a program repair tool works on 60%, then it is very likely that the compatibility of these two tools does not entirely overlap, leading to a compositional compatibility far lower than a theoretical maximum

of 60% in practice. The recent emergence of Large Language Models, however, has made the realization of a CRS possible, via a broad new design space where traditional analysis techniques are replaced or augmented by LLMs.

A CRS Framework To more clearly illustrate the challenges and opportunities presented by the construction of a CRS, we decompose it into seven sub-components, shown in Figure 5.1. A CRS might first use static analysis (①) to build a broad understanding of the SUT. This component could include general pre-analyses such as control-flow graph construction or more specific vulnerability scans to identify potentially buggy program locations. A CRS then needs to identify how to properly build and execute the SUT with new unit-tests or test inputs which could trigger a vulnerability (②). Once a CRS has a way to test the SUT, it (③) can search the space of possible testcases for those that may trigger a vulnerability via test generation. If a generated test triggers a vulnerability, the CRS can then analyze this test (④), along with other triggering and non-triggering tests to gather information about the vulnerability and determine if it has been previously discovered. This information can then be passed to patch generation tools (⑤), which attempt to find a repair for the given vulnerability. The CRS then selects (⑥) one or more patches that best fix the vulnerability without breaking the functionality of the program.

Figure 5.1 and the prior description presents a CRS as a mostly sequential pipeline. However, there may be other useful interactions between components; e.g. a crash analysis tool could incorporate information from a failed patch validation to refine its report (⑥ → ④). Additionally, each component may need to be scaled up or down dynamically: if a CRS finds many new vulnerabilities in a short period of time, test generation capacity could be reduced to shift additional resources towards crash analysis and repair of these vulnerabilities. As such, we identify orchestration (⑦) as an additional component of a CRS. The orchestrator determines resource allocation and facilitates communication between subcomponents. Any of these components, including the orchestrator itself, may leverage LLMs in lieu of or to complement existing program analysis tools.

The AIxCC Competition The recent DARPA AI Cyber Challenge (AIxCC) Competition is the origin of the term Cyber-Reasoning System (CRS) and brought wide attention to this problem. The competition invited teams to build a CRS which could autonomously find and fix vulnerabilities in C and Java projects (including the Linux kernel), with the winner receiving a large cash prize. In this chapter, we generalize the definition of CRS beyond the scope of the competition itself. For example, harness generation (②) was not part of the AIxCC competition, but is important for a CRS to be able to analyze real systems outside of a competition setting. Additionally, while the AIxCC competition was restricted to pre-trained models, a true CRS could include custom or fine-tuned models. on additional data. We also claim an ideal CRS should be able to analyze polyglot, heterogeneous software, including distributed systems, machine learning components, or embedded systems – which are were all outside the scope of the original competition.

An Example CRS To make the concept of CRS more concrete, we present an open source cyber-reasoning system example in the style of the AIxCC competition:

[`https://github.com/DARPA-AIxCC/asc-crs-healing-touch`].

The individual components of this CRS are outlined as bullet points in Figure 5.1. For input generation (③), it relies on an ensemble of fuzzers. As a baseline, it utilizes the existing grey-box fuzzers LibFuzzer and Jazzer[2] for C and Java programs, respectively, in addition to a custom grey-box fuzzer for the Linux kernel built with the LibAFL framework [60]. Additionally, it leverages LLMs to infer a grammar for the target program's inputs to allow a specialized fuzzer to generate well-formed test input. As fuzzers are a relatively mature and widely applied technology used in industry, we relied less on LLMs for this component. To analyze bugs found by our fuzzers (④), the CRS parses the output from common vulnerability detection tools (called *sanitizers*[3]) and determine the bug introducing commit. For this component, we did not utilize LLMs at all, as the sanitizer output was given in a fixed format, but future work could leverage LLMs to be less sensitive to different crash reporting

[2] `https://llvm.org/docs/LibFuzzer.html` and `https://github.com/CodeIntelligenceTesting/jazzer`

[3] For example, ASAN detects memory safety bugs during program executions that might otherwise exit without errors

mechanisms. For Java programs, the CRS also uses the Joern[4] static analysis framework (①) to extract path constraints and then solve these constraints using an LLM, described in Section 5.2.2. Given a vulnerable input from a fuzzer, the CRS uses Spectrum-Based Fault Localization tools (SBFL) [230] to augment crash reports with suspicious lines before passing them on to be repaired. It can utilize an existing SBFL instrumentation for Java [92], while needing fully custom implementations for the Linux Kernel and other C programs. From a crash report, the CRS attempts to repair (⑤) the bug with three different repair tools. The first is a Retrieval Augmented Generation (RAG) [121] repair tool, which attempts to fix the bug using a single prompt to an LLM that includes similar vulnerability reports and patches. The second is an LLM agent AutoCodeRover [244, 195], which was adapted to work on C and Java programs. Lastly, it also includes a repair tool that does not utilize LLMs, instead using lexical delta-debugging[5] [242] on the differing lines between the current and pre-bug versions of the project. Once a patch is generated by any of the tools, it is selected for submission (⑥) if the validation tool Valkyrie determines that both the failing test input and the subject's functional test suite are free of crashes and errors. For orchestration (⑦), the CRS uses a custom Python framework which used a greedy static resource allocation across available hardware and passes data between components as shown by the dashed arrows in Figure 5.1. In the case of our example CRS, neither ⑥ nor ⑦ leverage LLMs, although such an integration is eminently feasible.

Example CRS Evaluation While most of the subject programs from the AIxCC competition remain undisclosed, one –the `nginx` web-server– has been publicly released with 14 accompanying bugs from the semi-final.[6] To assess the capabilities and potential of a CRS in more detail, we evaluated the example CRS described above on this benchmark in 20 trials of 4 hours each, with 64 allocated CPU cores and 16GB of allocated RAM on a Ubuntu 20.04.6 LTS (5.15.0-139-generic) machine with an AMD EPYC 9554 64-Core Processor (Model 17) and 1.5TB of total RAM. This emulates the setup for a single node of the three provided in

[4] `https://github.com/joernio/joern`

[5] `https://security.googleblog.com/2024/06/hacking-for-defenders-approaches-to.html`

[6] `https://github.com/aixcc-public/challenge-004-nginx-cp`, commit `bd44905`

the AIxCC semi-final competition. We also ran the CRS both with and without LLM-integrated components for bug discovery (③) and repair (⑤), respectively, to assess the impact of these components on the system as a whole. We used OpenAI GPT-4o (2024-08-06) as the sole LLM-backend of the CRS[7]. Overall, we saw that the example CRS is capable of finding *and* fixing vulnerabilities in an integrated workflow. This CRS finds more than two unique vulnerabilities on average per-trial ($\mu = 2.4$) and generates a plausible[8] patch for at least one of those discovered vulnerabilities in nearly half of all trials (9/20). In some instances, the plausible patches were also identical to the ground-truth patch given by the competition organizers. Additionally, in its best trial, the CRS found four unique vulnerabilities and generated plausible fixes for all four of these bugs within the time limit.

We also found that the components integrating LLMs were *crucial* to the success of the CRS. Removing the LLM inferred-grammar fuzzer from the CRS (③), the remaining fuzzers, including LibFuzzer, are unable to discover a single bug within a 4 hour time limit in *any* of 20 trials. To confirm these findings, we conducted an additional experiment with the state-of-the-art fuzzer AFL++, and saw that it also was unable to discover any bugs in any of 20 trials within the time-limit. We attribute the failure of conventional fuzzing to a lack of initial valid inputs provided in the competition; these are typically given to grey-box fuzzers to allow them to reach deeper program behaviors quickly. Similarly, when omitting the LLM-based repair tools (⑤), the remaining repair tool was unable to generate a plausible patch for *any* of the found bugs, also across 20 trials. The delta-debugging repair tool struggles here because it is inherently sacrificing efficacy for generality: in order to be fully agnostic to the source language, it can only fix a small subset of regression bugs. We believe this difference highlights the importance of integrating LLM-assisted tooling into automated security workflows to create general and effective tools that can bridge gaps in existing analyses.

[7] `https://openai.com/index/hello-gpt-4o/`

[8] Here plausible means the patched program no longer crashes on the PoV input and passes all competition-provided tests

5.2 Opportunities for LLMs in Software Analysis

Each component of a CRS could leverage LLMs individually: static analysis (①) tools which analyze source code [4] could use LLMs to reduce the impact of syntactic noise; test harnesses (②) could be more robustly generated for different build systems and project structures by LLMs; fuzzers (③) could leverage LLMs for constraint solving; crash analysis and triage (④) using LLMs could respond dynamically to unforeseen issues in production that cannot be accounted for *a priori*; patch generation (⑤) and selection (⑥) are already one of the most ubiquitous use-cases for LLMs; and lastly orchestration (⑦) is particularly well suited to agentic AI for making resource allocation and other planning decisions. There are many such design points, so we present three broad overarching opportunities for LLMs which generalize or span across multiple components of a CRS.

5.2.1 Augmenting Individual Analyses with Developer Intent

While Language Models can be powerful tools in their own right, prior work [244] and our experience in the AIxCC has shown us that LLMs can be most effective *in combination* with existing analyses.

Incomplete Analyses Many algorithms in program analysis are *incomplete* in that they might report false positives, often because precise reasoning is algorithmically or practically impossible. A key opportunity for the application of language models lies in their ability to infer contextual intent, such as neighboring source code or comments, which can help filter or re-prioritize an analyzer's output. As an example, we used the SemGrep static analysis (①) tool to scan an open-source LLM orchestration project on Github[9] for hardcoded user credentials. The run of SemGrep generated a warning for the code shown in Figure 5.2 (b). This report is a false positive because the developer *intends* for users of the project to replace the `api_key` fields, as evidenced by the contents of the corresponding strings. Indeed, comments and other natural language artifacts often contain high-quality information, but this information has previously proven difficult to extract and leverage [119]. If we pass

[9] `https://github.com/Doriandarko/maestro/blob/main/maestro-gpt4o.py:d552e01`

the report and source code context from Figure 5.2 to GPT-4o, it determines that the report can be ignored in 26 out of 30 trials.

(a) Abbreviated SemGrep Rule for CWE-798

```
rules:
 - id: hardcoded-openai-token
  message:
    - A hard-coded credential was detected. It is not recommended to store credentials in
      source-code...
  metadata:
    cwe:
     - "CWE-798: Use of Hard-coded Credentials"
    vulnerability_class:
     - "Leaking Secrets"
  severity: WARNING
  languages:
   - python
  pattern: OpenAI("...");
```

(b) Source Code for False-Positive

```
...
# Initialize OpenAI and Anthropic API clients
openai_client = OpenAI(api_key="YOUR API KEY")
anthropic_client = Anthropic(api_key="YOUR API KEY")
...
```

Figure 5.2: Example of An Incomplete SemGrep Analysis

Unsound Analyses Many other components of the CRS can be implemented with *unsound* program analyses. Such analyses can suffer from a large amount of *false negatives*. As with their incomplete counterparts, the usefulness of unsound analyses is often directly tied to the tightness of this approximation; a fuzzer which misses many obvious bugs in the program is unlikely to be deployed in practice. Here, again, LLMs can leverage approximate developer intent to complement existing approaches and reduce false negatives. For example, an input grammar represents data that the program is *intended* to consume. Using an LLM, a CRS can quickly generate a grammar from the fuzzing harness. Producing such an input specification allows established techniques in grammar fuzzing [12] to rapidly explore the intended behavior of a program, as in our example CRS.

p == 0x13	c < 0xFF	h == "x-evil-backdoor"	0	SHA256(v) == SHA256("breakin the law")	0	s == "jazze"
Picker	Count	Header		Value		Sanitizer Check

Figure 5.3: Input Structure and Constraints for the Jenkins Exemplar Bug from the AIxCC Semi-final Competition

5.2.2 Generalizability and Flexibility of LLMs

Due to their immense training datasets, LLMs have been shown to generalize to a wide variety of technical tasks with little or no additional training [30].

Generalizability of Problem Domains A CRS covers many disparate subproblems that previously required bespoke analysis tools to solve individually. LLMs lower the barrier to entry for writing these tools, allowing CRS builders to create components that are robust to different software versions, build tools, and environments while often remaining as capable as more fragile, complex analyses. One such analysis task is to demonstrate the Proof Of a Vulnerability (PoV) in the form of an input which triggers the bug (②). Figure 5.3 showcases one such PoV input for an exemplar bug from the Jenkins exemplar program for the AIxCC. In order to reach the buggy code, a CRS must craft a structured message with several fields exactly matched to particular byte-strings. The `Value` field, however, is hashed before it is compared to a desired byte-string, making it effectively *impossible* for typical constraint-solving approaches, such as symbolic execution, to solve analytically as this would amount to reversing a SHA256 hash. Yet, without additional training, a Language Model is able to generalize enough to attain the *effect* of a constraint solver without this limitation.

To showcase this capability, we use a component of our example CRS (Section 5.1) – this tool gathers path constraints to a particular target location using static analysis and coalesces them into a prompt, see Figure 5.4. When querying the language model, out of 30 prompts, it synthesizes six correct and four nearly correct inputs. The nearly correct test inputs have only an additional null terminator, but are otherwise an exact match. We also ran the grey-box fuzzer Jazzer for 24 hours, providing it with a "dictionary" file that contains all string constants present in the source code of the SUT, including the `Header` and `Value` strings needed for triggering the bug. In *none* of the 20 trials was the fuzzer able to find the vulnerability.

Flexibility Across Languages We manually translated the extracted constraints and harness in Figure 5.4 from Java to Python, resulting in Figure 5.5. We then passed these Python constraints to an LLM in the exact same prompting template with *no modifications*. In this scenario, the model was still able to solve the Python

Prompting Template

You are a penetration testing engineer tasked with testing a Jenkins application. Now, we notice there is one or more possible vulnerabilities of `{{Vulnerability Class}}`.
Note that we have a very different definition of this type of vulnerability. Here is the definition:
`{{Sanitizer Definition}}`
Please ensure:
`{% for c in Path Constraints %} {{c}} \n {%endfor%}`
Here are the dataflow slices of the program:
`{% for s in Dataflow Slices %} {{s}} \n {%endfor%}`
This is a file that is related to the input data:
`{{Fuzz-Harness Source Code}}`
Generate a Python program that dumps the expected data to the file 'input.bin'

Sample Templated Values

```
{
"Vulnerability Class": "OsCommandInjection",
"Sanitizer Definition": "for OSCommandInjection, APIs like `java.
    lang.ProcessBuilder' must contain a magic word named `jazze'
    rather than the actual command",
"Path Constraints": [
 "containsHeader(rqst.getHeaderNames(),\"x-evil-backdoor\") ==
    true",
 "MessageDigest.isEqual(sha256, providedHash)) == true" ],
"Dataflow Slices": [
  "boolean isAllowed=jenkins().hasPermission(Jenkins.ADMINISTER);"
    ,
  "byte[] sha256 = DigestUtils.sha256(\"breakin the law\");",
  "String backdoorValue = request.getHeader(\"x-evil-backdoor\");"
    ,
  "byte[] providedHash = DigestUtils.sha256(backdoorValue);",
...
}
```

Figure 5.4: Prompting Template and Templated Values

program constraints in 8/30 trials. This is remarkable in that the model did not require any changes to solve these constraints at the source level in *both* languages.

5.2.3 Opportunities for Agentic AI

AI Agents go beyond basic in-context learning by enabling LLMs to iteratively plan, reason, and take actions — such as invoking program analysis tools. As a result, they are often able to achieve complex, multi-step goals fully autonomously.

Agents can reasonably be used to instantiate any individual components of the CRS. Indeed, our example CRS uses the AutoCodeRover [244, 195] LLM agent to fix vulnerabilities (⑥). This agent uses code navigation and analysis tools to iteratively develop and refine a patch. AutoCodeRover has now been integrated in the widely used SonarQube static analysis tool to automatically fix detected security bugs, showing its practicality.

Beyond individual analysis steps, the orchestration layer (Figure 5.1, ⑦) is

Templated Values Translated to Python

```
"Vulnerability Class": "OsCommandInjection",
"Sanitizer Definition":
 "for OSCommandInjection, APIs like `subprocess.
     Popen' must contain a magic word named `jazze
     ' rather than the actual command",
 "Path Constraints": [
   "\"x-evil-backdoor\" in request.headers",
   cryptography.constant_time.bytes_eq(sha256,
     providedHash)",
],
"Dataflow Slices": [
 "isAllowed=jenkins().hasPermission(Jenkins.
     ADMINISTER)",
 "sha256=cryptography.hashes.Hash(crypto.hashes.
     SHA256()).update(\"breakin the law\").
     finalize()\")",
 "backdoorValue = request.headers[\"x-evil-
     backdoor\"]",
 "providedHash = cryptography.hashes.Hash(crypto.
     hashes.SHA256())
       .update(backdoorValue).finalize()\")",
     ...
]
```

Figure 5.5: Python Constraints Passed to an LLM in the Template of Figure 5.4 for the *Translated* Jenkins Exemplar

particularly well-suited to AI agents; it involves numerous and repeated challenging planning decisions that would previously have been the purview of a human auditor. For example: If a vulnerability is found, should the CRS conduct additional static analysis of crash location or should it instead allocate those resources to further dynamic analysis of the vulnerable execution? What information from these analyses should be passed as context to the patch generator? As agents have begun to show significant progress in many individual tasks such as remediation [244], a more general and comprehensive deployment of LLM Agents *as the CRS itself* is promising area of future research.

5.3 Challenges for LLMs in CRSs

Unsoundness A key strength of many static vulnerability scanners (①) is their soundness; they can sometimes *guarantee* a program to be free of particular vulnerabilities. An LLM cannot guarantee soundness alone [192], so we see determining how to recover these guarantees or increased assurance of correctness as a key challenge for LLM-based tools in a Cyber Reasoning System.

Incompleteness As Language Models are statistical models at their core, they can also give *false positive answers* to analysis queries, amounting to an incomplete analysis. This may be mitigated in some situations where an efficient checking oracle exists [224] – for example, the input generation (③) task in Section 5.2.2 the LLM can be called iteratively until the oracle is passing i.e. the bug is triggered. Identifying these oracles will be an important area of research in CRSs.

Training and Evaluation Data Language Models are highly dependent on the data used to train them. Because of their size, their corresponding training sets must also be extremely large to avoid over-parameterisation. Assembling large enough datasets for all components of the CRS workflow without introducing risk of *data leakage* [38] will be crucial to development and evaluation of CRSs.

5.4 Threats to Validity

As CRSs are an emerging research area, our study was limited to the only dataset available for end-to-end CRS evaluation. As such, our results may not generalize beyond this dataset (c.f. Section 5.3). We did not evaluate the repair components in our CRS against traditional program repair research tools — a threat to the internal validity. However, we excluded these tools from the CRS due to their poor generality (c.f. Section 5.2.2), and similar comparisons have been published in prior work [233].

5.5 Discussion

Innovative technologies often disrupt established sectors in unexpected ways with surprising speed. We believe that LLMs represent such a change, not just in how we write code, but also in how we reason about it. Automating this reasoning will be critical to keep up with our ever-growing, increasingly complex software infrastructure, and its security. We can already see concrete benefits in augmenting or replacing existing software analysis with LLMs, such as fuzzing, but many more possibilities remain. Broadly, we believe LLMs can complement gaps in existing analyses by grasping developer intent, generalizing to different programming constructs and environments, and providing agentic multi-step reasoning spanning numerous individual analyses. By focusing on these opportunities, researchers will

be able to push the frontier of what is possible in an integrated Cyber-Reasoning System.

Part II

Horizontal Expansion to Concurrent Systems

Chapter 6

Structured Grey-box Concurrency Fuzzing

Thus far, we have looked at a vertical expansion of fuzzing to new software workflows. However, it is equally important to expand the *kinds* of software which can be analyzed in a highly automated manner. In particular, concurrent programs and associated concurrency bugs have long been a challenge for program analysis techniques. In the modern era, such programs are ubiquitous, consisting of distributed systems, operating system kernels, web browsers and many others. Yet the space of concurrent behaviors observable in different program interleavings remain largely out of reach for conventional fuzzers. In this chapter, we bring concepts from fuzzing to create a new algorithmic framework for interleaving-space exploration.

A common approach adopted in practice to detect concurrency bugs is to conduct stress testing of the software against exceptional workloads. These approaches can be termed as *optimistic* approaches since they hope that applying varied and large workloads will expose corner cases of the program behavior. Another manifestation of this *optimistic* approach towards concurrency testing is random sampling. Here randomized algorithms sample in the space of interleavings or partial orders with the goal of exploring various neighborhoods in the search space of behaviors evenly.

An alternative category of techniques that have been widely studied in the research community is systematic testing and exploration – the *pessimistic* approach. These approaches insert certain hooks into the program or model the program to control the scheduling and systematically explore the space of interleavings [71]. One common observation to reduce the search space of behaviors is to either employ partial order reduction [5, 62, 118] or heuristics such as context bounding - where

the number of context switches is bounded by an arbitrary constant. We view the enumerative approaches such as CHESS [161] or GenMC [118] as *pessimistic* since they attempt to go toward exhaustive exploration, albeit in a reduced search space.

In this chapter, we propose a new direction in concurrency testing - which is neither optimistic, nor pessimistic. Our key idea is to conduct a biased, randomized search over a reduced space of functionally equivalent interleaving partitions, based on their "reads-from" information. The reads-from relation induced by an execution, maps each read event to the corresponding write event it observes. This relation serves as a *semantic* abstraction for a given execution of a concurrent program. To effectively explore the reduced space of reads-from partitions, we leverage a pro-active, controlled scheduler which drives executions towards or away from individual partitions. Finally, we also use the reads-from relation as feedback to bias the search towards novel partitions - if a new reads-from pair is encountered, the execution is deemed novel and prioritized in the search. In this way we can use the reads-from relation to effectively pivot our search, and adapt it towards new and semantically different search neighborhoods, by mutating previously encountered schedules. Our approach is, thus, neither *optimistic* nor *pessimistic* - we deem it to be *pragmatic*.

Thematically, our approach can be seen as grey-box fuzz testing, adapted to concurrent programs. Grey-box fuzzing has been extremely successful for finding bugs and vulnerabilities in sequential programs in practice [9, 27]. Part of this success comes from the adaptive and non-enumerative nature of the search, enabling it to scale to sequential applications such as the PDF readers, multimedia encoders and compression libraries – whose input spaces can be large, encompassing all possible byte combinations of multi-Kilobyte to multi-Gigabyte sized files. Similarly, the concurrency-centric grey-box fuzzer we develop for this work, `RFF`, does not enumerate all possible interleavings or even reads-from partitions, but rather conducts an adaptive, randomized search of the reduced space of partitions.

While grey-box fuzzers for testing concurrent programs have been developed by prior works [97, 41, 234, 100], our tool, `RFF`, advances the state-of-the-art due to fundamental design and implementation choices. `RFF` fully controls the schedule, as relying on intermittent or loose control of the schedule (e.g. delay injection) [41, 97] could allow the test environment itself to render some interleavings difficult or impossible to reach. `RFF` uses the semantic, reads-from relation as feedback to

bias its search, rather than execution statistics such as function-call pairs [100] or event-pair orderings [234]. Also, unlike other concurrency-aware fuzzers, RFF structures its search in terms of a novel abstraction, again leveraging reads-from information. Aided by our pro-active scheduler, this abstraction effectively reduces the exponentially large space of interleavings.

We evaluated RFF against state-of-the-art bug-finding tools [226, 118] and algorithms [241, 32] on 49 programs containing concurrency bugs. We find that RFF not only consistently finds bugs in more programs than any other approach –46 programs on average– it also typically finds these bugs in fewer schedules. We also show that RFF explores the scheduling space as partitioned by the reads-from equivalence relation evenly, and that a fuzzing-inspired search leverages this partitioning effectively as compared to a common reinforcement learning framework [182].

Contributions. Concretely our contributions are:

- Conceptually, we bring ideas from grey-box fuzzing of sequential programs (such as power schedule or feedback function) into the exploration of concurrent program interleavings.
- We structure the schedules as a collection of "reads-from" constraints to define the fuzzer search space. This structured search space helps in bug finding effectiveness as shown by our experiments. We additionally utilize the "reads-from" relation as supplementary feedback function to guide this search.
- We implement our approach as a fuzzer called Reads-From Fuzzer (RFF). Experiments on well known benchmarks like SCTbench [199] show the bug-finding effectiveness of RFF's biased random search approach vis-a-vis random search (such as Partial order sampling or POS) as well as vis-a-vis systematic testing (such as state of the art model checkers like GenMC).
- We make RFF and its associated instrumentation and execution framework available for usage and research:

  `https://doi.org/10.6084/m9.figshare.23911299`

```
1  static int a = 0, b = 0;
2  void setThread() {
3      a = 1;
4      b = -1;
5  }
6  void checkThread() {
7      if(!((a == 0 && b == 0) || (a == 1 && b == -1))){
8          assert(0); // Bug found
9      }
10 }
11 int main() {
12     const int n = 100;
13     std::vector<std::thread> st;
14     for (int i = 0; i < n; ++i) {
15         st.push_back(std::thread(setThread));
16     }
17     std::thread ct(checkThread);
18     return 0;
19 }
```

Figure 6.1: SCTBench subject `reorder_100` (repurposed)

6.1 Overview

The central challenge encountered in exposing concurrency bugs is the sensitivity to thread schedules — even for a fixed input, a bug may surface only under certain interleavings. Further, the number of interleavings grow exponentially in the length of executions. Enumerative search over interleavings is thus difficult even after employing optimizations such as partial order reduction [70, 7].

Randomized concurrency testing, often offering probabilistic guarantees [32, 241], has therefore emerged as a promising paradigm for exposing hard to find interleavings. Existing state-of-the-art randomized techniques though often fall short. Consider, for example, the (stripped down) program snippet shown in Figure 6.1, derived from the SCTBench benchmark suite [199]. The `main` thread spawns $\mathtt{n} = 100$ *setter* threads $\mathtt{st}_1, \ldots, \mathtt{st}_\mathtt{n}$ that each set the value of the shared variables `a` and `b` to `1` and `-1` respectively, as well as a single *checker* thread `ct` that asserts that the values of (`a`, `b`) are either (`1`, `-1`) or (`0`, `0`). Observe that the assertion in `ct` gets violated only when `a` observes the value written by one of $\mathtt{st}_1, \ldots, \mathtt{st}_\mathtt{n}$, while `b` observes the initial value 0. This happens when `ct` is executed after at least one setter thread executes its first step '`a = 1;`', and before any setter thread executes its second step '`b = -1;`'.

Assuming that the read events of `a` and `b` in thread `ct` happen atomically, the total number of executions of this program is $\frac{(3n+2)!}{(n+1)!(3!)^n(2!)}$[1], while, the number of interleavings, that hit the assertion violation is $\sum_{i=1}^{n} \frac{\binom{n}{i}(n+i+1)!(2n-i)!}{(n+1)!2^n}$[2]. A sampling strategy that uniformly samples each execution will hit the violation with a probability of about 2.8×10^{-14} making the expected number of steps to hit the violation prohibitively large. Partial Order-aware Sampling (POS) [241] offers a similar guarantee. Randomized sampling strategies that rely on *bug depth hypothesis* such as PCT [32] also work poorly here — needing at least one ordering constraint from the first step of some setter thread to the checker thread, and 100 ordering constraints to order the checker thread before the second steps of each setter thread. The depth of this bug therefore is at least 101, which is beyond the tractable boundary of PCT given that the probability guarantee of PCT scales exponentially in the depth. Our experimental evaluation also confirms this analysis — both POS and PCT struggle to hit the bug in a reasonable number of trials.

In contrast, our concurrency testing tool `RFF` exposes the bug in about 6 iterations in each of the 20 trials we performed. The effectiveness of `RFF` stems from two important insights. The first is the observation that two concurrent executions that follow the same thread-local control and data flow are both equally capable of exposing the same assertion violation; thus exploring one of them suffices. The *reads-from* function, mapping each read event of a shared memory location to its corresponding writer event, serves as a good proxy for control and data-flow. More importantly, the number of different *classes* induced by reads-from equivalence can be exponentially fewer than interleavings, partial orders [6] or even classes induced by bounded-depth ordering constraints. ***RFF*** *covers all executions that are reads-from equivalent in one go, by generating only one of them.* In the program from Figure 6.1, the assertion violation can be exposed by any execution that satisfies the following

[1]The numerator counts the number of arrangements of $(3n + 2)$ steps (n+1 thread spawning steps of the main thread + $2n$ steps of the setter threads + 1 step of the checker thread), while the denominator offsets overcounting of the arrangements of the spawning events, and the total order within each setter and checker thread (together with their spawn events)

[2]The ith term in the numerator counts the number of interleavings where thread `ct` executes after $n + 1$ (thread spwan) steps + i steps (`a = -1;` of i setter threads). The term (n+1)! offsets overcounting the permutations amongst the spawn events performed by `main` while the term $2^n = 2^i \cdot 2^{n-i}$ offsets the relative order of first steps of the i setter threads w.r.t spawn events, and the relative order within setter threads for the later $n - i$ setter threads

abstract schedule:

$$\alpha_{\mathsf{violation}} = \{\, \mathtt{w(a)}@\ell_3 \xrightarrow{\mathsf{rf}} \mathtt{r(a)}@\ell_7, \quad \mathtt{w(b)}@\ell_1 \xrightarrow{\mathsf{rf}} \mathtt{r(b)}@\ell_7 \,\}$$

The constraints above demand that the read of `a` in line `7` (in `checkThread()`) observes the write of `a` in line `3`, while the read of `b` observes the the initial write in line `1`. While the total number of different reads-from constraints is just 4 (2 choices for `a` and 2 choices for `b`), the chance of generating $\alpha_{\mathsf{violation}}$ when (almost) uniformly sampling this space is large. Combined with our *proactive reads-from scheduler* that carefully attempts to schedule any given abstract schedule, RFF can successfully uncover the bug with very few trials.

Our second insight towards the design of a more effective concurrency testing technique is the introduction of (moderate amount of) *statefulness* in contrast to the otherwise unbiased random search techniques. Statefulness in the exploration can help identify previously explored parts of the schedule search space and can be leveraged to drive exploration away from them. RFF effectively tracks information about the set of schedules observed so far, and utilizes it to generate a new abstract schedule. In the program in Figure 6.1, for example, RFF generates $\alpha_{\mathsf{violation}}$ by extracting a list of events observed in previous schedules and stitching them, through random mutations, to arrive at the constraints in $\alpha_{\mathsf{violation}}$. RFF also uses information about when a previously explored abstract schedule led to *novel* behavior and uses this as further *feedback* to guide what kinds of abstract schedules to generate next.

We show that these insights can be combined into a *grey-box fuzzing* approach. Grey-box fuzzers [9, 27] have proven effective in discovering software vulnerabilities in sequential programs. While traditional grey-box fuzzing techniques represent a biased random search over the space of program inputs in a sequential program, the support for exposing rare concurrency-centric behaviors in such techniques is either absent or remains rudimentary. Section 6.2 describes the details of our approach of formulating concurrency testing as a grey-box fuzzer.

6.2 Core Approach behind RFF

As our core approach, we adapt a well-known grey-box fuzzing algorithm [27] to the space of interleavings/schedules; Algorithm 3 shows our overall template. The

Algorithm 2: Grey-box Concurrency Fuzzing

Input: Initial corpus of schedules S_{init}

```
1:  S ← S_init, S_fail ← ∅
2:  if S = ∅ then S ← { ε }                          /* Empty Schedule */
3:  repeat
4:      (σ, η_σ) ← PickNextAndAssignEnergy(S)
5:      for i ∈ { 1, ..., η_σ } do
6:          σ_mut ← mutateSchedule(σ, S)
7:          if σ_mut crashes then S_fail ← S_fail ∪ { σ_mut }
8:          if isInteresting(σ_mut, S) then
9:              S ← S ∪ { σ_mut }
10: until timeout
11: return S_fail
```

algorithm begins with an initial set of schedules. During each iteration of the fuzzing loop, an existing schedule σ is selected from the working set of schedules (line 3). The algorithm then mutates σ to create a new schedule σ_{mut} (line 5), which will be used to test the program. If the program crashes under σ_{mut}, then σ_{mut} is added to the output set of crashing schedules (line 6). Additionally, if the fuzzer determines that the execution of σ_{mut} includes new behavior (line 7) then that schedule is added to the working set of schedules for future exploration via mutation (line 8). This loop is repeated, mutating each schedule proportionally to its behavioral novelty score (η_σ in line 3) before moving on to the next schedule in the working set.

A few observations about the grey-box fuzzing template are in order. First, unlike purely random sampling based approaches such as PCT and Partial Order Sampling [32, 241], the algorithm maintains state *across executions* in the form of the corpus of schedules S. It uses this state to guide the next iteration of schedule generation. Further, instead of storing all schedules, the algorithm maintains only a subset of schedules, filtering out schedules with redundant behavior via the `isInteresting()` function. In order to determine what is interesting, the traditional grey-box fuzzing algorithm relies on a notion of (lightweight) code-coverage *feedback* that can be collected as the input schedule is being executed. For more granular feedback, energy assignment (line 3) can further prioritize some schedules. The mutation generation works by first randomly choosing members of the existing corpus and then applying a class of pre-defined operations on them to obtain a

schedule in a neighborhood that is expected to (but may not) be close to the existing corpus. The mutation operation is thus responsible for the core randomness in the exploration, while the feedback and the energy assignment together bias the otherwise random search towards under-explored regions in the space. This bias is achieved indirectly by controlling the frequencies of schedules for mutation.

While the overall fuzzing template is straightforward, there are several key challenges in adapting this template to the domain of concurrent-program interleavings: First, the space of schedules or thread interleavings can be prohibitively large, given that subtle bugs often only manifest in long executions. Second, traditional feedback metrics such as code-coverage may not be fine-grained enough to expose subtle interleavings, warranting a fresh look at the feedback. Third, unlike the traditional setting where the fuzzing algorithm generates inputs that can always be executed, a synthetic schedule may not be feasible, leading to many redundant iterations, slowing the overall testing loop. In the following we give the specifics of how we address these challenges.

Reads-from relation and equivalence. The key to the effectiveness of our approach stems from the *semantic* treatment of concurrent program executions — we identify when two executions are semantically equivalent and systematically avoid exploring multiple interleavings from the same equivalence class, an insight also central to partial order reduction based model checking techniques [181, 62, 6]. We choose *reads-from* information to define this equivalence relation over schedules. Let us first define events and schedules. An event is a tuple $e = \langle id, t, op(x)@\ell \rangle$, where id is the unique identifier of e, t is a thread identifier, op is an operation such as '`r`' (read) or '`w`' (write), x is the object (variable/memory location, etc) that e operates upon and ℓ is the source code location that e corresponds to. A schedule σ is a sequence of events, and we use Events_σ to denote the set of its events. A schedule σ is said to be *feasible* for a program P if P can execute σ. The reads-from function of σ, denoted rf_σ maps each read event e in σ to the corresponding write event $f = \mathsf{rf}_\sigma(e)$ it observes its value from. Two schedules σ_1 and σ_2 are said to be reads-from equivalent, denoted $\sigma_1 \equiv_{\mathsf{rf}} \sigma_2$ if they both observe the same events (i.e., $\mathsf{Events}_{\sigma_1} = \mathsf{Events}_{\sigma_2}$) and the same reads-from function (i.e., $\mathsf{rf}_{\sigma_1} = \mathsf{rf}_{\sigma_2}$). If $\sigma_1 \equiv_{\mathsf{rf}} \sigma_2$, we note that σ_1, σ_2 must observe the same control flow and thus identically expose

any given bug.

Abstract events and schedules. To reduce the search space of program interleavings, we represent schedules by $\equiv_{\mathsf{rf}}$-equivalence class, rather than a concrete sequence of events. We do so by virtue of generating and mutating only *abstract schedules* comprising of *abstract events*. An abstract event is a tuple $ea = op(x)@\ell$ containing an operation op, a memory location x and a code location ℓ. We say that an event $e = \langle id, t, op(x)@\ell \rangle$ instantiates abstract event $ea = op'(x')@\ell'$ if $op' = op, x' = x$ and $\ell' = \ell$. An abstract schedule α is a set of positive and negative reads-from constraints: $\alpha = \alpha^+ \uplus \alpha^-$, where $\alpha^+ = \{ C_1^+, C_2^+, \ldots C_{n_1}^+ \}$ and $\alpha^- = \{ C_1^-, C_2^-, \ldots C_{n_2}^- \}$. Each positive constraint C_i^+ is of the form $C_i^+ = ea_i \xrightarrow{\mathsf{rf}} ea_i'$ and each negative constraint is of the form $C_j^- = ea_j \overset{\mathsf{rf}}{\not\rightarrow} ea_j'$ for some read ea_i, ea_j and write abstract events ea_i', ea_j' (over the same memory location). We say that a (concrete) schedule σ is an *instantiation of* an abstract schedule α if it satisfies the constraints of α, i.e.,

1. for every positive constraint $C_i^+ = ea_i \xrightarrow{\mathsf{rf}} ea_i' \in \alpha$, there are events e_i and e_i' in σ that instantiate ea_i and ea_i' respectively and satisfy $\mathsf{rf}_\sigma(e_i') = e_i$, and

2. for every negative constraint $C_i^- = ea_i \overset{\mathsf{rf}}{\not\rightarrow} ea_i' \in \alpha$, there are no events e_i and e_i' in σ that instantiate ea_i and ea_i' respectively and satisfy $\mathsf{rf}_\sigma(e_i') = e_i$

If $\sigma_1 \equiv_{\mathsf{rf}} \sigma_2$, then either both or none of them are an instantiation of any given abstract schedule α.

Schedule $\sigma = e^{(1)}_{\mathtt{main}} \ldots e^{(\mathtt{n}+1)}_{\mathtt{main}} e^{\mathtt{a}}_{\mathtt{st}_1} e^{\mathtt{b}}_{\mathtt{st}_1} \ldots e^{\mathtt{a}}_{\mathtt{st}_\mathtt{n}} e^{\mathtt{b}}_{\mathtt{st}_\mathtt{n}} e^{\mathtt{a}}_{\mathtt{ct}} e^{\mathtt{b}}_{\mathtt{ct}}$, for example, is a sequence of $3\mathtt{n}+2$ events for the program in Figure 6.1. Here $e^{(i)}_{\mathtt{main}} = \langle i, \mathtt{main}, \mathtt{spawn}(\mathtt{st}_i)@\ell_{15} \rangle$ represents the event that spawns thread $\mathtt{st}_i$, while $e^{\mathtt{a}}_{\mathtt{st}_i}$ and $e^{\mathtt{b}}_{\mathtt{st}_i}$ represent the write to $\mathtt{a}$ and $\mathtt{b}$ respectively by thread $\mathtt{st}_i$. Events $e^{\mathtt{a}}_{\mathtt{ct}}$ and $e^{\mathtt{b}}_{\mathtt{ct}}$ are read events on $\mathtt{a}$ and $\mathtt{b}$ in thread $\mathtt{ct}$. This concrete schedule σ spawns and executes all setter threads before executing the checker thread. The reads-from function maps $\mathsf{rf}_\sigma(e^{\mathtt{a}}_{\mathtt{ct}}) = e^{\mathtt{a}}_{\mathtt{st}_\mathtt{n}}$ and $\mathsf{rf}_\sigma(e^{\mathtt{b}}_{\mathtt{ct}}) = e^{\mathtt{b}}_{\mathtt{st}_\mathtt{n}}$. Now consider the alternate schedule

$$\sigma' = e^{(1)}_{\mathtt{main}} \ldots e^{(\mathtt{n}+1)}_{\mathtt{main}} e^{\mathtt{a}}_{\mathtt{st}_\mathtt{n}} e^{\mathtt{b}}_{\mathtt{st}_\mathtt{n}} e^{\mathtt{a}}_{\mathtt{ct}} e^{\mathtt{b}}_{\mathtt{ct}} e^{\mathtt{a}}_{\mathtt{st}_1} e^{\mathtt{b}}_{\mathtt{st}_1} \ldots e^{\mathtt{a}}_{\mathtt{st}_{\mathtt{n}-1}} e^{\mathtt{b}}_{\mathtt{st}_{\mathtt{n}-1}}$$

that performs all events of the setter thread $\mathtt{st}_\mathtt{n}$ and the checker thread before events of other setter threads. The reads-from function here also maps $\mathsf{rf}_{\sigma'}(e^{\mathtt{a}}_{\mathtt{ct}}) = e^{\mathtt{a}}_{\mathtt{st}_\mathtt{n}}$ and $\mathsf{rf}_{\sigma'}(e^{\mathtt{b}}_{\mathtt{ct}}) = e^{\mathtt{b}}_{\mathtt{st}_\mathtt{n}}$, thus $\sigma \equiv_{\mathsf{rf}} \sigma'$. Thus both *concrete* schedules are instantiations of

same the abstract schedule

$$\alpha_{\mathsf{miss}} = \{\, \mathtt{w(a)}@\ell_3 \xrightarrow{\mathsf{rf}} \mathtt{r(a)}@\ell_7, \quad \mathtt{w(b)}@\ell_4 \xrightarrow{\mathsf{rf}} \mathtt{r(b)}@\ell_7 \,\}$$

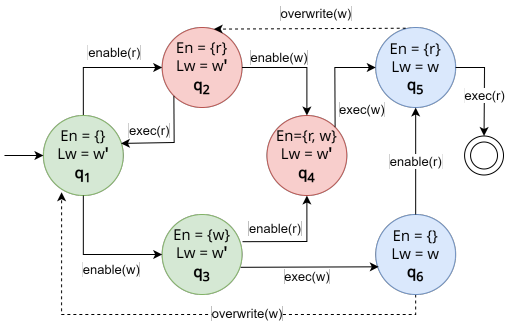
Any instantiation of α_{miss} will not find the assertion violation. In contrast, *all* instantiations of $\alpha_{\text{violation}}$ (outlined in Section 6.1) will witness the bug. Overall, there are only 25 abstract schedules (5 options each for the reads-from constraint on $\mathtt{r(a)}$ and $\mathtt{r(b)}$) despite the exponentially larger number of concrete schedules.

Mutating abstract schedules. The choice of abstract schedules as the search space also naturally allows us to mutate them in a *structured* manner instead of resorting to event-level mutations. By changing the higher-level abstract schedule constraints, each mutation will definitively result in different reads-from behavior if that schedule can be feasibly executed. This notion bears semblance to the concept of structured-input fuzzing [184, 22, 74]. The `mutateSchedule()` function in our approach is implemented in two steps. First we randomly choose one of the four mutation operators:

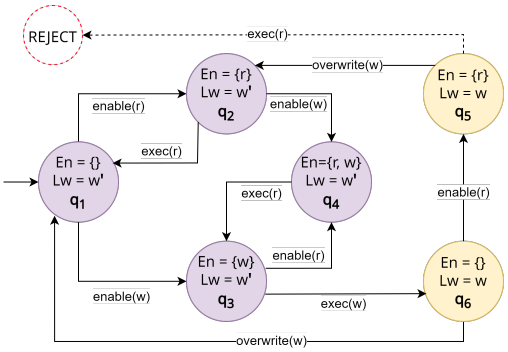
$$\begin{aligned}
&\textsc{insert}(\alpha, C) = \alpha \cup \{\, C \,\} \\
&\textsc{swap}(\alpha, C_1, C_2) = (\alpha \setminus \{\, C_1 \,\}) \cup \{\, C_2 \,\} \\
&\textsc{delete}(\alpha, C) = \alpha \setminus \{\, C \,\} \\
&\textsc{negate}(\alpha, C) = \textsc{swap}(\alpha, C, \neg C)
\end{aligned}$$

Here, for constraint $C = \mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$ (resp. $C = \mathtt{w} \overset{\mathsf{rf}}{\not\to} \mathtt{r}$), we use $\neg C$ to denote the negated constraint $\mathtt{w} \overset{\mathsf{rf}}{\not\to} \mathtt{r}$ (resp. $C = \mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$). Then we randomly pick potentially conflicting events from E, the set of all events observed, to form constraints $C_1 \ldots C_n$ which are needed by these mutation operators.

Reads-from feedback. In addition to control-flow feedback used by grey-box input-fuzzers, we employ reads-from information as concurrency-aware feedback. We remark that such feedback is strictly more granular than control flow information. Our implementation of `isInteresting(`σ_{mut}, S`)` from Algorithm 3 returns true if either (a) there is a reads-from pair (e_1, e_2) in σ_{mut} such that no other the abstract schedules in S instantiates it, or, (b) if the schedule results in a crash (mirroring the behavior of input-level grey-box fuzzers [9]). Additionally, we utilize a cut-off



(a) Scheduling a positive constraint $\mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$. Blue states prioritize `r` and deprioritize any write other than `w`. Red states prioritize `w` and deprioritize `r`. Green states represent no prioritization.



(b) Scheduling a negative constraint $\mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$ Purple states prioritize `r` and deprioritize `w`. Yellow states prioritize all writes apart from `w` and deprioritize `r`.

Figure 6.2: State-Transition for scheduling reads-from constraints. En and Lw represent the set of enabled events and last write event on the same location as `w`. Solid arrows indicate actions that facilitate the constraint. Dashed arrows indicate actions that inhibit the constraint. Label enable(op) indicates that op is ready to be executed. Label exec(op) indicates that the action executes op. Label overwrite(w) indicates executing another write $\mathtt{w}'$ on same variable as `w`.

exponential power-schedule [27] to further bias exploration towards rarely observed reads-from constraints, discussed in Section 6.3.1.

Proactive Scheduling of Reads-from Constraints. Our algorithm RFF utilizes feedback and mutations to arrive at abstract schedules. However, given such an abstract schedule, it is not trivial to find a *concrete* execution which satisfies those scheduling constraints. Indeed, a generated abstract schedule may not even be feasible in the program at all. We design a *proactive reads-from scheduler* to drive the program to meet the constraints of the abstract scheduler. The scheduling algorithm works by carefully determining which events to execute next when multiple *enabled* events are possible.

To push the execution towards satisfying abstract scheduling constraints, we use a greedy scheduling algorithm to delay or immediately execute events involved in these constraints. Such a priority change ensures that the event will *always* (or never) be chosen next over other enabled events that are not in the abstract schedule. For example, to satisfy $\mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$, we can boost the priority of a write event $e_{\mathtt{w}}$ that instantiates the (abstract) event `w`, and then subsequently a read event $e_{\mathtt{r}}$ that

instantiates $\mathtt{r}$, to satisfy the reads-from constraint. However, simply boosting and lowering the priorities when the relevant events are *both* enabled is often inadequate to ensure that a desired abstract schedule constraint is met. Relevant events are often widely separated in execution traces and thus likely will not be simultaneously enabled without further intervention. Therefore, we maintain a succinct state machine for each constraint in the abstract schedule to determine how priorities of relevant events should be proactively adjusted. A schematic description of the state machines for positive ($\mathtt{w} \xrightarrow{\mathsf{rf}} \mathtt{r}$) and negative ($\mathtt{w} \overset{\mathsf{rf}}{\not\rightarrow} \mathtt{r}$) reads-from constraints are depicted in Figure 6.2a and Figure 6.2b respectively.

Looking first at Figure 6.2a, we see that, once $\mathtt{r}$ has been enabled but not yet executed (states $\mathtt{q}_2$ and $\mathtt{q}_4$ in red), we lower the priority for the thread of the read. This means the read *will not be executed* until the corresponding write has been observed (state $\mathtt{q}_5$) or all other threads are blocked. If the scheduler is forced to execute $\mathtt{r}$ in this state, it reverts these priority changes (state $\mathtt{q}_1$). When $\mathtt{r}$ has been enabled, the scheduler also increases the priority for the corresponding write $\mathtt{w}$ while $\mathtt{r}$ is being delayed (states $\mathtt{q}_2$ and $\mathtt{q}_4$ in red). Once the relevant write $\mathtt{w}$ has executed (states $\mathtt{q}_5$ and $\mathtt{q}_6$ in blue), the scheduler lowers the priority for all $\mathtt{w}'$ which access the same memory location to avoid overwriting the desired write. At this point, the scheduler also prioritizes the relevant read $\mathtt{r}$ to satisfy the constraint as soon as $\mathtt{r}$ becomes enabled. Positive constraints are implicitly existentially quantified — we consider such a constraint satisfied after it has been executed at least once on any two threads; if so the scheduler removes it from the abstract schedule for the remainder of the execution.

In Figure 6.2b, for a negated constraint, our scheduler gives the read $\mathtt{r}$ higher priority and deprioritizes the write $\mathtt{w}$ as long as another write $\mathtt{w}'$ was the last observed (states $\mathtt{q}_1, \mathtt{q}_2, \mathtt{q}_3, \mathtt{q}_4$, purple). This will greedily execute the relevant read whenever the reads-from can be avoided while prolonging this window for as long as possible. Once $\mathtt{w}$ has been observed (states $\mathtt{q}_5$ and $\mathtt{q}_6$, yellow), then the scheduler deprioritizes $\mathtt{r}$ to avoid violating the negated constraint. It also prioritizes any other write to that location, $\mathtt{w}'$, in an effort to overwrite $\mathtt{w}$ and bring the system back into a state where $\mathtt{r}$ can be safely executed. Negative constraints are universally quantified — we only consider these constraints satisfied if the entire execution does not contain the corresponding reads-from pair. Finally, the REJECT state can occur if a negated

constraint is unavoidably violated, e.g., if only a single thread is actually runnable.

Our discussion of the pro-active scheduling algorithm so far describes how to (de)prioritize events relevant to the abstract schedule, but an additional mechanism is still needed to make scheduling decisions in the absence of relevant events or in the case of multiple conflicting constraints. Here we employ randomization, à la partial order sampling (POS) [241]. The POS algorithm, first assigns each event a random score if it does not already have one. It then picks the event with the highest score to execute next, resetting that event's score along with the scores of any racing events. POS has shown promising empirical results [241] [240] and, like PCT, provides a bound on the probability a particular interleaving will be exercised. We modify the POS algorithm with our greedy scheduler, such that we adopt POS *only* when our greedy scheduler cannot make a definitive scheduling decision. Our proactive reads-from scheduler, gracefully degrades to baseline POS in the absence of abstract schedule constraints or with multiple competing constraints.

Alternative Design Choices. In initial stages of developing RFF, we investigated a fuzz-testing like (i.e., partially stateful) technique that treats concrete schedules as first class inputs and generates them using a random search guided by reads-from "coverage". Unfortunately, we ruled this approach out because it was less effective than even random testing (POS) in our early evaluations. When the schedule is concrete, large fraction of the mutations of an observed schedule are infeasible. Further, the space overhead of storing concrete schedules (or a reasonable subset of them) also becomes a performance bottleneck. This is one of the key challenges that our proposed notion of abstract schedules aims to address — abstract schedules only partially describe an execution and thus some feasible instantiation of them can usually be discovered using our proactive scheduler. Second, the space of concrete schedules (or even Mazurkiewicz partial orders [241]) is much larger than the space of abstract schedules, allowing us to sample the effective search space without redundancy.

6.3 RFF Architecture and Implementation

For scheduling control in RFF, we take inspiration from prior works on *Deterministic Multi-Threading* (DMT), such as Kendo [174] and CoreDet [19]. Here, the

basic idea is to explicitly *serialize* the thread interleaving using instrumentation, as illustrated below:

```
1 void on_event(event_t id) {
2   mutex_lock(&GLOBAL);
3   thread_t *T = schedule(id);
4   if (T != self()) {
5     cond_signal(&T->wake);
6     cond_wait(&self()->wake, &GLOBAL);
7   }
8   record(self(), id);
9   mutex_unlock(&GLOBAL);
10 }
```

```
1   on_event(id);  // Instrumentation injects this function call
2   x = *y   or   *x = y   or   (thread primitive)
```

Each *visible* event (i.e., memory operation, concurrency primitive) is instrumented with a call to a `on_event()` which uses a global mutex (`GLOBAL`) to serialize all threads. Furthermore, each thread T is associated with a thread-local condition variable (`T->wake`) to control when T can run. When invoked, the `on_event()` function determines which thread to run next via the `schedule()` routine. The schedule routine implements the scheduler proper, as discussed below. If a different thread is selected, it will be signalled (woken) and the current thread will be put to sleep. Although serialization removes the performance benefits of parallelism *per instance*, it makes it possible to run multiple *instances* of the fuzzer in order to achieve full resource utilization.

The scheduling algorithm is implemented as follows:

1. First, the thread must be *runnable* (i.e., not blocked).
2. Secondly, over the set of runnable threads, the Figure 6.2a and 6.2b algorithms are used to bias thread selection towards the abstract schedule. If executing e on thread T violates the abstract schedule, then a different thread should be selected if possible.
3. Finally, more than one thread may still be selectable. For this we revert to Partial Order Sampling (POS) over a predetermined random seed.

If no thread is runnable then a *deadlock* has been detected. Otherwise, `schedule()` will return a valid runnable thread with a bias towards the abstract schedule.

Deterministic Multi-Threading. Reproducibility is a key challenge in finding and especially *fixing* concurrency related bugs, as the default scheduler for all mainstream operating systems is non-deterministic. Unfortunately, Deterministic Multi-Threading (DMT) at the granularity of memory operations is known to incur high overheads [135]. In the context of our fork-based fuzz testing driver, however, the overheads of the `fork()` system call tends to dominate over instrumentation costs [96, 15, 69]. To determine the set of runnable threads, we borrow from earlier DMT works by intercepting operations that can potentially block (`mutex_lock`, `cond_wait`, `read`, `poll`, etc.). Our runtime also hooks pthread functions to track thread creation and destruction. To ensure that events observed by our tool are deterministic across executions, we disable address space layout randomization (ASLR) during a fuzz campaign.

Instrumentation. Our implementation uses low overhead static binary rewriting with E9Patch [55, 69] in order to instrument the program more efficiently than dynamic instrumentation used in prior work [239]. While not a primary focus of this work, this means that we can also test concurrent programs without access to their source code.

Memory Model. Our implementation assumes sequential consistency, following the recommendations of the SCTBench authors [199], as this benchmark makes up a large component of our evaluation for user-space programs.

An overview of system architecture of `RFF` is shown in Figure 6.3. Here, the basic system architecture consists of two main components: (1) an *instrumented Program Under Test* (PUT) in combination with a deterministic user-mode scheduler (`libsched.so`), and (2) a *schedule fuzzer* (`RFF`), based on a modified version of AFL [9]. Unlike traditional fuzzers over the input space, `RFF` fuzzes the PUT over the *schedule space* for a given fixed input. To do so, the `RFF` fuzzer maintains a corpus of "interesting" abstract schedules $\{\alpha_1, \alpha_2, \dots\}$. For each iteration of the fuzzing loop, new abstract schedule α_{mut} is generated by mutating one (or more) existing schedules from the corpus. The mutant abstract schedule consists of a set of (possibly negated) reads-from relations over the events observed from the corpus,

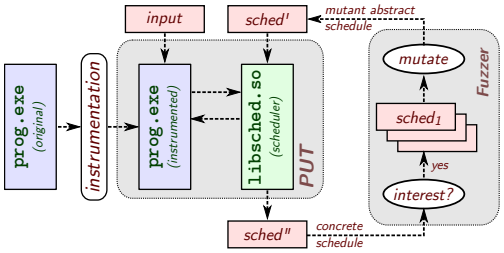


Figure 6.3: Overall system architecture of RFF. The two main components are (1) the instrumented *Program Under Test* (PUT), and (2) the *schedule fuzzer*. The workflow takes in an uninstrumented program (`prog.exe`) and a test *input*, and outputs a set of observed racey schedules (if any). The PUT is executed with mutant schedules, with observed schedules as feedback.

which may or may not be feasible. Next, the instrumented PUT is executed using a deterministic user-mode scheduler (`libsched.so`). The scheduler attempts to bias the actual thread interleaving into satisfying the constraints specified by α_{mut}, based on the proactive reads-from scheduler (see Figures 6.2a and 6.2b). After the instrumented program executes, a concrete schedule (σ_{mut}) representing the actual thread interleaving will be observed. The trace of this concrete schedule is analyzed for interesting behavior. If the concrete schedule σ_{mut} is deemed interesting according to the feedback metrics discussed in Section 6.2, its corresponding *abstract* schedule α_{mut} is saved to the schedule corpus. Note that our design assumes that the thread-interleaving can be sufficiently controlled. Thus it is necessary to manage thread interleavings at the event level, including individual memory and thread primitives. The (default) kernel-mode scheduler is coarse-grained and non-deterministic, so is not suitable. Instead we implement our own fine-grained (user-mode) scheduler.

6.3.1 Schedule Fuzzer

The second main component is the schedule fuzzer itself (RFF), which is implemented as a modification of AFL [9]. AFL is an industry-strength fuzzing tool which has found many critical vulnerabilities in prominent open source software projects. We use AFL's fork-based testing driver to efficiently execute the program without expensive `execve` calls. Additionally, we utilize AFL's shared memory region for efficient inter-process communication between the instrumented program and the test execution driver. We adapt AFL to the domain of schedules by changing

its fuzzing loop to reflect Algorithm 3.

Power Schedule.

The `PickNextAndAssignEnergy()` function in Algorithm 3 additionally also computes an energy value η_α for abstract schedule α to dictate how many times α will be used as a base to create additional schedules via `mutateSchedule()`. As a result, it allows for a more fine-grained prioritization of particular schedules as compared to the binary feedback of `isInteresting()`. This function is shown below:

$$p(\alpha) = \begin{cases} 0 & \text{if } f(\alpha) > \mu \\ min(\frac{\gamma(\alpha)}{\beta} * 2^{s(\alpha)},\ M) & \text{otherwise} \end{cases} \quad \mu = \frac{\sum_{\alpha \in S^+} f(\alpha)}{|S^+|}$$

For example, even if both schedule α_1 and α_2 observe new interesting behavior, we can prioritize the neighborhood around α_1 over that surrounding α_2. In the preceding, $s(\alpha)$ is the number of times the abstract schedule α has been chosen since it was last skipped, $\gamma(\alpha)$ is the performance score for the schedule, β is a hyperparameter, and $f(\alpha)$ is the frequency that the reads-from combination exercised by α has been observed. M is the maximum iterations per fuzzing stage. S^+ is the working set of schedules deemed interesting by the fuzzer. According to this energy distribution, schedules which produce combinations of reads-froms which are more common than average are given zero energy (i.e. skipped entirely). Under-explored reads-from combinations are given exponentially increasing energy until they are skipped. In doing so, we explore *under-explored* regions of the scheduling space rapidly until they *become* over-explored.

6.4 Evaluation

In this section, we discuss how we evaluate our concurrency testing approach. `RFF` is inherently a randomized testing approach and its efficacy can be best determined by the diversity of the behaviors it explores. Specially, its ability to cover the search space as quickly and evenly as possible. To quantify this exploration we use `RFF` to find bugs in real-world applications and examples from existing concurrency-bug benchmarks; we also examine the distribution of schedules explored over a given

partitioning of the schedule space.

Research Questions. The main goal of our evaluation is to answer the following three research questions:

RQ1 Is our tool more effective at finding bugs than other state-of-the-art concurrency testing techniques?

RQ2 How does our focus on the abstract schedule space contribute to the effectiveness of our approach?

RQ3 How evenly distributed is our search in the space of reads-froms?

RQ4 Is an alternative approach leveraging reads-from information as effective as our fuzzing-inspired biased random search?

6.4.1 Experimental Setup

Benchmarks. To evaluate the effectiveness of our approach, we utilize two widely used benchmarks of (pthread) concurrent programs: SCTBench [199] and ConVul [35]. We choose these benchmarks for their availability and breadth, covering a wide variety of bug-types, workloads, and applications. Additionally, we selected our evaluation benchmarks based on the standards established in the concurrency testing community, to allow fair comparison with prior work. The programs in SCTBench are derived from existing concurrency-bug benchmarking suites and real-world programs containing concurrency bugs. SCTBench encompasses several smaller benchmarks used in prior work such as file downloading and compression tools [238], implementations of work stealing queues [161], small multi-threaded algorithms from [46], JavaScript engines for modern web browsers [94], parallel numeric computing applications [231], and other sample programs [236]. Some programs, such as the `RADBench` programs or `CB/pbzip2` consist of several thousand lines of production C/C++ code. We use the version of SCTBench from [226] (commit `cb8deb9`), omitting two programs due to incompatibilities with our instrumentation framework. The ConVul benchmark contains 10 programs with *critical* memory and pointer

related concurrency vulnerabilities in real-world programs [35]. Together these benchmarks provide a large, diverse set of programs for evaluation.

Bugs. Of the 49 programs evaluated in these benchmarks, most (34) bugs are manifested as assertion violations. Four programs contain deadlocks. Lastly, 13 programs contain memory safety issues related to concurrency. Our evaluation criteria is the number of schedules needed to find the first instance of a bug in each program. We choose this as our criteria to avoid issues with bugs masking other failures that might occur later in that same execution. For bug detection, we use only `RFF`'s built-in deadlock-detector and a simple crash oracle (e.g. assertion failure, segmentation fault etc.), following the advice of [199].[3]

Baselines. To determine whether our approach is competitive with the state-of-the-art in controlled concurrency testing and model checking, we compare with PERIOD [226] (commit `cb8deb9`), PCT [32] and GenMC (`v0.9`)[118, 117]. PERIOD is a systematic testing tool that explores schedules in parallel using Linux deadline-based task scheduling and GenMC is a stateless model checker that enumerates all possible behaviors of a program to find bugs. PCT is a well known randomized concurrency testing algorithm with probabilistic bounds for finding bugs based on the bug's *depth*. We implement PCT ($depth = 3$)[4] within our framework to provide a fair comparison in terms of number of instrumented events and execution overhead. We also conduct an ablation study of `RFF` without its key components: abstract schedule constraints and reads-from feedback. This includes a comparison with our own implementation of Partial Order Sampling [241] as a baseline. In addition, we implement a custom Q-Learning [182] algorithm within our concurrency testing framework to evaluate an alternative approach for leveraging reads-from information for concurrency testing

Unfortunately the implementations for state-of-the-art concurrency-aware fuzzers, MUZZ [41] and CONZZER [100], are not publicly available. We reached out to the authors of these works during the early development of `RFF`, but were not able to obtain access to either implementation. Other available concurrency fuzzers [75, 97, 234] are specific to the Linux kernel and thus incompatible with the user-space

[3]Many of the programs in SCTBench contain multiple data-races which would otherwise mask more challenging bugs if considered.

[4]A depth of 3 provided the best results for PCT in [199]

benchmark programs in our evaluation, but we will examine them more closely in Chapter 8.

We attempted to implement MUZZ's approach to interleaving exploration, namely (1) changing OS thread priorities on creation and (2) modifying AFL's instrumentation to give per-thread edge coverage on each execution. For (1) we set each thread priority on entry to a random value with the `sched_setscheduler` system call. For (2) we include the thread ID in the hash for each edge recorded by the coverage instrumentation of AFL. However, we found that even on simple benchmark programs, this implementation was not able to trigger bugs in practice. On the example program from Figure 1, it was not able to find the bug after millions of executions on only the three-thread version of this program. CONZZER has a far more complex implementation and we are not confident of re-implementing and replicating CONZZER system.

Experimental Environment. We conduct all experiments on a Intel(R) Xeon(R) Gold 6258R CPU @ 2.70GHz (Model 85) running Ubuntu 20.04 in a Docker container (kernel 5.4.0-155-generic) without disabling Spectre mitigations. We run each benchmark program for 5 minutes as this is sufficient for at least one tool to discover a bug on 47/49 benchmark programs. This departs from prior concurrency testing evaluations in favor of recommendations found in literature for input-space exploration in grey-box fuzzing [113]. Using a timeout rather than a bounded number of schedules allows us to compare each approach while taking into account the overhead introduced by instrumentation, schedule serialization and more sophisticated analyses. We repeat each run 20 times for each tool to account for variance in the results, excepting GenMC, which is deterministic. We did observe some variance in the output of PERIOD, and we incorporate this variance across runs into our analysis. We attribute the variance to a timeout for each period inherent in their deadline-based scheduling implementation, which may result in missed orderings if it expires. Despite this, results from our evaluation of PERIOD agree strongly reported results on these benchmarks in [226].

Data Availability We make `RFF`, along with all data and software (with pinned versions) necessary to reproduce openly available in our artifact:

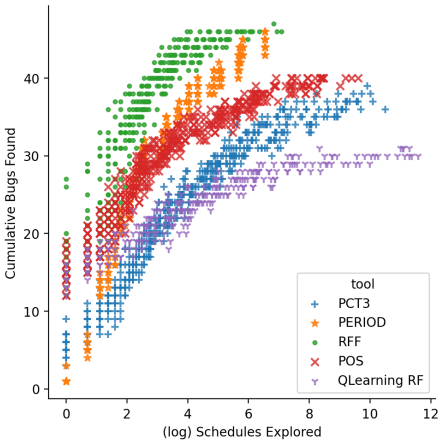


Figure 6.4: Total Bugs Discovered After Log(# Schedules) Across All Trials (higher is better)[5]

`https://doi.org/10.6084/m9.figshare.23911299`

6.4.2 Comparison with the State-of-the-Art (RQ1)

We applied PERIOD, Partial Order Sampling (POS), GenMC and `RFF` on SCTBench [199] and ConVul benchmarks [35]. Our primary metric for assessing the effectiveness of our tool is the number of schedules to the first bug found in each program. We include a table with the mean number schedules-to-bug for each tool and program in Section 6.7.

Unfortunately, GenMC does not run on 36 of the evaluated programs. In most cases, this seems to be a limitation of the GenMC implementation, which supports only a subset of valid LLVM IR programs. On the programs which GenMC is able to check, it explores fewer schedules than `RFF`'s average schedules-to-bug on 5 programs. However, `RFF` explores fewer schedules on average relative to GenMC on 7 programs. Overall, `RFF` appears to be competitive with state-of-the-art model checkers in terms of bugs found per schedule, while also being more broadly applicable to real-world programs.

PCT performs well, finding approximately 37 bugs on average. However, `RFF` finds bugs in the most programs on average ($\mu = 46.1$), followed closely by PERIOD ($\mu = 44.6$). Despite the narrow gap, this difference in the number of bugs found over 20 trials is statistically significant by the Mann-Whitney U-test (p-value < 0.001). `RFF` consistently finds bugs in 46/49 programs and finds bug in the 47th program in

some fuzz campaigns (hence $\mu = 46.1$). To visualize the bugs found per schedule executed for each tool, we plot the cumulative bugs found as the (log) number of total schedules increases across all programs in Figure 6.4. This graph shows the number of bugs found by each tool after a certain number of schedules, rather than only counting the number of bugs found at the end of our timeout. For each of our 20 trials, we track the schedule number M at which the Nth bug was computed on each benchmark. This figure plots a point at (M, N) for each bug found, in ascending order.[5] Here we see that RFF consistently finds more bugs per schedule than either PERIOD or POS at *all* schedule counts measured in our evaluation. In other words, apart from finding more bugs than prior work, RFF generally finds those bugs in fewer schedules. In fact, our tool finds bugs in significantly fewer schedules than PERIOD on 30/49 programs, whereas PERIOD finds bugs in fewer schedules for only 9/49 programs.[6] We attribute the success of our tool over PERIOD to our abstract schedule structure. PERIOD explores all possible *orderings* of synchronization and shared-access events below a given depth bound. Many different orderings of these events correspond to the same "reads-from" relations.

RFF finds more bugs in fewer schedules than state-of-the-art concurrency testing tools.

6.4.3 Contribution of the Abstract Schedule (RQ2)

We compare RFF with Partial Order Sampling POS because our scheduling algorithm leverages POS on the space of *abstract schedules* while the original POS operates on *concrete schedules*. We show the results of this comparison on the SCTBench [199] and ConVul benchmarks in Figure 6.4. We can see from the average number of bugs found by each tool that the abstract schedule structure improves the bug-finding ability of our tool significantly. By searching the space of reads-froms, we are able to find approximately six more bugs on average! The cumulative plot in Figure 6.4 similarly shows a strong effect from the schedule structure. Here we

[5] For example, using only two trials and three programs, we might find bugs after $[3, 5, 7]$ schedules and $[3, 6, 9]$ schedules respectively. We then plot a point for the cumulative number of bugs found within each trial. In this case, a point at $(3, 1)$, $(5, 2)$ and $(7, 3)$ for the first trial and $(3, 1)$, $(6, 2)$ and $(9, 3)$ for the second trial.

[6] Significance determined by the log-rank test [145] ($p < 0.05$)

can see that our tool with abstract schedule constraints (green) finds slightly more bugs relative to POS (red) after exploring a small number of schedules. This initial gap widens as the number of schedules explored increases. In other words, POS is comparable in efficacy to our more structured search for finding relatively easy bugs (found in small numbers of schedules). For harder bugs, however, POS fails to find them as quickly or at all.

We observed that POS alone fails on several programs with larger number of threads, such as `reorder_100`, `reorder_50` and `twostage_50`, etc. As discussed in Section 6.1, the probability for POS to trigger a particular interleaving in these examples is exponentially low in the number of threads. `RFF` overcomes this problem by defining events in terms of their effects (reads-from) rather than which thread executed them. Looking more closely at each approach, we find that a structured random search finds the bug in significantly fewer schedules on 16/49 programs (via the log-rank test). In contrast POS does not find the bug in significantly fewer schedules on *any* of the benchmark programs. This implies that our tool gracefully degrades to POS in the absence of abstract schedule constraints, and the schedule structure improves POS.

Searching over the reduced space of abstract reads-from schedules is crucial for `RFF`'s bug-finding effectiveness. `RFF` provides a strict improvement over Partial Order Sampling in practice.

6.4.4 Exploration of the Schedule Space (RQ3)

To assess `RFF`'s ability to evenly explore the space of *meaningfully different* interleavings, we examine the number of times each unique *combination* of "reads-from" relation is observed by `RFF` during a fuzz campaign. Figure 6.5 shows the (log) frequency each combination of reads-from in the `SafeStack` program was executed after 10000 schedules. We choose this program because it is the most complex in either benchmark suite evaluated, both in terms of bug depth and difficulty. If we expect a bug to be equally likely to occur in any of the reads-from combinations, then a strategy which explores each individual combination evenly will be more successful. Here we can see that without grey-box feedback (top), there is a heavily

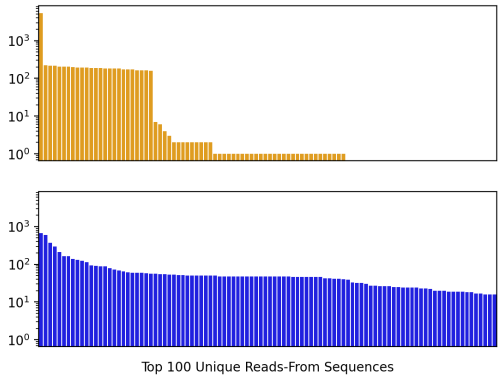


Figure 6.5: Log-scale frequency of reads-from sequences observed by POS (top) and Sect (bottom) in 10000 executions of the `SafeStack` program

skewed distribution. A single reads-from sequence comprises more than 50% of all executions! Including grey-box feedback (bottom) results in a more even exploration of the reads-from space. We attribute this success to our prioritization of rarely observed reads-from sequences via a suitable power-schedule, which was described in Section 6.3.1.

Figure 6.5 (top) shows that random exploration alone results in a tendency to execute some reads-from sequences exponentially more than others. By incorporating feedback via our power schedule, we can bias the search towards under-explored sequences.

Grey-box feedback promotes an even exploration of the schedule space and can lead to significant improvements on certain programs.

6.4.5 Reads-from Testing via Q-Learning (RQ4)

To further evaluate the effectiveness of our fuzzing-inspired approach, we compare it with a different framework leveraging the same reads-from information: Q-learning [182]. Q-Learning is a well-understood reinforcement learning technique which maximizes the reward over several actions which can be taken from each of a many successive states. To do so, a *Q-function* maps each state-action pair to a score: $Q : S \times A \to \mathbb{R}$. This score, $Q(s_t, a_j)$ represents the probability with which the action a_i should be taken from the state s_t. After sampling over these probabilities to determine the next action, the algorithm updates the Q-score for the state-action pair

via a *reward function*. This technique can be applied to the problem of concurrency testing by considering each scheduling decision to be an action.

Our goal is to investigate Q-Learning as an *alternative framework* to leverage the reads-from relation, which is critical in uncovering both bugs and new control flows in concurrent programs. Thus, *in contrast* to prior applications of Q-Learning to concurrency testing [160], we use the reads-from relation to distinguish states in our Q-Learning implementation. More specifically, we represent the state of a partial execution of the first t events in a concrete schedule $\sigma_{1...t}$ as a running commutative hash of the N reads-from event pairs observed so far in that execution:

$$h((e_{w_1}, e_{r_1}),\ h((e_{w_1}, e_{r_1}),\ \ldots\ , (e_{w_N}, e_{r_N})\ )))$$

for each pair (e_{w_i}, e_{r_i}) such that $\mathsf{rf}(e_{r_i}) = e_{w_i}$. We utilize a constant negative reward function for observed state-action pairs, as in [160]. Looking at Figure 6.4, we can see that our instantiation of "Reads-From Q-Learning" (Q-Learning RF) is not as effective in utilizing the reads-from relation to find bugs on our two datasets as RFF. In total it finds only about 30.2 bugs on average relative to RFF's 44 bugs. Additionally, RFF finds bugs in significantly fewer schedules on 30 of the 49 programs evaluated. However, we do note that the Q-Learning RF approach consistently finds the bug on the first trial in more instances than any other tool (13 programs). We attribute this one-shot success to the reinforcement learning taking place on partial traces. In comparison, RFF considers its grey-box feedback only after a complete execution.

Grey-box schedule fuzzing provides an effective framework to leverage the reads-from relation, as compared to our instantiation of a reads-from based Q-Learning approach.

6.5 Threats To Validity

We mitigate selection bias in our choice of tools and benchmarks, a threat to *internal validity* by evaluating the reported state-of-the-art tools on the most widely used benchmarks in this domain. There is a risk that these results do not generalize beyond our evaluated dataset, a threat to *external validity*. We believe

that our evaluation of 49 programs –real-world and hand-crafted– on two well-known benchmarks minimizes this risk. Our approach is intended to effectively search the space of interleavings for bugs in concurrent programs. We reduce the threat of *construct validity* by directly measuring the number of bugs found in the benchmarks evaluated. Finally, we conduct an ablation study against POS and a comparison with another algorithmic framework in Q-Learning to confirm the impact of key aspects of our approach and avoid threats to *conclusion validity*.

6.6 Discussion

In this chapter, we have proposed a new approach which effectively searches the interleaving space of concurrent programs for bugs. To formulate this approach, we took inspiration from the success of biased random search approaches such as grey-box fuzzing in sequential programs. By leveraging the reads-from relation, our approach both reduces the search space and guides the search towards under-explored regions of the search space (of interleavings). We believe that biased-random search presents a *pragmatic* middle ground between purely randomized search algorithms and heavy-weight systematic testing via model checkers.

6.7 Mean Number of Schedules to 1st Bug

Benchmark/program	PCT3	PERIOD	RFF	POS	QLearning RF	GenMC
CB/aget-bug2	**1 ± 0**	9 ± 0	**1 ± 0**	**1 ± 0**	**1 ± 0**	Error
CB/pbzip2-0.9.4	-	**45 ± 6***	**2 ± 0***	-	-	Error
CB/stringbuffer-jdk1.4	195 ± 174	27 ± 37	**15 ± 18**	**18 ± 23**	1405 ± 1592	Error
CS/account	9 ± 7	10 ± 0	**1 ± 0**	**1 ± 0**	6 ± 8	5
CS/bluetooth_driver	161 ± 162	**9 ± 0**	45 ± 35	72 ± 79	155 ± 154	4
CS/carter01	5 ± 4	4 ± 1*†	2 ± 1	2 ± 1	**1 ± 0**	4†
CS/circular_buffer	5 ± 4	3 ± 0	**2 ± 1**	**2 ± 1**	**2 ± 1**	8
CS/deadlock01	20 ± 20	3 ± 0†	5 ± 4	4 ± 3	**1 ± 0**	3†
CS/lazy01	10 ± 6	7 ± 2	**6 ± 6**	**5 ± 4**	**12 ± 15**	5
CS/queue	12 ± 14	4 ± 1	**1 ± 0**	**1 ± 0**	**1 ± 0**	22
CS/reorder_10	2356 ± 2302	27 ± 0	**6 ± 4**	-	-	Error
CS/reorder_100	7447 ± 0*	297 ± 0	**6 ± 4**	-	-	Error
CS/reorder_20	2128 ± 2284	39 ± 0	**6 ± 4**	-	-	Error
CS/reorder_3	241 ± 336	**6 ± 0**	**7 ± 5**	223 ± 166	45843 ± 32338*	Error
CS/reorder_4	395 ± 320	9 ± 0	**6 ± 5**	1464 ± 1829	-	Error
CS/reorder_5	1126 ± 1045	12 ± 0	**6 ± 4**	4377 ± 4208*	-	Error
CS/reorder_50	12346 ± 6682*	129 ± 0	**6 ± 4**	-	-	Error
CS/stack	2 ± 2	8 ± 0	2 ± 1	2 ± 2	**1 ± 0**	20
CS/token_ring	8 ± 6	**2 ± 0**	5 ± 5	7 ± 5	12 ± 12	14
CS/twostage	**9 ± 9**	**4 ± 0**	8 ± 7	15 ± 16	336 ± 501	3
CS/twostage_100	3888 ± 3473*	690 ± 0	**56 ± 71**	-	-	Error
CS/twostage_20	188 ± 168	76 ± 0	**22 ± 19**	185 ± 215	-	Error
CS/twostage_50	849 ± 870	286 ± 0	**35 ± 27**	1984 ± 1238*	-	Error
CS/wronglock	88 ± 98	4 ± 2	**1 ± 0**	**1 ± 0**	37 ± 32	3
CS/wronglock_3	40 ± 36	5 ± 1	**1 ± 0**	**1 ± 0**	37 ± 32	Error
Chess/InterlockedWorkStealQueue	24 ± 19*	57 ± 0	**1 ± 0**	**1 ± 0**	-	Error
Chess/InterlockedWorkStealQueueWithState	16 ± 0*	224 ± 80	**7 ± 6**	**9 ± 9**	16 ± 14	Error
Chess/StateWorkStealQueue	12 ± 0*	249 ± 101	**1 ± 0**	**1 ± 0**	-	Error
Chess/WorkStealQueue	**12 ± 14**	57 ± 0	**10 ± 8**	**10 ± 9**	-	Error
ConVul-CVE-Benchmarks/CVE-2009-3547	6 ± 5	2 ± 0	**1 ± 0**	**1 ± 0**	**1 ± 0**	Error
ConVul-CVE-Benchmarks/CVE-2011-2183	9 ± 9	3 ± 0	2 ± 2	2 ± 1	**1 ± 0**	Error
ConVul-CVE-Benchmarks/CVE-2013-1792	87 ± 65	**13 ± 0**	**23 ± 43**	50 ± 62	388 ± 361	1
ConVul-CVE-Benchmarks/CVE-2015-7550	8 ± 7	3 ± 0	6 ± 5	7 ± 7	**1 ± 0**	Error
ConVul-CVE-Benchmarks/CVE-2016-1972	-	**3 ± 0***	39 ± 29	86 ± 78	74 ± 39*	Error
ConVul-CVE-Benchmarks/CVE-2016-1973	8 ± 5	6 ± 0	**3 ± 3**	7 ± 6	5947 ± 6063	Error
ConVul-CVE-Benchmarks/CVE-2016-7911	16 ± 13	3 ± 0	13 ± 10	12 ± 11	**1 ± 0**	Error
ConVul-CVE-Benchmarks/CVE-2016-9806	**4 ± 3**	**6 ± 0**	11 ± 8	14 ± 10	554 ± 577	Error
ConVul-CVE-Benchmarks/CVE-2017-15265	-	**11 ± 0**	36 ± 39	-	-	Error
ConVul-CVE-Benchmarks/CVE-2017-6346	15 ± 11	5 ± 0	5 ± 4	13 ± 14	**1 ± 0**	Error
Inspect_benchmarks/boundedBuffer	15 ± 16	**8 ± 7***	**8 ± 7**	**6 ± 5**	14 ± 13	Error
Inspect_benchmarks/ctrace-test	**1 ± 0**	3 ± 0	**1 ± 0**	**1 ± 0**	**1 ± 0**	1
Inspect_benchmarks/qsort_mt	3838 ± 4458	**27 ± 0**	322 ± 344	646 ± 753	-	Error
SafeStack	-	-	-	-	-	Error
Splash2/barnes	-	**2 ± 0**	**3 ± 3**	**2 ± 2**	**2 ± 1**	Error
Splash2/fft	**1 ± 0**	2 ± 0	**1 ± 0**	**1 ± 0**	**1 ± 0**	Error
Splash2/lu	-	2 ± 1	**1 ± 0**	**1 ± 0**	47 ± 38	Error
RADBench/bug4	15599 ± 9907*	-	**163 ± 151**	**216 ± 209**	-	Error
RADBench/bug5	-	-	-	-	-	Error
RADBench/bug6	61 ± 49	24 ± 0†	4 ± 3	11 ± 8	**1 ± 0**	Error

Each cell shows the (mean ± standard deviation) number of schedules to first bug across 20 trials

* indicates the tool did not find the bug in at least one trial
† indicates the tool does not explicitly detect deadlocks
- indicates the tool did not find the bug in any trials
bold indicates the statistically significant (by the log-rank test [145], $p < 0.05$) best result on that program
Error indicates we were unable to successfully run the tool on this program

Chapter 7

Directed Concurrency Fuzzing

While generalized search of *all* concurrent program behaviors as in the previous chapter can be a useful starting point, typically some parts of the behavioral space are known or suspected to be more likely to harbor bugs. In *Directed Grey-Box Fuzzing* (DGBF) [26], program states which are likely to affect behaviors of such a targeted code region are prioritized over broader behavior exploration. For example, behaviors of recently changed code can be targeted to avoid regression bugs (c.f. Chapter 4). DGBF approaches can scale to extremely large behavior spaces because this space is effectively *pruned* by directing the search towards the most relevant code regions. Despite the numerous applications of DGBF for *input* fuzzing [205, 243, 154], this approach has yet to be applied to direct the search over the vast *interleaving space* of concurrent programs.

In this chapter, we bring the concept of *directedness* to controlled concurrency testing. We begin by building a taxonomy of three challenging patterns for randomized scheduling algorithms, namely **(C1)** preemption depth, **(C2)** state density, and **(C3)** scheduling breadth. To address the first challenge, we formulate a novel grey-box fuzzing framework for concurrency testing, leveraging the *Reads-From Value* (RFV) relation as feedback. In order to overcome the second and third patterns, we introduce *Directed Concurrency Fuzzing* (DCF). To enable DCF, we construct a concurrency-aware *Program Dependency Graph* (PDG) consisting of intra-thread dataflow dependencies, inter-thread dataflow dependencies, and control dependencies. We use this graph to estimate the likelihood of intermediate states to propagate value changes to target locations, represented by a distance calculation over the PDG, addressing the second challenge of state density. To address the third

challenge, we improve upon prior work in selective scheduling algorithms [245] to achieve *directed scheduling* of relevant operations, as determined by reachability in the Program Dependency Graph.

In summary, we make the following contributions:

1. We outline three key markers of difficulty for concurrency bugs: **(C1)** preemption depth, **(C2)** state density, and **(C3)** scheduling breadth.

2. We define *Directed Concurrency Fuzzing* as a principled approach to solve **(C1)**, **(C2)**, and **(C3)** and thus find difficult concurrency bugs.

3. We implement and evaluate our approach on leading CCT benchmarks and in change-base fuzzing scenarios on real-world programs

7.1 Motivation and Taxonomy

We define the problem of *directed* concurrency testing as a search of the concurrent program behaviors at specific target code location(s) for a *fixed input*. The aim is to achieve comprehensive coverage at these behaviors – irrespective of the coverage of other behaviors in the program.

To motivate our approach, we first examine the question: *What aspects of a program make finding a particular buggy behavior challenging?* Broadly, we categorize a concurrency bug's difficulty in three measures: preemption depth, state density, and scheduling breadth. The preemption depth of a concurrency bug is the *minimal* number of context switches necessary to trigger the bug. The state density represents the proportion of program states that affect if the bug is reachable. Lastly, scheduling breadth is the window size in which context switch constraints must be met.

To illustrate each of these measures, we have constructed a parameterized example program in Figure 7.1. The example program spawns two threads, each of which have `D` conditional statements. Here, the target program location is the assertion at line 3 of the `main` function. This assertion will fail if and only if the execution alternates back and forth between the two threads precisely such that each conditional check reads from the previous increment/decrement on the opposing thread. As a result, the *preemption depth* of the bug is proportional to `D`. The `log` variable is irrelevant

```
1  void thread_A() {
2  for(int i = 0; i < M; i++)
3    {log *= 2;}
4  d = 0;
5  if (s == 1) {
6   for(int i = 0; i < N; i++)
7      {log = i;}
8   d -= 1;
9   if (s == 2) {
10   d -= 1;
11   // ...  repeat nesting
12      if (s == D) {
13        d -= 1;
14 }}...}
15 }
```

```
1  void thread_B() {
2  for(int i = 0; i < M; i++)
3    {log *= 2 + 1;}
4  s = 1;
5  if (d == -1) {
6
7
8   s += 1;
9   if (d == -2) {
10   s += 1;
11   // ...  repeat nesting
12      if (d == -D)) {
13        s += 1;
14 }}...}
15 }
```

```
1     void main() {
2     // create and then join both threads...
3     assert(d != -(D + 1)); }
```

Figure 7.1: Motivating example concurrent program. Highlighted in red are instructions contributing to a noisy *state* space ($\propto 2^{\texttt{M}}$). Blue regions introduce noise in the *schedule* space ($\propto$ `N`). Preemption depth is $\propto$ `D`.

to the assertion in this program, however it impacts many common randomized scheduling approaches in two ways. Firstly, in red, the `log` variable creates many possible program states through racy increments (read + write) within loop bodies on both threads, bloating the *state space* in the program exponentially proportional to the loop size `M`. Secondly, in blue a single loop consisting of only write operations does not significantly impact program state. However, it does enlarge the window (namely line 5-8 on `thread_A`) where a context switch should *not* occur (i.e., line 9 on `thread_B` should not be executed), increasing *scheduling breadth* proportionally to `N`. In both cases, the `log` variable is meant to simulate noisy program behaviors which are not directly relevant to triggering a particular bug.

Figure 7.2 shows the Restricted Mean Survival Time (RMST) [194] of the assertion failure using various scheduling algorithms, as each measure of difficulty increases (from left to right: preemption depth, state density, scheduling breadth, and all simultaneously). The RMST is the area under the curve for a Kaplan-Meir survival plot [105]; a *lower* RMST indicates that the assertion failure was triggered more quickly in a fixed budget of 5000 schedules.

(C1) ***Bug Preemption Depth.*** In Figure 7.2 (left), we see *stateless* scheduling

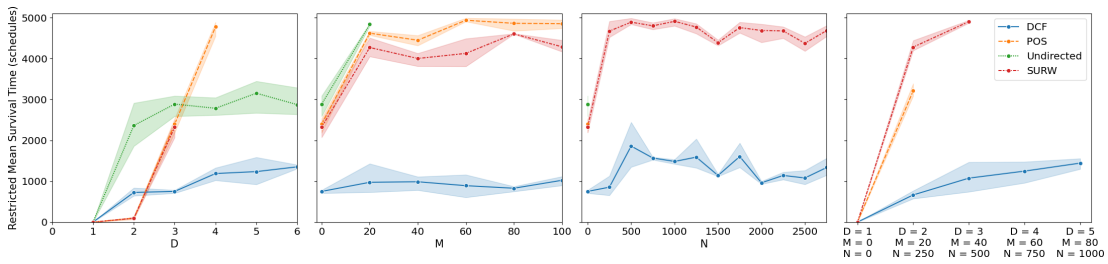


Figure 7.2: Comparison of Restricted Mean Survival Time (RMST) across different scheduling algorithms. **Lower** RMST indicates faster bug discovery, truncated lines indicate the bug was *not* found in any trial within 5000 schedules. Each curve shows the average RMST for the assertion in Figure 7.1 as depth (D), state noise (M) and/or schedule space noise (N) increase.

algorithms – those which do not preserve information across different executions – such as Partial Order Sampling (POS) [240] and Selectively Uniform Random Walk (SURW) [245] are able to reach the assertion relatively quickly for lower preemption depths. However, as the preemption depth increases to `D=3`, both approaches steeply decline in their effectiveness, and for `D>3`, a stateful approach is necessary to trigger the assertion. This is because each subsequent preemption represents a link in a Markov Chain of constraints [166], similar to those formed by nested branch conditions in input fuzzing [27]. As a result, the probability of reaching the end of the chain using unbiased sampling is exponentially unlikely in the length of the chain, in this case the preemption depth `D`. By incrementally saving program states after each link in the chain and boosting the sampling rate of rarely observed states, fuzzing-style stateful scheduling algorithms (described in Section 7.2.1) are able to scale to higher preemption depth. In this example, due to a more directed prioritization of states, DCT is able to achieve even better scalability with `D` than its Undirected concurrency fuzzing counterpart.

(C2) ***Program State Density.*** As `M` increases in Figure 7.2 (left-center), we can see the main drawback of the stateful approach necessary for detecting high-depth bugs. Namely, Undirected concurrency fuzzing cannot distinguish between relevant and irrelevant program states. As a result, as the proportion of irrelevant states in the program increases, an undirected approach will expend increasing proportions of its execution budget exploring these irrelevant states. Our *directed* approach, however, can determine whether or not a particular state is relevant to the bug via analysis of a Program Dependency Graph (PDG). By saving *only*

relevant states, its effectiveness does not degrade as irrelevant state density increases. Algorithms such as POS and SURW, being stateless, are not affected as the state space increases with `M`, aside from an initial spike from the increased length of the program trace.

(C3) ***Program Scheduling Breadth.*** Figure 7.2 (right-center) shows the impact of increasing scheduling breadth on scheduling algorithm effectiveness. Here we see approaches reliant on POS (including the Undirected fuzzing algorithm) struggle; POS, like most randomized scheduling algorithms, is exponentially unlikely in the length of the repetition to run the same thread repeatedly. SURW, however, can perform better by *selectively* sampling the interleavings of only a subset of events [245]; if the next event on a thread is in the selected set and chosen to run, it may be run for an arbitrarily large number of events until the next event in the selected subset is reached. However, in SURW, the selected subset is chosen randomly from a *static* set of all possible memory locations gathered in an initial trace. DCT leverages the same selectivity to be robust to scheduling breadth, while also improving on SURW by (1) sampling the selected subset of events only from reachable program locations as determined by the PDG and (2) leveraging *reservoir sampling* [221] to continuously update the selective subset as testing progresses and deeper program states are reached.

Combinations. While individual algorithms and approaches found in prior are able to overcome some of these challenges individually, none are able to scale in all cases as difficulty increases. Additionally, real-world applications generally present a mix of all three modes of difficulty intermixed – they contain both a large state space, a broad scheduling space, and may contain bugs of high-depth. DCT is not only the most scalable with all three measures of difficulty, but also can scale as all three parameters are increased *simultaneously*, as in Figure 7.2 (right).

7.2 Approach

We draw inspiration in our approach to solving DCT from the related Directed Grey-Box Fuzzing (DGBF) [26], a lightweight technique that has proven effective for sequential programs. DGBF uses simple, efficiently-computable heuristics to guide exploration towards reaching target locations stochastically *without* expensive

constraint solving. These heuristics are typically informed by maintaining a *distance metric*, which estimates how likely it is that a particular execution state can be randomly perturbed to reach the goal.

In contrast to sequential programs where inputs can be mutated upfront and distances can be measured syntactically [26], applying DGBF to concurrent programs introduces several unique challenges. First, we must define a compact schedule representation that retains incremental progress for high-depth bugs, as a total ordering of program events is ill-suited for the use case [229]. Secondly, we must define a distance metric to determine the relevance of program states that admits a *constant time* calculation at runtime. This is crucial for scalability reasons, since distances are queries repeated many times within a single execution during testing.

To address the first challenge, we shift from reasoning about complete executions to *schedule prefixes*. Intuitively, a schedule prefix specifies an interleaving up to a certain point in the execution, leaving the remainder unconstrained. This representation enables incremental saving since all extensions of a prefix naturally inherit its previous progress. We note that this representation is made possible by the explicit control over the interleaving in CCT, allowing an arbitrary execution prefix to be stored compactly as a random seed and replayed deterministically later.

Given our prefix-based representation, a key question remains: *How do we measure whether a prefix is making progress toward the target location (i.e. its relevancy)?* Our insight builds on a simple observation: given a fixed input to the program, a change in data flow to the target location initiated by *reading a new value from another thread* is a *necessary* condition for changes in program behavior at the target. In other words, there must be inter-thread value changes *earlier* in the execution that propagate to the target location for the behavior at that location to diverge. Therefore, we define new value observations as a signal of progress, and prioritize extending prefixes that are more likely to propagate the effect of new values. In particular, we monitor *reads-from values* (RFV), pairs of $\langle \ell, v \rangle$ that track the concrete value v observed by a read at a program location ℓ. To determine state relevance, we pre-process the program statically by combining thread-local and inter-thread analysis. This analysis allows us to quantity the likelihood of data propagation from any program location to the target. During testing, we consult the precomputed distance in constant time, allowing the fuzzer to filter out unreachable

Algorithm 3: Grey-box Concurrency Fuzzing

```
1:  S ← {ϵ}, S_fail ← ∅
2:  repeat
3:      (σ, η_σ) ← PickNextAndAssignEnergy(S)
4:      for i ∈ {1, ..., η_σ} do
5:          σ_mut ← mutateSchedule(σ, rand())
6:          if σ_mut crashes then S_fail ← S_fail ∪ {σ_mut}
7:          if isInteresting(σ_mut) then
8:              S ← S ∪ {σ_mut}
9:  until timeout
10: return S_fail
```

states, prioritize promising prefixes that produce new RFVs closer to the target, and selectively schedule relevant events.

7.2.1 Tackling (C1) With Unstructured Concurrency Fuzzing

In this section, we start by introducing the grey-box fuzzing algorithm adapted for CCT, as presented in Algorithm 3. The algorithm starts with a trivial corpus S that contains only the empty schedule. During each iteration, it tries to expand the corpus iteratively by extending a promising schedule prefix that has been saved. More specifically, an existing schedule σ is first picked from the corpus with an assigned *energy* η_σ (line 3). σ is then "mutated" (extended) to generate a new schedule σ_{mut} (line 5). This process is repeated for η_σ times. Each mutated schedule σ_{mut} is added to S_{fail} if the program crashes under the schedule (line 6). Additionally, the fuzzer saves σ_{mut} for future exploration if the program reaches an *interesting state* during execution (line 7-8).

Concurrency fuzzers are able to make incremental progresses by combining seed *retention* (line 8), *prioritization* (line 3), and *mutation* (line 5) strategies. These strategies are typically configured to steer towards unexplored program behaviors, approximated by code coverage and concurrency-aware feedback such as the *reads-from* relation (c.f. Chapter 6). Generally, mutation in the context of scheduling constitutes extending a schedule prefix. This extension can be achieved by using any scheduling algorithm with a random seed for the suffix of the execution. For our Undirected baseline in Figure 7.1, we use Partial Order Sampling (POS) [240] as the

mutation strategy. As a retention policy for our undirected baseline, we save prefixes for any novel RFVs $\langle \ell, v \rangle$ observed during execution. Lastly, for prioritization in our undirected baseline, we sample states which are *under-explored* as determined by (1) the prefix length – emphasizing shorter prefixes as they are less constrained – and (2) the prefix yield – emphasizing prefixes which have lead to more novel states in past executions.

However, attaining the goal of directedness requires a more specialized strategy. Intuitively, we would want to favor prefixes that are more likely to reach the target location, and extend these prefixes towards that goal accordingly. To do so, we construct a dependency graph for concurrent programs (Section 7.2.2) and leverage this graph to inform our mutation and retention policies (Section 7.2.3 and 7.2.4).

7.2.2 Concurrency-aware Program Dependency Graph

Consider a novel intermediate program state $\langle \ell, v \rangle_\sigma$ that a schedule prefix σ leads to. To assess the quality of this prefix, we are essentially posing a probabilistic reachability query: *How likely it is for an extension of σ to reach the target location ℓ_t with a target state $\langle \ell_t, v' \rangle$ that arises from $\langle \ell, v \rangle_\sigma$?* To answer such a query, we would need to know what the precise dependencies that lead from ℓ to the ℓ_t. As such, we construct a Program Dependency Graph as a preprocessing step to capture inter-thread concurrency, intra-thread dataflow, and intra-thread control dependencies between program locations.

Our context-insensitive Program Dependency Graph $G = (V, E)$ is a directed graph in which each node corresponds to a unique read or write instruction in the program. An edge $u \to v$ is formed if v is dependent on u in one of the following ways:

① intra-thread control dependency;

② intra-thread data dependency (i.e., local taints);

③ inter-thread data dependency (i.e., reads-from relations);

Each of these dependency edges is determined by a separate *conservative* analysis pass attempting to minimize false negatives, as sound and precise reasoning is not generally possible.

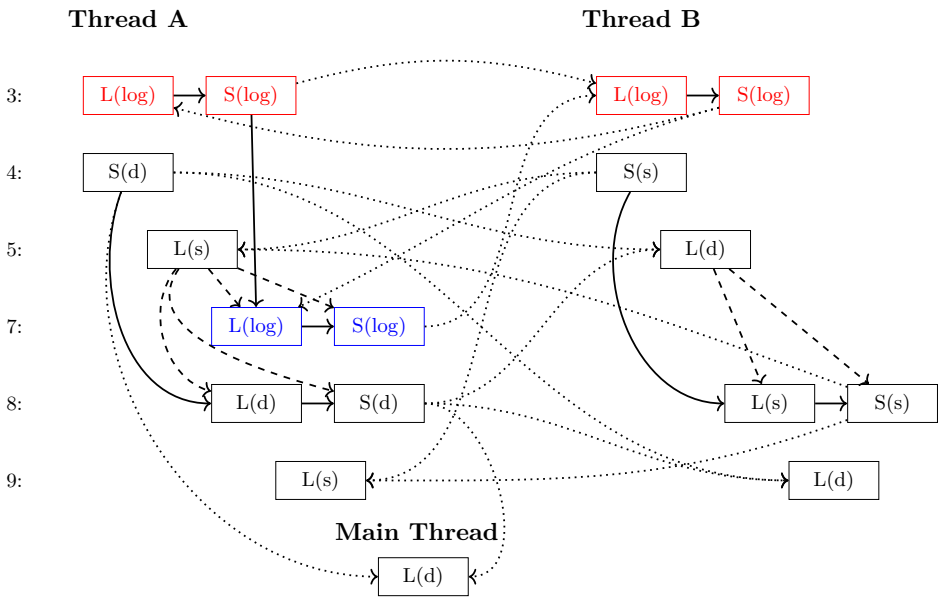


Figure 7.3: Simplified Program Dependency Graph (PDG) for the program in Figure 7.1 up to line 9. Dashed edges represent ① control dependencies, solid edges represent ② intra-thread dataflow, and dotted edges represent ③ inter-thread reads-from relations. Each node represents a single load (L) or store (S) instruction of a variable in the program. The assertion targeted is the $L(d)$ on the Main Thread.

Figure 7.3 shows a partial PDG for the motivating example from Figure 7.1. Notably, this simple program contains *all three* kinds of dependencies. We can see that several nodes related to accesses of the `log` variable (in red and blue) *cannot* reach the target at all and thus can be excluded from any directed analysis of the PDG. Different values reached at these instructions *cannot* impact the state at the target location.

Now let us consider comparisons among reachable nodes. Both the first store on Thread B and the last store on Thread A can reach the target in the PDG via multiple paths. However, because the PDG is generated by *conservative* analyses and context insensitive, many of these dependency edges may be spurious or otherwise irrelevant to the target location's state. To quantify this comparison, we assume each edge $e \in E$ carries a constant probability that the dependency is realized. We approximate this probability by the *type* of dependency it represents. This simple model already captures the inherent non-determinism in concurrent programs. For example, a potential inter-thread reads-from dependency only has certain probability of being realizable in an execution. With this model, the probability of a given

path being realized can then be calculated as the *product* of all edge weights on the path. Similarly, aggregating probabilities over all possible paths between two nodes captures the likelihood of reachability. In practice, since the number of paths can be exponential in the size of the graph, we estimate this likelihood by sampling paths up to a bounded length.

7.2.3 Addressing (C2) With Directed Fuzzing

State retention A schedule prefix, ending at program location ℓ where a concrete value v is observed, induces a program state captured by a Reads-From Value $\langle \ell, v \rangle$. For concurrent programs under a fixed input, a new state necessarily means a different dependency chain that ends at location ℓ has been observed in the execution. For each mutated schedule σ_{mut}, the fuzzer retains it only if it `isInteresting` (Algorithm 3). Specifically, σ_{mut} is considered interesting if it leads to a program state that is both *novel* (i.e., has not been observed in previous executions) and *reachable* (i.e., has a path in the PDG to the target). The reachability condition, adds *directedness* by pruning the search space. We note that our representation can be queried in constant time by pre-computing reachability information *before* beginning testing. As a result, DCF only preserves meaningful progress towards the target.

Schedule prioritization Prioritization of saved schedules dictates the ordering and budget with which saved prefixes are extended, and is implemented in procedure `PickNextAndAssignEnergy`. This procedure allows the fuzzer to prioritize promising schedules that are "closer" to the target. To quantify the closeness of a program state $\langle \ell, v \rangle_\sigma$ reached by a prefix σ, we compute a distance metric $d_{\ell_t}(\ell)$ as the inverse of the reachability likelihood from ℓ to the target ℓ_t on the PDG. Unreachable locations are assigned a distance of infinity. Essentially, the distance metic implicitly emulates the effect of a backward search by averaging probabilities over potential *extensions* of σ that connect $\langle \ell, v \rangle_\sigma$ to potential target states at ℓ_t.

DCF picks "closer" seeds with a higher probability and assigns a higher energy to them. A given σ is picked with probability η_σ:

$$\epsilon = \left(|E(\langle \ell, v \rangle)| - \frac{1}{2}|E| * (\|R(\langle \ell, v \rangle)\| + \|d_{\ell_t}(\ell)\|) \right)$$

Algorithm 4: Directed Schedule Mutation

Input: Schedule prefix $\sigma_{\mathsf{current}}$, RNG seed γ

```
σ_mut ← σ_current
executeSchedule(σ_current)
E_key ← sampleKeyEvents(γ)
while E ← getEnabled() ≠ ∅ do
    T_next ← pickNextThread(E, E_key)
    executeEvent(nextEvent(T_t))
    if not isKeyEvent(σ_mut) then
        extend(σ_mut, nextEvent(T_next))
return σ_mut
```

$$\eta_\sigma \propto (\epsilon > 0) \times -\epsilon \times Y(\langle \ell, v \rangle) \times d_{\ell_t}$$

Here, $\|R(\langle \ell, v \rangle)\|$ represents the normalized prefix length rank, $\|d_{\ell_t}(\ell)\|$ is the normalized distance metric, $|E|$ is the total number of executions, $|E(\langle \ell, v \rangle)|$ is the number of times a given state has been executed, and $Y(\langle \ell, v \rangle)$ is the number of novel states reached from state $\langle \ell, v \rangle$. Intuitively, we compute the expected number of executions we hope to see for a given state based on the average of the normalized distance and prefix rank. If difference between the expected and actual number of executions is positive (the state is *under-explored*), we sample it proportional to the difference, scaled by the yield of the state so far and distance.

7.2.4 Mitigating (C3) With Directed Mutation

We have so far discussed how DCF navigates among a set of partial dependency chains that reach different interesting program states. In this subsection, we discuss how to reach the next interesting state by extending a selected schedule that is close to the target.

Our directed mutation strategy is presented in Algorithm 4, which takes in a partial schedule prefix and a seed for a pseudo-random number generator (RNG). After replaying the existing prefix, DCF continues scheduling to extend this prefix using a stateless scheduling algorithm. As our stateless algorithm in DCF, we choose a variant of SURW [245]. SURW prioritizes the interleavings of a selected subset of events, irrespective of other events in the program. In its original implementation, SURW chooses this subset by randomly sampling over an initial execution trace. In contrast, DCF uses this framework to selectively sample interleavings of *only*

memory locations accessed by events E_{key} in the suffix that are *reachable* from the target within the PDG (line 3).

The original SURW implementation also suffers from the limitation that the selected subset is determined only by sampling a from a *single* execution trace. However, if a particular element is not accessed in that trace –due to a missing control-flow dependency, for example– then that element can *never* be sampled by SURW. To address this issue, we reimagine SURW using *reservoir sampling* [221]. By incrementally sampling E_{key} to determine the next selected subset on *each* execution, we guarantee that any observable element in E_{key} has some probability of being sampled, even if it is not observed in all traces.

7.2.5 Opportunistic Prefix Reduction

Thus far we have defined a state as an RFV $\langle \ell, v \rangle$. However, the same RFV may be observed from different prefixes. In particular, a prefix may include events which are *unnecessary* for reaching that RFV. For example, consider a program with three threads t_1, t_2, and t_3 where only events from t_1 and t_2 are necessary to observe a particular RFV, $\langle \ell, v \rangle_{t_1,t_2}$. A prefix of $t_3 * t_2 t_1$ will always observe $\langle \ell, v \rangle_{t_1,t_2}$, but may include arbitrarily many events from t_3 as well. Our key observation here is that these additional unnecessary events *constrain* the possible extensions from the RFV state – if events from t_3 are included in the prefix, they will be necessarily *excluded* from the suffix. As a result, many behaviors which stem from $\langle \ell, v \rangle_{t_1,t_2}$ that require events from t_3 may not be reachable from our saved prefix for that state of $t_3 * t_2 t_1$.

To mitigate this issue, we leverage *opportunistic prefix reduction*. For each saved RFV state, we also track the number of events in the schedule prefix needed to reach that state. If we encounter the same RFV later in the fuzzing campaign with a *shorter* prefix, we replace the previously saved longer prefix. This requires maintaining only a single additional integer for each saved prefix, but allows DCF to discover *less constrained* prefixes for RFV states slowly over time.

7.3 Implementation

We instantiate Directed Concurrency Testing in a directed concurrency testing tool, called DCF, implemented in Zig. DCF requires three analyses as a pre-

processing step for a given program, modeling ① control flow dependencies, ② intra-thread dataflow and ③ multi-threaded interactions through race pairs. In our implementation, we implement a standard dominator analysis for ①, and leverage SVF [214] for ② and ③. In addition, we adopt a simple dynamic trace analysis for ② in the case of larger programs such as LevelDB (see Section 7.4.3). However, conceptually many different static or dynamic analysis tools and approaches could be used to generate the required dependency graph.

Control Dependencies. To identify control dependencies, we formulate this problem as a query over a Dyck language [43], similar to parenthesis matching. We implement this analysis as a Python script operating on the control flow graph of the LLVM IR for the given program. In the analysis pass, we add a control dependency edge between a branch instruction and a constituent load or store if the load or store is reachable in the control flow graph while the branch instruction has not yet been joined. We identify and omit loop back-edges from this pass using a straightforward dominator analysis, also written in Python [136].

Intra-thread Dependencies. In our implementation of DCF, we use SVF's *whole program analysis* (`wpa`) to capture the thread local data-flow dependencies through register and memory operations, and generate a value-flow graph that is eventually incorporated into the PDG.

Inter-thread Dependencies. For inter-thread data dependencies, we utilize both static and dynamic analyses. We use SVF's multi-threaded analysis (`mta`) [213] to statically find potential race-pairs, which become inter-thread edges in our PDG. However, as `mta` can be resource intensive for larger programs, we alternatively provide a custom Rust-based dynamic trace analysis to detect conflicting accesses. As pre-processing step, this tool checks a subject's execution trace to identify instructions which interact with memory locations accessed on multiple threads. We add edges from these instructions to all other instructions which access these locations to the Program Dependency Graph.

Combined Graph Representation. Given ① control dependency edges, ② dataflow graph, and ③ race pairs, we combine all three analysis results into a single unified graph with three kinds of edges using Python. As an initial step to reduce the size of the graph, we drop nodes without any inter-thread dependency

edges. Finally, we compute the reachability and distance between each node and the target node(s) using a depth first search, sampling up to η paths for each *path length*. For our implementation, we set $\eta = 1$. We set the probabilities for the distance calculation as 1.02^{-1} for intra-thread dataflow edges, 1.10^{-1} for inter-thread edges, and 1.15^{-1} for control dependency edges. Each edge has a distance inversely proportional to its probability. We compute the distance from a given node to the target location as the mean over the distances of each of the sample paths. For long paths with a probability of less than 2^{-10}, we consider them to have a vanishingly small probability of being realizable, and omit them from the distance and reachability analysis. As a final output, our preprocessing step emits a file containing the computed distances for all nodes which can reach the target.

Instrumentation. To leverage our analysis during execution of the program, we implemented a Rust-based tool that instruments all load and store operations in the LLVM bitcode of the target program. During execution, at each such instruction DCF can look up the associated distance and reachability information for that instruction via an in-memory hashmap in constant time.

Runtime Scheduling. The runtime scheduler leverages shadow locking [122], in which a single global mutex is utilized to serialize the execution. Threading primitives are intercepted via `LD_PRELOAD` a custom `pthread` implementation as in Chapter 6. When the execution reaches a synchronization primitive or load/store instruction, the scheduler chooses one runnable thread to acquire the global mutex before continuing, ensuring all other threads are suspended. Schedule prefixes are persisted as random seeds, which are used to initialize and replay schedules up to a certain number of operations. As such, all non-determinism must be controlled in the program by intercepting non-deterministic interactions with the operating system like `clock_gettime` and disabling Address Space Layout Randomization (ASLR).

7.4 Evaluation

In this section, we aim to understand the effectiveness of DCF in terms of bug-finding ability and state exploration in a patch testing scenario. Concretely, we answer the following research questions:

RQ1 Bug Finding. How effective is DCF at finding bugs relative to other approaches on established concurrency benchmarks?

RQ2 Patch Testing. How exhaustively can DCF test individual commits from real-world embedded database systems?

RQ3 Program Dependency Graph Effectiveness. What is the accuracy and precision of Program Dependency Graph analysis?

7.4.1 Experimental Setup

7.4.1.1 Benchmarks and Subject Programs

Bug Finding We evaluate DCF on benchmarks utilized in prior concurrency testing work [226]. All bugs across these benchmarks manifest as crashes, assertion violations or deadlocks – which can be detected by DCF.

First, we select difficult examples from the SCTBench [199] and ConVul [35] benchmarks, using the version of this dataset curated in [226] (commit `cb8deb9`). These benchmarks are comprised of a mix of full real-world programs such as the JavaScript engine for Firefox, concurrent data-structure implementations and other examples extracted from real-world concurrency bugs. To focus on difficult bugs, we only include programs and bugs for which (1) at least one of POS or SURW can *not* find the bug in less than an average of 50 schedules [245], (2) are compatible with our concurrency testing instrumentation and framework and (3) are not redundant parameterized versions of the same program. In particular, for (2) we exclude one program from the ConVul benchmark due to an incompatibility with Address Sanitizer and our LLVM instrumentation. For (3) we include only the most difficult instances of the `CS/reorder` and `CS/twostage` programs, which are parameterized by the number of threads. After removing programs which are trivially found by stateless algorithms (1) we are left with 12 remaining programs/bugs.

We also evaluate DCF on the RaceBench dataset [191, 125] (commit `cb79cc5`). This benchmark consists of 15 large, real-world concurrent programs, including video encoding software, scientific computing applications and ray-tracing libraries. The dataset curators injected synthetic concurrency bugs into each of these programs

Table 7.1: Bug-Finding results on RaceBench [191] subjects. Each row shows the Mean ± Standard Deviation and total unique ('Uniq') bugs found out of 100 across N=10 trials. A † indicates the target was instrumented with sampling instrumentation to increase throughput for all tools. **Bold** indicates the statistically highest mean number of bugs found at $p < 0.05$ [143].

	DCF		Undirected		SURW		POS	
Benchmark	**Mean ± Std**	**Uniq**	**Mean ± Std**	**Uniq**	**Mean ± Std**	**Uniq**	**Mean ± Std**	**Uniq**
blackscholes	**30.30 ± 1.49**	40	21.00 ± 1.76	26	23.70 ± 1.34	31	20.90 ± 1.20	30
bodytrack†	**52.30 ± 2.41**	70	41.00 ± 6.27	65	46.60 ± 1.51	64	**51.20 ± 2.70**	65
canneal	**60.20 ± 1.14**	68	55.10 ± 2.51	60	58.20 ± 0.79	62	57.90 ± 1.45	63
cholesky†	**67.30 ± 1.42**	85	46.20 ± 6.23	69	61.60 ± 1.90	70	65.00 ± 1.89	75
dedup	**61.20 ± 1.48**	74	28.50 ± 3.75	56	58.10 ± 1.85	71	55.90 ± 2.38	68
ferret	**84.20 ± 1.23**	91	38.30 ± 5.46	55	49.40 ± 2.32	58	54.80 ± 0.79	59
fluidanimate†	**53.30 ± 3.02**	67	33.50 ± 4.06	59	50.10 ± 1.73	60	**55.80 ± 1.23**	65
pigz	**88.40 ± 1.35**	94	53.80 ± 5.39	69	76.40 ± 0.52	77	76.50 ± 0.53	77
raytrace	**68.90 ± 1.73**	81	62.00 ± 2.31	68	62.90 ± 1.29	66	67.10 ± 0.99	70
raytrace2	**59.00 ± 1.70**	74	50.20 ± 4.10	60	55.00 ± 1.41	63	57.20 ± 1.69	62
streamcluster†	**70.00 ± 1.70**	77	40.80 ± 4.85	68	66.00 ± 1.41	71	67.80 ± 1.62	72
volrend†	**24.70 ± 1.83**	31	**23.90 ± 3.54**	34	**24.60 ± 2.07**	33	**27.30 ± 2.41**	35
water_nsquared†	**10.50 ± 2.37**	21	5.10 ± 2.38	17	**9.20 ± 2.20**	21	7.30 ± 1.70	19
water_spatial†	**69.60 ± 2.95**	85	28.50 ± 5.42	64	**68.60 ± 1.58**	82	65.70 ± 2.95	80
x264	**48.90 ± 2.92**	72	35.20 ± 2.97	47	39.10 ± 1.29	55	30.60 ± 2.41	42
Total		1030		817		884		882

that are intended to mimic real-world bug-patterns such as deadlocks, atomicity violations and order violations. Each program contains 100 injected bugs, split evenly across 5 otherwise identical instances of the program.

Patch Testing To evaluate DCF's capabilities in patch testing, we select three real-world embedded database systems, namely SQLite [89], LevelDB [77], and HCTree [88]. These programs are in inherently concurrent and are significantly larger than the benchmark programs, making exhaustively searching for all interleavings impossible in practice. For each of these systems, we select two commits, setting the changed lines in the commit as targets for DCF. We identify these commits by (a) running the current version of the system under the POS scheduling algorithm (b) sorting all memory reads by number of value changes observed under POS in the first 100 executions and (c) performing a `git blame` on the source location for that memory read to identify a commit. We repeated this process on subsequent memory reads until we obtained two commits from each project that (1) compiled and (2) continued to exhibit value changes under the POS scheduling algorithm at the older program version. This process ensured that we identified commits which affect *non-trivial* concurrent behaviors.

7.4.1.2 Experimental Parameters

We ran experiments for the SCTBench and ConVul bugs, as well as patch testing, on a Intel(R) Xeon(R) CPU E5-2660 v4 @ 2.00GHz (Model 79) with 56 logical cores and 64 GB of RAM on Ubuntu 20.04.4 LTS (5.4.0-90-generic). We ran experiments for RaceBench programs on a Intel(R) Xeon(R) Gold 6258R CPU @ 2.70GHz (Model 85) with 112 logical cores and 188 GB of RAM on Ubuntu 22.04.5 LTS (5.15.0-173-generic). Each tool ran in a Docker container without disabling Spectre mitigations, pinned to a single CPU core.

Bug Finding. For bug-finding experiments, we allocate a fixed time budget to each scheduling algorithm. In the case of the selected SCTBench and ConVul bugs, we use a timeout of 5 minutes per bug. In the case of RaceBench, each instance of a program contains 20 bugs, so we provide DCF one unique bug as a target in 20 discrete 3 minute intervals for each instance. For other algorithms and tools, we give a *contiguous* 1 hour block for all 20 bugs in each instance. For each bug in SCTBench, we manually identify the target locations from the source code and bug reports in [226]. In the case of RaceBench, we take the guard statement to the injected test-oracle as the target location for constructing the PDG. To construct the PDG we use the static analysis described in Section 7.2.2. Additionally, because some programs in the RaceBench dataset result in extremely long traces, we *sample* each instrumented scheduling point with probability 2^{-5} for all tools and algorithms, similar to the methodology used in [245]. If a scheduling point is not sampled at runtime, we do not invoke the scheduler and resume executing that thread without a context-switch. Programs which use sampled instrumentation are denoted with a † in Table 7.1.

Patch Testing. In our patch-testing experiment we allocate each algorithm 1000 schedules, with the changed lines in each commit as target locations for DCF. To construct the PDG, we use *dynamic analysis* to discover potentially racing instructions from a series of trial-run executions of the program, rather than utilize SVF's `mta` analysis (Section 7.2.2). Specifically, we conducted a single trial run pre-PDG construction, and passed all conflicting instructions (accessed the same memory location on different threads) as races to our PDG construction. Post-construction, we did a lightweight run of 100 schedules and added any remaining

missing races highly correlated with value changes directly to the graph.

7.4.1.3 Baseline Scheduling Algorithms

As baseline algorithms we compare DCF against two stateless scheduling algorithms SURW [245] and POS [241]. For a stateful fuzzing algorithm, we compare with `RFF`, which also uses feedback to bias its search across multiple execution. We also compare against an ablative variant of DCF, which we dub Undirected. Undirected utilizes the same prefix-saving and prefix-trimming fuzzing of DCF, but lacks directedness in both its scheduling and feedback. Specifically, it uses POS as its scheduling algorithm, and saves any prefix which reaches a new RFV *anywhere* in the program (c.f. Section 7.2.1).

Table 7.2: Number of bug discoveries in selected subjects from SCTBench [199] and ConVul [35] (N=30 trials).

Program	**DCF**	**POS**	**SURW**	**Undir.**	**RFF**
CB/pbzip2-0.9.4	17/30	28/30	3/30	29/30	2/30
CS/bluetooth_driver	30/30	30/30	30/30	30/30	30/30
CS/reorder_100	26/30	0/30	30/30	0/30	30/30
CS/twostage_100	16/30	3/30	27/30	4/30	30/30
Inspect/qsort_mt	30/30	30/30	30/30	30/30	30/30
ConVul/CVE-2016-1972	30/30	30/30	30/30	30/30	30/30
ConVul/CVE-2013-1792	30/30	30/30	30/30	30/30	30/30
RADBench/bug2	29/30	30/30	30/30	29/30	–
RADBench/bug3	30/30	30/30	30/30	30/30	–
RADBench/bug4	30/30	30/30	30/30	30/30	30/30
RADBench/bug5	4/30	0/30	1/30	0/30	0/30
SafeStack	1/30	0/30	0/30	2/30	0/30
Total	273/360	241/360	271/360	244/360	212/300
Unique	12	9	11	10	10

7.4.2 RQ1: Bug Finding Ability

RaceBench Across the RaceBench dataset, DCF finds more bugs on average and more unique bugs than other approaches. In the case of `pigz`, DCF is even able to find 94/100 total bugs! We note that the number of bugs found is heavily *program* dependent. For example, all approaches perform poorly on some benchmark programs, such as `water_nsquared` or `blackscholes`, despite all bugs in the benchmark having been injected by the same methodology. In the case of `water_nsquared`, we

can attribute the low effectiveness of all tools to poor testing throughput; even with selective instrumentation, this program does not terminate within a 10-second timeout of under any scheduling algorithms.

Looking at the results of the ablative Undirected fuzzer, we see that it performs *worse* even than the two stateless scheduling algorithms, POS and SURW. This is despite the RaceBench programs containing many high preemption-depth bugs. The sole difference between the Undirected and POS algorithms is that Undirected saves schedule prefixes for states which reach new RFVs. As such, we can conclude that it performs worse than POS because it spends too much time exploring states *irrelevant* to the buggy locations – in essence a lack of *directed feedback* **(C2)**. With directedness, DCF is able to improve over both Undirected (due to **(C2)**) *and* SURW (due to **(C1)** and **(C3)**).

We also ran `RFF` on the RaceBench dataset. However, we found that it underperforms other approaches on these programs, finding only 264 bugs out of 1000 total [1] due to scalability issues. The Racebench dataset was not available at the time of the work in Chapter 6, and as a result the tool was not developed against large programs with frequent control-flow changes. In particular, `RFF` runs a dynamic conflict analysis over a total-order trace every time control flow changes in the program under test. Because control flow frequently changes in the Racebench programs (as a result of the injected bugs being encoded as `if` statements), it spends a large portion of its execution time budget writing and analyzing full traces, which can be on the order of millions of events for some of these programs. While many (if not most) real-world programs, will *not* exhibit such frequent changes in control flow, this is an important limitation of `RFF` which is addressed by the work in this chapter.

SCTBench and ConVul In the selected SCTBench and ConVul programs, we see that Undirected fuzzing performs approximately as well as POS. This is because these programs have a much *smaller* state space than those in the RaceBench benchmark. As a result, Undirected is less affected by an explosion of irrelevant program states **(C2)**. This smaller state space also means that we see a smaller

[1] `RFF` does not run on 5 programs due to limitations in the instrumentation and concurrency primitive modeling in `RFF`

benefit from DCF's directedness.

In general, DCF is comparable in efficacy for most bugs in this benchmark as compared to `RFF` and SURW. However, the two most challenging bugs in the benchmark `RADBench/bug5` and `SafeStack`, DCF is the only approach to find both of these bugs within the allotted time. Indeed, in prior work [245, 226, 241], the `Safestack` has been reported to have been found only very rarely, after hundreds of thousands or even millions of schedules. `RADBench/bug5` has not been found in *any* of these recent prior research works.

Across difficult examples from leading benchmarks, DCF is able to consistently find more bugs as compared to existing CCT approaches.

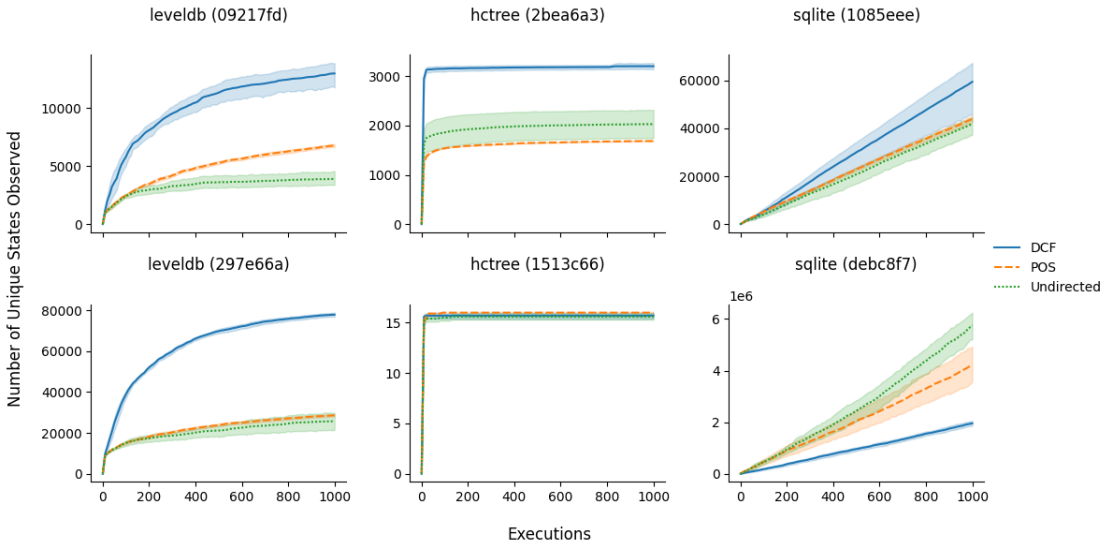


Figure 7.4: Cumulative unique RFVs observed at changed locations in patch-testing of embedded database systems.

7.4.3 RQ2: Patch Testing

One notable application of directed fuzzing is to test *code changes* in project as it evolves. In this research question, we examine DCF in the context of testing *concurrent* behavior of individual code changes. We evaluate DCF's capability in patch testing on six commits across three prominent embedded database systems. Figure 7.4 visualizes the cumulative state coverage achieved by each approach

averaged over 10 independent trials. Here, we approximate program states by unique RFVs observed at changed lines in the commit.

We first observe that there is extremely high variance in state-space size across both programs and commits, indicating that we have successfully constructed a diverse dataset for this experiment. `hctree-1513c66` has only 16 states, which are found by all approaches almost immediately. However, `hctree-2bea6a3` and the two `leveldb` commits exhibit a more gradual curve, similar to the logarithmic growth observed in input fuzzing [24]. In contrast, `sqlite-1085eee` and `sqlite-debc8f7` both seem to grow at a linear rate, with no indications of slowing in our 1000 schedule budget, but the latter has an order of magnitude larger number of states explored by each algorithm.

Across the 4/6 commits, DCF achieves the highest state coverage. However, we do observe that for `sqlite-debc8f7` DCF performs significantly worse than both Undirected and POS. We then ran a follow up experiment using SURW and a version of DCF without state-saving. In both cases, we found their effectiveness to not be significantly different than DCF. Thus the state-space of this commit appears to be more amenable to the distribution of schedules sampled by POS. In theory, ensemble methods could be used to sample different distributions of schedules within the same fuzzing run, which we believe could be a promising research direction.

DCF is effective in covering diverse behaviors at patch locations in large, real-world software.

7.4.4 RQ3: Static Analysis Effectiveness

The experiments shown in Figure 7.2 demonstrate that DCF can be extremely effective at mitigating all three challenging constructs given an accurate and precise PDG. However, obtaining a good PDG for large, real-world programs can be difficult in practice. To evaluate the quality of our Program Dependency Graph on large programs, we again utilize the RaceBench dataset. For each bug in RaceBench, one `struct` has been injected into the program by the benchmark infrastructure. Statements accessing fields on this `struct` *must* be interleaved to reach the bug. By identifying instructions which access these injected `structs` we have ground truth

for (a) a set of true positive code locations —those which access the corresponding `struct`— and also (b) a set of true negative code locations —those which access a `struct` injected for a *different* bug. Using this ground truth data allows us to *estimate* the precision and recall of our PDG construction, shown in the first two columns of Table 7.3.

Here we see that the estimated precision of the PDG is generally worse than its recall (median 0.66). However, in both cases, there are a few outliers with very low precision or recall. We note that false positives will generally be introduced by the conservative static analysis (c.f. Section 7.2.2). False negatives can be introduced by excessively large distances in the PDG, which we discard, or by gaps in the static analyses (e.g. incomplete modeling of program constructs).

Despite that precision for the PDG is relatively lower than recall, indicating a larger number of false positives, our construction of the PDG also allows us to estimate the likelihood that the a dependency chain in the PDG is not spurious via its "distance" to the target locations in the PDG. As such, even when the precision of the PDG is low, if the PDG distances given to DCF are an accurate approximation of likelihood, then DCF can de-prioritize high-distance locations.

To assess this capability, we measure the effect size of distances between true-positive and false-positives in the PDG in the last column of Table 7.3. Specifically, we use the non-parametric Vargha-Delaney $\hat{A}_{12}$ effect size [220]. Here we see that there is generally a *large* effect size ($\hat{A}_{12} > 0.71$ [219]) between the true-positive and false-positive distances, indicating that their distributions are widely different. We visualize these distributions for two example RaceBench programs in Figure 7.5. In the case of `raytrace2` (left), we can see two distinct and separable distributions. For `blackscholes` (right), we see that the distributions are overlapping, but the overall shape is still strongly bimodal.

DCF's statically computed PDG can have a high number of false positives; however, its distance metric helps separate false positives from true ones.

Table 7.3: Estimated statistics for the PDG on RaceBench programs. **Bolded** effect size indicates statistical difference between true and false positive distances at $p < 0.05$ [143].

Program	Precision	Recall	$\hat{A}_{12}$
blackscholes	0.28	0.72	**0.74**
bodytrack	0.23	0.18	**0.80**
canneal	0.23	0.73	**0.89**
cholesky	0.29	0.73	**0.90**
dedup	0.64	0.50	**0.95**
ferret	0.93	0.25	0.54
fluidanimate	0.50	0.60	**0.98**
pigz	0.06	0.57	**0.67**
raytrace	0.51	0.66	**0.93**
raytrace2	0.40	0.70	**0.88**
streamcluster	0.37	0.49	**0.96**
volrend	1.00	0.71	N/A
water_nsquared	0.95	0.69	**0.94**
water_spatial	0.95	0.57	**1.00**
x264	0.09	0.79	**0.84**
Median	0.40	0.66	0.90

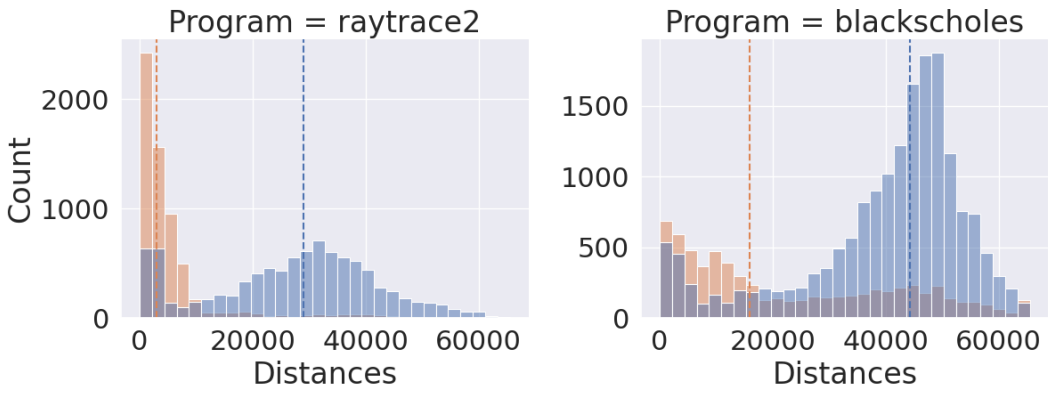


Figure 7.5: Distribution of distances in PDG for blackscholes (left) and raytrace2 (right); true positives in orange, false positives in blue. Medians shown with dashed lines.

7.5 Discussion

This chapter brings directed search to domain of Controlled Concurrency Testing (CCT), showing that targeted exploration of the interleaving space is both feasible and highly effective. By defining three fundamental challenges in CCT, we clarify *why* existing scheduling algorithms fail to observe specific behaviors in concurrent programs. Furthermore, we show that *statefulness* and *directedness* are key components in overcoming these challenges. DCF leverages concurrency-aware PDG for directed grey-box feedback and directed scheduling. In an evaluation

across established CCT benchmarks and real-world systems, we see that this form of directedness yields substantial improvements in both bug-finding ability and depth of exploration. Our results suggest that the principles that made directed fuzzing so successful in input-space search can be fruitfully adapted to schedule-space search as well. We see this as a step toward a new class of concurrency testing tools that can scale to large programs and executions through targeted search.

Chapter 8

Concurrency Fuzzing in Kernel Code

The previous chapters describe systems which attain control of the schedule by leveraging a shared library reimplementation of `pthreads` which is interposed via `LD_PRELOAD`. While this approach works well for user-space applications, it *cannot* be directly applied to kernel space code. This begs the question: how can we achieve fine-grained scheduling control necessary for fuzzing-style algorithms in kernel space? The following chapter extends prior work [81] to achieve *fine-grained* scheduling control for kernel code via eBPF. In particular, we provide a new compile-time instrumentation approach, and a new comprehensive of the eBPF scheduler's throughput and bug-finding ability on a benchmark of existing kernel concurrency bugs.

Most coverage-guided fuzzers for kernel code [63, 76, 235, 99] explore the space of inputs in the form of system call programs. Complementing this direction, concurrency-oriented fuzzing techniques [97, 100, 75, 98] have been proposed to control thread scheduling and explore different interleavings, leveraging both static and dynamic analysis in combination with custom heuristics. However, these existing approaches still face the following challenges and limitations:

C0: Full, fine-grained scheduling control. Many concurrency fuzzers either provide only weak control over thread scheduling by injecting delays [234] or limited coarse-grained scheduling points [97, 100]. With only weak or limited, coarse-grained control, many concurrency bugs may be rendered unreachable during testing. While trading completeness for performance is sometimes reasonable, these works fundamentally cannot enforce a higher degree of control, even in cases when performance is not an issue.

Table 8.1: State-of-the-art kernel concurrency fuzzers.

Tool	**Hypervisor Agnostic**	**Forward Compat.**	**Scheduling Control**	**Schedule Algo.**	**Runtime Cost**
Razzer [97]	✗	✓	limited	custom	high
Snowboard [75]	✗	-	✓	custom	high
SKI [64]	✗	✗	✓	PCT	high
KRACE [234]	✓	✗	weak	custom	low
SegFuzz [98]	✗	✗	✓	custom	high
Sect	✓	✓	✓	RW+POS+PCT	low

C1: High runtime overhead. Traditional concurrency fuzzers with full control typically operate at the hypervisor level [98, 75]. These hypervisor-specific techniques are extremely heavyweight and exhibit poor scalability. For example, Snowboard [75] introduces a substantial performance overhead, slowing execution by more than 10×. Additionally, the limitations of the hypervisor's functionality restrict SegFuzz's support [98] for parallel fuzzing processes, resulting in a significant reduction in overall throughput (c.f. Section 8.3.2).

C2: Lack of forward compatibility. OS kernels have been under continuous development, with frequent updates and ongoing evolution. In addition to custom hypervisors, existing fuzzers usually require extensive customization tailored to specific kernel versions [98, 234, 97]. For instance, Krace [234] involves invasive kernel features, introducing a patch of over 10k lines of code across 105 files. This level of modification increases the cost and effort required to port the fuzzer to newer versions, and may also reduce its effectiveness. As a result, the practical usability and maintainability of such approaches are often compromised.

C3: Restricted extensibility. Scheduling algorithms in existing concurrency fuzzers are often bespoke and tightly integrated with critical components of the underlying implementation. Extending or duplicating them in a new codebase is typically extremely challenging and demands deep domain expertise. As a result, traditional fuzzers that rely on default scheduling behavior are often still used to attempt to find concurrency bugs in practice.

Table 8.1 summarizes existing fuzzing approaches, all of which suffer from the aforementioned challenges. To address the challenges and improve the efficiency of kernel fuzzing, we propose the Sched-Ext Concurrency Tester (Sect) in this work. Sect is a lightweight but comprehensive concurrency fuzzing framework that

provides scheduling control over threads for interleaving exploration. Specifically, SCT reuses the Linux eBPF feature `sched_ext`—originally designed for performance optimization — as the basis for concurrency testing. The Extended Berkeley Packet Filter (eBPF) enables dynamic in-kernel customization at runtime without modifying the Linux kernel code. It allows developers to load and run restricted programs in a safe, sandboxed, in-kernel virtual machine (VM) [159, 51].

`sched_ext` eBPF API (merged in 6.12) decouples scheduling policy from the core kernel scheduling mechanism. It exposes user-defined scheduling logic through public interfaces [237], and allows developers to implement customized scheduling policies through eBPF programs. Only processes explicitly set with the `SCHED_EXT` scheduling class are affected by the user-defined policies. While `sched_ext` is originally designed for writing domain-specific schedulers for performance optimization, our novelty is to leverage this feature to support kernel fuzzing for CCT.

SCT serializes non-deterministic thread execution in a controlled manner without hypervisor modification. SCT incorporates several CCT scheduling algorithms geared towards schedule exploration, and is fully extensible. SCT is also capable of *fine-grained* control of scheduling at the individual instruction level via a custom LLVM instrumentation pass which can inject additional cooperative scheduling points. We have also integrated SCT into a basic *input* fuzzing framework, allowing for simultaneous exploration of the input and scheduling space.

We implement and evaluate SCT on recent versions of the Linux kernel in parallel with its development. In our evaluations, SCT achieves 38% higher branch coverage than the state-of-the-fuzzer concurrency fuzzer SegFuzz with minimal overhead introduced (57% reduction in overhead as compared to SegFuzz). On a benchmark of known bugs, SCT demonstrates effective scheduling control (**C0**) without high-overhead hypervisor customization (**C1**). Because SCT is natively supported by the Linux kernel, it can be ported to new targets with minimal code changes (**C2**). Lastly, we provide four classic CCT algorithms implemented in SCT, highlighting its potential as an extensible platform for exploring new CCT algorithms in the future (**C3**).

In summary, our work makes the following contributions:

- We propose to refine kernel concurrency fuzzing techniques to enable lightweight,

effective, and extensible control over thread behavior, mitigating the challenges **C0**, **C1**, **C2** and **C3**.

- We develop SECT, a concurrency fuzzing framework. SECT features a custom CCT scheduler leveraging eBPF for thread control, a fine-grained strategy for injecting preemptions into kernel code, and a two-phase mutation approach for effective fuzzing.
- We evaluate SECT on recent versions of the Linux kernel. SECT has 11.4× the throughput of state-of-the-art concurrency fuzzing tools. It also finds new kernel bugs, with five fixed by developers.

8.1 Design of SECT

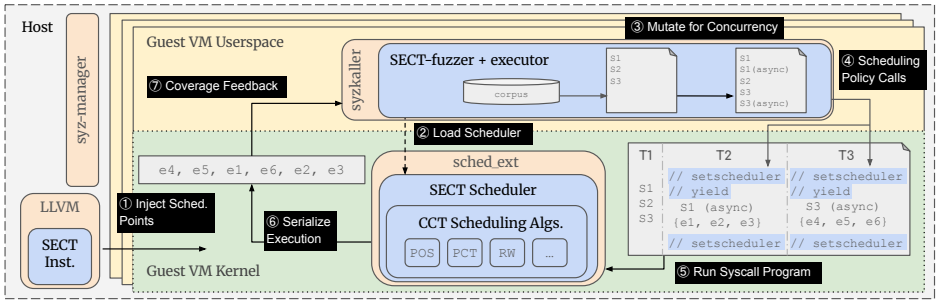


Figure 8.1: Overview of SECT (components of SECT in blue).

Figure 8.1 shows the design components and the overall workflow of SECT. To enable fine-grained thread control in this second phase, SECT can optionally inject additional safe preemption points into the kernel code (①) at compile time. The scheduler can be loaded once at test-time in a few seconds (②) without needing to reboot or recompile the kernel. We leverage Syzkaller with some minor modifications (③) to generate multi-threaded syscall programs. As SECT's core, we propose a lightweight and effective scheduler to control the concurrent behavior of the test case under the `sched_ext` scheduling policy (④). As such, during execution of a multi-threaded test input (⑤), SECT replaces the native OS scheduling algorithm for a selected subset of threads. When a scheduling point (e.g. an I/O block) is reached, SECT enforces a schedule decision and selects the next thread to run from

the set of runnable candidates according to a CCT scheduling algorithm. SECT repeats this scheduling process until all concurrent threads are finished. In doing so, SECT can effectively serialize (⑥) non-deterministic multi-threaded executions in a controlled order according to CCT algorithms, thereby increasing the likelihood of triggering bugs that manifest due to rare interleavings. During the execution of sequential or controlled concurrent test cases, SECT monitors whether the kernel under test crashes or terminates with any errors, in addition to collecting code coverage feedback (⑦) to drive exploration of the input space. Through steps ① - ⑦, there are four main components to our approach, shown in blue in Figure 8.1: the SECT scheduler itself, the CCT scheduling algorithms implemented in SECT, the SECT instrumentation, and the SECT fuzzing loop. In the following section, we explain each of these components.

8.1.1 SECT-Scheduler for Concurrency Fuzzing

The goal of our SECT scheduler is to carefully control the interleaving of kernel thread executions to expose elusive data race conditions in the kernel. Although prior work has shown that such fine-grained control is possible using heavy-weight solutions, doing so without modified hypervisors and/or special hardware support was an unsolved technical challenge. Here, we detail our solution to achieving controlled kernel interleaving using only lightweight, native eBPF kernel extensions.

We implement the SECT scheduler in an eBPF program which is loaded during test runtime. The scheduler uses the interface exposed by `sched_ext` to control kernel thread scheduling. According to (**C0**) we serialize the execution of concurrent kernel threads under test to fully control their event ordering (e.g. accesses to shared data). To do so, the scheduler maintains the invariant that only one task from a desired subset of tasks may run at a time, even if additional hardware resources are available. As such, our scheduler only dispatches a single task from the desired subset to a CPU core. Once the dispatched task has returned to the scheduler as either quiescent (i.e. blocked) or enqueued (i.e. runnable), the scheduler then chooses and dispatches another task from among the runnable tasks. By default, this means that the serializing scheduler code will be invoked *only* when the lone running thread either yields control because of a blocking operation or it has exhausted its

time slice.

Algorithm 5: Controlled Scheduling

```
   Input: set of active tasks T
   Input: current task t
   Output: serialized execution trace S
 1: initialize_scheduling_algorithm()
 2: while |T| > 0 do
 3:     g ← get_sched_ext_threads(t)
 4:     if is_sleeping(t) then
 5:         t.enabled ← False
 6:     else if is_exiting(t) then
 7:         T ← T \ t
 8:     else
 9:         t.enabled ← True
10:         T ← update_thread_weights(T, g, t)
11:     update_fairness_bound(t)
12:     T' ← get_enabled_threads(g, T)
13:     t' ← pick_highest_weight_thread(T')
14:     next_t ← dispatch_and_run(t')
15:     S ← append(S, t'.next_event)
16:     t ← next_thread()
```

To enable running parallel test instances in the same VM, the scheduler partitions threads spawned under different instances into different scheduling subsets. Scheduling decisions among the subsets are completely decoupled, though threads from different subsets may time-share the same VM cores. Algorithm 5 shows a more detailed description of the SKI scheduler procedure. When the scheduler is invoked, either due to timer interrupt or the current running thread transitioning to the quiescent state, the algorithm locates the corresponding scheduling subset (Line 3), removes the current task if it is exiting (Line 6) or disables the task from scheduling if it is blocked (Line 4). Otherwise, the algorithm updates the weights of each task in the subset (Line 10); details of this weight update are delegated to a CCT algorithm. Finally, the scheduler selects an enabled thread from the subset with the highest weight and dispatches the thread to run. This architecture enables flexible thread scheduling for various concurrency testing strategies (**C3**): different exploration strategies simply define a weight-assignment function, without modifying the overall scheduling loop.

Enabling CCT in `sched_ext`. The `sched_ext` framework primarily utilizes priority queues to manage task dispatching. While this is an excellent data structure for general-purpose scheduling where latency requirements are critical, most concurrency testing algorithms, such as PCT [32] and the algorithm described in Chapter 6, make *dynamic changes* to thread priorities continually as the program executes. This dynamism is crucial to allow the scheduling algorithm to adjust to events in the program as they occur – for example, when the current thread is identified as racing with an event on another paused thread, we might want to increase the weight of the paused thread to allow it to run first [241]. Unfortunately, the priorities of threads in `sched_ext` dispatch queues are *immutable* after insertion.

As a result, in SCT, we abandon the default priority queues available in `sched_ext`, and instead define and manage our own queue via eBPF maps which contains each task's dynamic priority. Dynamic priorities are algorithm-specific; at each scheduling decision the scheduling algorithm determines what each task's priority should be and our scheduler dispatches the task with the highest priority.

8.1.2 Scheduling Point Injection

When SCT dispatches a task, the task runs until a blocking operation occurs or its time slice expires, after which the scheduler is invoked. Relying solely on blocking operations for voluntary scheduling, however, may be too coarse-grained to expose many concurrency issues (**C0**). In principle, any visible operation that causes thread synchronization or communication, such as locking mutexes, acquiring spin locks, or accessing shared memory, can lead to concurrency bugs if reordered. Their code locations are therefore interleaving candidates for CCT. To improve granularity in finding concurrency bugs, SCT injects additional voluntary preemptions into the kernel under test.

Identifying Preemption. We delegate the majority of the responsibility of identifying important and safe scheduling points to run-time; static approaches to identify all possible sources of inter-thread communication (including shared pointers) without large numbers of false negatives and/or false positives are extremely difficult for a C codebase the size of the Linux kernel. In this setting, falsely excluding a source of inter-thread communication can render large portions of the potential

scheduling space unreachable (i.e., reducing the completeness of SECT– **C0**). As such, we instrument memory access instructions (`load` and `store` LLVM IR instructions), as well as 37 other synchronization functions that we manually identified in the kernel source code. These synchronizations include the acquisition and release of spin locks, semaphores, RCU locks, and other concurrency primitives available to kernel developers. To avoid excessive fuzzing overhead from all the instrumented instructions/functions, SECT takes a sampling rate as an input parameter. By default, we invoke the scheduler with 10% probability at each instrumented scheduling point during fuzzing. This setting reduces SECT's theoretical completeness, but significantly improves testing performance without affecting soundness. Crucially, unlike other weak and limited control concurrency fuzzers, sampling rates and distributions are fully customizable within the SECT framework.

Preemption Safety. When injecting scheduling points into the kernel, it is critical to preserve the assumptions made by kernel developers regarding where context switches may occur. Failure to do so can lead to false positives that are caused by context switching when it is not safe to do so. Example "unsafe" kernel locations for context switching include NMI and IRQ context, when preemption or IRQs are disabled, spinlocks or RCU read locks are being held, and within the scheduler core. However, a precise context switching predicate is not encoded into the kernel source code, nor recorded in comments or documentation. Fortunately, to aid Linux kernel development, developers have created some safety assertions. For the SECT instrumentation, we extract these checks from the widely used synchronization helper functions (e.g., `might_sleep()`) and use them as a predicate to invoke the scheduler. We note that passing these checks does not guarantee safety to invoke the scheduler, but failing them will likely lead to deadlocks or kernel crashes. Indeed, without these checks, we observe that spurious kernel panics happen very frequently when injecting scheduling points. After applying these checks, we observed no false positives due to instrumentation in our evaluation of SECT (c.f. Section 8.3).

8.1.3 Input Fuzzing Loop

In order to explore schedules for concurrent code, it is a prerequisite that we have a given input workload which exhibits concurrency. Rather than generating

```
if (current->policy == SCHED_EXT
  && get_current_state() == TASK_RUNNING
  && !current->non_block_count
  && !is_idle_task(current)
  && preempt_count() == 0
  && !irqs_disabled()
  && rcu_preempt_depth() == 0 ) {
      ...
      yield();
}

```

Figure 8.2: The safety check for additional scheduling points injected by SECT instrumentation.

these workloads manually, we delegate the task of identifying these workloads to an existing kernel fuzzer, Syzkaller [76], with some modifications. In the case of Syzkaller, the format for a kernel workload is a system call sequence. We modify Syzkaller to selectively duplicate a random *subset* of system calls twice, running each duplicate asynchronously on a worker pool of threads. During execution, we establish a synchronization barrier to ensure that the selected system calls begin execution simultaneously, and annotate each thread with the `SCHED_EXT` policy so that it will be scheduled by our custom serializing scheduler. The remaining system calls are executed under the normal scheduling policy of the kernel. As a result, we pay the cost of serialization overhead and controlled scheduling *only* for that small subset of system calls. We run each sequence generated in this way M times to allow for exploration of multiple schedules, before moving on to the next system call sequence.

We exemplify the mutation and execution process in Figure 8.1. At step ③, a syscall program with three syscalls, $S1, S2, S3$ is mutated to create a new program with duplicated calls for $S1$ and $S3$ that are marked as asynchronous. Each of these syscalls has some events (i.e. instruction blocks) $e1, e2, e3$ and $e4, e5, e6$ which are separated by scheduling points. At runtime (⑤), the events of $S1$ (`async`) and $S2$ (`async`) are interleaved by the SECT scheduler (⑥), while all other syscalls and events execute normally.

8.2 Implementation

We instantiate SECT's scheduler as a C/eBPF program. In total, the SECT scheduler framework is a 790 line eBPF/C program, with another 96 lines of C

code for the command line utility. Each scheduling algorithm is implemented as individual C/eBPF files, which are included by the SCHED_EXT scheduler at compile time. In total, the algorithms together add another 345 lines of code in aggregate to the SCHED_EXT. The SCHED_EXT fuzzing loop builds on top of Syzkaller, changing 358 lines. Lastly, we implement the instrumentation as a LLVM pass in 409 lines of C++ code. The helper function that invokes the scheduler is injected by the pass. It consists of an additional 23 lines of C code, including necessary safety checks. We show the simplified snippet of safety checks, including those extracted from kernel debug assertions (c.f. Section 8.1.2) in Figure 8.2.

Our LLVM pass instruments load/store instructions on cross-thread variables, identified by their scope (`syncScope::System`) in LLVM IR. For semantic scheduling algorithms such as POS, the pass sends a message to the scheduler before yielding the CPU. This message encodes the target memory address and whether the operation is destructive (i.e., a store instruction), using scheduling-policy-specific fields in the task struct.

8.3 Evaluation

In this section, we evaluate SCHED_EXT on recent versions of the Linux kernel in comparison with the state-of-the-art fuzzers. We seek to answer the following research questions:

RQ1 How much overhead is introduced by SCHED_EXT?

RQ2 How does SCHED_EXT perform in bug detection and reproduction?

8.3.1 Experiment Setup

Kernel under Test. We choose the Linux kernel versions supporting the `sched_ext` feature as our evaluation targets, such as Linux 6.13-rc4. The kernels are compiled using the same complication configurations recommended by syzbot [216], with `KCOV` enabled for collecting coverage feedback and `KASAN` enabled for detecting memory corruption. We note that using `KASAN` incurs significant performance overhead, ranging from around 1.1x up to roughly 2.5x over native execution [42].

While not inherent to SECT, which can be paired with other test-oracles, `KASAN` is the defacto standard for detecting *high-impact* memory safety issues in the Linux kernel, and is estimated to be responsible for detecting nearly 1/3 of all bugs found by Syzkaller [42]. While using a weaker, but faster oracle (such as only detecting native kernel panics without `KASAN`), would lead to higher testing throughput, it might result in many concurrency related memory safety issues going undetected. We leave investigation into these tradeoffs among test-oracles to future work, adopting the default bug-finding configuration for kernel fuzzing for our experiments.

Baselines. We compare SECT with Snowboard [75] (commit `3295f82`), SegFuzz [98] (commit `e44fe05`), and Syzkaller [76] (commit `f919f20`). Snowboard and SegFuzz are state-of-the-art concurrency kernel fuzzers, while Syzkaller is the most widely used sequential-oriented fuzzer. We manually ported these fuzzers to the recent kernel version. Unfortunately, we found Snowboard to have two severe practical limitations: (1) it required a prohibitively large amount of resources for pre-processing, including roughly 100Tb of disk space and (2) the pre-processing analysis of Potential Memory Conflicts by Snowboard does not support many newer kernel features, several of which are present in the reproducers for bugs in our benchmark. As such, we only compare against Snowboard's throughput and not its bug-finding ability. We evaluate three CCT algorithms in SECT —SECT_RW, SECT_POS, SECT_PCT— as well as an ablative baseline, SECT-. The baseline SECT- consists of SECT_RW without any additional injected scheduling points (c.f. Section 8.1.2). We exclude the RP algorithm as POS is a refinement of it. Unless otherwise noted, SECT refers to the default algorithm of RW.

Experimental Parameters. For each fuzzing experiment, we conduct no less than 10 trials of 48 hours to enable statistical reproducibility. For a fair comparison, we configure Syzkaller, SegFuzz and SECT with the same resources, including 2 cores and 2 GB of memory for each VM. In addition, all the fuzzers use the same seed corpus according to [175]. As input seeds for all fuzzers and all trials, we use the same fixed set of Syzkaller programs scraped from the `syzbot` issue tracker. We used commit `5b49b39` of our artifact, which we plan to release publicly, for all experiments.

Hardware. All experiments were conducted on a 64-bit Ubuntu 22.04 LTS (6.8.0-52-generic) machine with Intel Xeon Platinum 8468V (Model 143) with 96

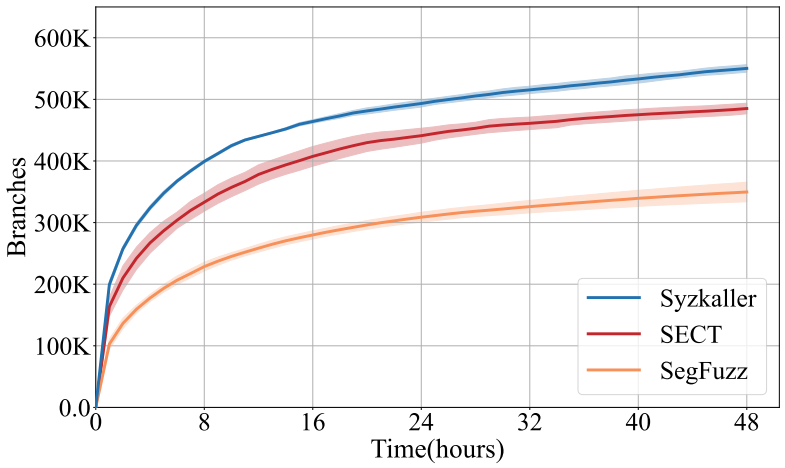


Figure 8.3: Code coverage achieved by Syzkaller, SegFuzz and SECT (N=10).

Table 8.2: Average fuzzing throughput and concurrent testcase-latency over 48 hours (N=10).

Type	**Fuzzer**	**Exec. Time (ms/test)**	**Throughput (#tests)**
Sequential Oriented	Syzkaller	89	8.3M
Concurrency Oriented	Snowboard	2498	-
	SegFuzz	228	0.38M
	SECT	99	4.4M

physical cores and 504 GB of RAM.

8.3.2 Code Coverage and Performance Overhead

To answer **RQ1**, we examine the throughput and latency of each fuzzer. Table 8.2 shows the results. In terms of the latency per test-case, SECT's the overhead of executing a given syscall input is roughly 11% more than that of sequential Syzkaller while achieving a 72% reduction compared to other concurrency fuzzers on average. We consulted the authors of Snowboard and they confirmed that a significant drop in performance was expected for their fuzzer as a result of the heavyweight control it imposes at the hypervisor level. In terms of total throughput, SECT has roughly half that of Syzkaller. SegFuzz also has greatly reduced throughput, achieving only 5% of Syzkaller's throughput. A contributing factor to this poor performance is a result of how it enforces scheduling control: only a single instance of SegFuzz can be run at a time in each VM due to its reliance on a limited number of hardware

breakpoints. In contrast, Syzkaller and SKI are able to run multiple instances within a single VM, providing better scalability (**C1**).

We also monitored the fuzzing process and recorded the branch coverage achieved by Syzkaller, SegFuzz and SKI. Figure 8.3 illustrates the comparison of branch coverage obtained by each fuzzer. As shown in the figure, SKI achieves 38% higher branch coverage than SegFuzz. In comparison to the sequential-oriented fuzzer Syzkaller, SKI and SegFuzz reach 88% and 63% of its branch coverage, respectively. We explain SKI's coverage reduction compared with Syzkaller with three factors. First, concurrency fuzzers allocate a portion of the computing resources to exploring thread interleavings, which generally contributes little to branch coverage. As a result, fewer resources are available for sequential fuzzing, leading to reduced coverage. Second, the schedule for CCT incurs additional overhead for execution and reduces the throughput. Third, SKI has a higher chance to trigger concurrency issues (discussed in Section 8.3.3), which causes a costly VM reboot and also diminishes the throughput.

8.3.3 Scheduling Effectiveness

To answer **RQ2**, we construct a ground-truth benchmark consisting of known concurrency bugs in the Linux kernel. We collect these bugs by referencing prior work [80] or analyzing Linux commit messages in the past two years based on heuristic rules (e.g., `^(?=.syzbot)(?=.race).*`). We excluded entries that lacked an associated patch or proof-of-concept, manually ported the remaining bugs to our evaluation kernel, and verified their reproducibility. On the ground-truth dataset, we evaluate two key capabilities: (1) the number of concurrency bugs discovered within a given time budget, reflecting the overall bug-finding capability of the fuzzers, and (2) the number of attempts required to reliably reproduce a concurrency bug from a given sequential input, highlighting the effectiveness of the underlying schedulers.

8.3.3.1 Results: Bug-Finding

In our first configuration, we evaluate the effectiveness of SKI in bug finding. In this setup, the fuzzer must explore *both* the sequential input space and the thread interleaving space to find the bugs. At the end of the time budget, we identify which bugs in the benchmark have been discovered, and record their cumulative

Table 8.3: Known concurrency bugs found by SegFuzz, Syzkaller, SCT (N=20). Note that SCT_RW is default SCT.

ID	Vulnerability	SegFuzz	Syzkaller	SCT-	SCT_RW	SCT_POS	SCT_PCT
1	CVE-2023-31083 [168]	-	-	-	-	-	-
2	CVE-2024-42111 [169]	-	-	● (3)	● (8)	● (9)	●(10)
3	CVE-2024-44941 [170]	-	●(10)	●(12)	● (1)	● (2)	● (3)
4	CVE-2024-49903 [171]	-	●(13)	●(11)	● (7)	●(10)	● (9)
5	CVE-2024-50125 [172]	-	● (4)	● (5)	● (1)	● (2)	● (1)
6	CVE-2024-57900 [173]	-	-	-	● (1)	-	-
7	cb2239c1 [132]	-	-	-	-	-	-
8	61179292 [130]	-	-	-	● (1)	● (4)	● (1)
9	3b9bc84d [129]	●(15)	●(16)	●(18)	●(16)	●(19)	●(19)
10	88b1afbf [131]	-	●(15)	●(12)	●(12)	●(18)	●(15)
	Total	1	5	6	8	7	7

occurrences across multiple trials. We present the results of this experiment in Table 8.3. Over a 48-hour period, SCT finds up to eight out of ten bugs across all trials, depending on the algorithm used. In contrast, Sykaller, SCT- and SegFuzz find only five, six and one bugs, respectively. Two bugs in the benchmark remained undiscovered by all fuzzers.

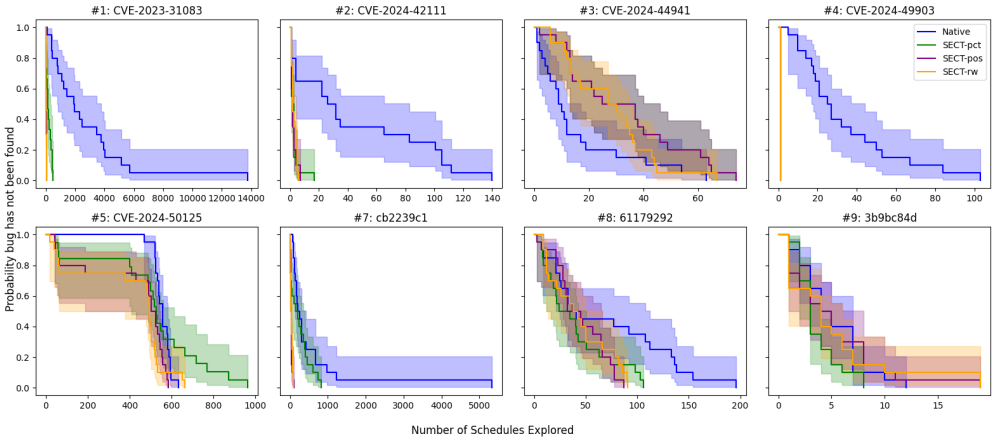


Figure 8.4: Probability that the bug is not found after a number of schedules have been explored (lower is better) (N=20).

Perhaps surprisingly, despite finding more bugs than Syzkaller, SCT finds some bugs *significantly less* consistently (e.g. #3, $p \leq 0.018$)[1]. This can be attributed to the inherently reduced throughput of concurrency oriented fuzzers; SCT has fewer chances to generate sufficient test cases to reach the buggy code path. Indeed, as is further evaluated in Section 8.3.3.2, the SCT scheduler is more effective at

[1]Using the chi-squared test with $\alpha = 0.05$ for SCT _PCT and Syzkaller

producing buggy interleavings. Thus the variation in the consistency stems from differences in throughput. In other words, SECT pays a cost in terms of consistency of *input-space* fuzzing for increased focus on the *interleaving-space*. SegFuzz lags behind all other tools in our benchmark, finding only one bug in all 20 trials. We have previously reached out to the authors of SegFuzz and received assistance in configuring their tool. We attribute its poor performance to its extremely low throughput.

Without additionally injected scheduling points, we see that SECT- can find fewer bugs overall than SECT. However, even lacking this extra granularity, SECT- is still able to find one bug (#2) that Syzkalller is unable to find in any trials. However, the limited scheduling points significantly constrain its ability to expose more stringent bug-triggering conditions (e.g., bug #2 and #8). These ablation results highlight the effectiveness of SECT-scheduler and the benefit of injecting additional scheduling points.

8.3.3.2 Results: Bug-Reproduction

In addition to bug-finding, we further assess the effectiveness of the proposed CCT scheduler in thread interleaving exploration. To do so, we feed the SECT scheduler with syscall sequences which have been reported as triggering the corresponding concurrency bugs. We then record the number of execution attempts required by the scheduler before the kernel crash is triggered. We use the native OS scheduler as our baseline. Bugs without an available syscall program is omitted in the experiment. We present the Kaplan-Meier [105] survival curves for each scheduler in Figure 8.4. These plots show the probability that a bug has *not been found yet* (y-axis) after a certain number of schedules have been explored (x-axis). As a result, curves which are lower and reach zero faster indicate superior bug reproduction ability.

While the number of schedules needed to expose each bug varies considerably, the SECT scheduler —regardless of the specific scheduling algorithm used— largely reproduces these bugs in as many or fewer attempts compared to the default OS scheduler. Overall, SECT achieves an 11.4× speed-up in bug reproduction on average. In some cases, such as for bug #1 and #7, this difference can be substantial. It takes up to 13,000 trials to find bug #1 with the native OS scheduler, but SECT always finds this bug in less than 500 trials. We also found that the PCT algorithm

performs significantly worse than the RW algorithm in some cases, such as bug #1 and bug #7 ($p < 0.005$)[2], while the RW and POS algorithms show minimal differences in performance on the benchmark. However, the visual difference seen for bug #5 is not significant due to higher variance ($p < 0.53$)[3]. Additionally, there is not a strong differentiation between RW and POS on these benchmarks. This may be due to the tendency of the PCT and POS algorithms to induce starvation of some tasks. While these algorithms may be suitable for testing user-space code, implicit fairness assumptions can be stronger in kernel code, resulting in highly similar error paths when they are violated. We hypothesize that more sophisticated fair scheduling algorithms [245] may have greater success in this context as future work.

8.4 Real-world Bug Finding

We employed SCT on multiple versions of the Linux kernel for evaluation. For crashes, we performed triage as follows: (1) internet search to discard duplicate reports (2) minimize the syz-programs (3) perform manual triage to validate the issue. Through this process, we have identified 6 previously unrecognized bugs in the Linux kernel, shown in Table 8.4. Among these, five have already been fixed, and one have been confirmed by kernel developers.

Table 8.4: Kernel discovered by SCT

ID	Version	Bug Type	Location	Subsystem	Status
1	6.13-rc4	use-after-free read	sl_sync	driver/net/slip	confirmed
2	6.13-rc4	null-ptr-deref	jfs_ioc_trim	fs/jfs	fixed
3	sched_ext	RCU anno missing	xa_get_order	kernel/mm	fixed
4	sched_ext	deadlock	pernet_ops_rwsem	kernel/sched	fixed
5	sched_ext	memory leak	scx_ops_enable	kernel/sched	fixed
6	sched_ext	deadlock	lockdep_assert_rq_held	kernel/sched	fixed

8.4.1 Case Study: Use-after-free in `sl_sync`

This bug is a concurrent use-after-free in `sl_sync`. It is caused by a kernel thread improperly freeing a Serial Line IP Device (`slip_dev`) without first acquiring an `rtnl_lock`. The use occurs when attempting to clean up existing open SLIP

[2]Using the log-rank test [145] with $\alpha = 0.05$

[3]Significance determined by the log-rank test with $\alpha < 0.05$

channels. Average number of trials to reproduce this bug appears in Figure 8.5. SCT reproduces this bug in an order of magnitude fewer schedules than default OS scheduler.

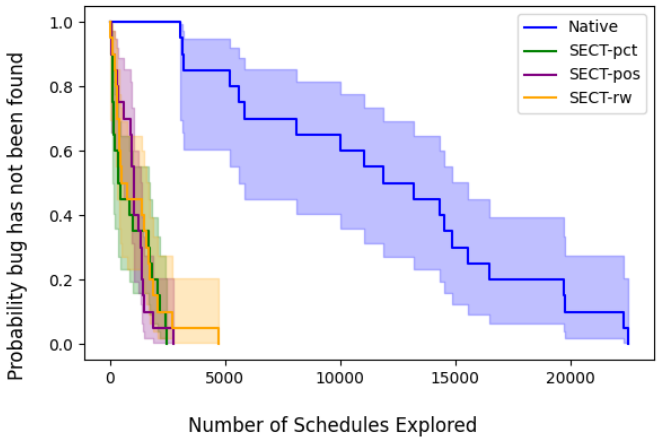


Figure 8.5: Kaplan-Meier survival plot for a newly confirmed use-after-free bug found by SCT in `sl_sync` (N=20).

8.4.2 Case Study: Partial Fix in JFS

To show the need for a CCT tool for the Linux kernel, we examine a concurrency bug that we found in the `fs/jfs` module that results in a null pointer dereference. It is closely related to a recent bug which had been reported as fixed [133]. The previous issue was a use-after-free (UAF) due to missing synchronization in the `jfs_ioc_trim` function that allowed it to interleave execution with `dbUnmount`. We show both functions in Figure 8.6 with the partial fix highlighted in blue and the additional lines needed for a complete fix in yellow. The UAF was triggered when `dbUnmount` freed `bmap` in one thread while `jfs_ioc_trim` still had a reference to it in another thread. It was resolved by adding a read-write semaphore such that the free cannot interleave between obtaining the pointer and dereferencing it in `jfs_ioc_trim`. Unfortunately, the accepted patch was still susceptible to a null pointer dereference, which was not caught by kernel developers but was found by SCT. In particular, `JFS_SBI(ipbmap->i_sb)->bmap` is set to null in `dbUnmount` after it is freed. Thus, while the UAF bug can no longer occur, if the critical section in `dbUnmount` executes first, the resulting pointer will be null instead of the old address. As the pointer is dereferenced in `jfs_ioc_trim`, this leads to a null

pointer dereference under some interleavings. We observed that this bug cannot be reproduced by the default OS scheduler in 10,000 executions of a syscall sequence under which the bug was observed; with SECT, we are always able to reproduce the issue in less than ten executions. This fix also demonstrates the need for more concurrency testing.

```
--- a/fs/jfs/jfs_discard.c
+++ b/fs/jfs/jfs_discard.c
int dbUnmount(struct inode *ipbmap, int mounterror) {
         // ...
         kfree(bmp);
         JFS_SBI(ipbmap->i_sb)->bmap = NULL;
}

--- a/fs/jfs/jfs_discard.c
+++ b/fs/jfs/jfs_discard.c
int jfs_ioc_trim(...) {
- struct bmap *bmp = JFS_SBI(ip->i_sb)->bmap;
+ struct bmap *bmp;
   // ...
+         down_read(&sb->s_umount);
+         bmp = JFS_SBI(ip->i_sb)->bmap;

-         if (minlen > bmp->db_agsize ||
+         if (bmp == NULL ||
+             minlen > bmp->db_agsize ||
      start >= bmp->db_mapsize ||
-             range->len < sb->s_blocksize)
+             range->len < sb->s_blocksize) {
+                 up_read(&sb->s_umount);
   return -EINVAL;
+        }
   // ...
+        up_read(&sb->s_umount);
  range->len = trimmed << sb->s_blocksize_bits;
  return 0;
}
```

Figure 8.6: A patch for a partial fix for bug in `jfs` filesystem highlighted in blue. The additional changes needed for a complete fix are highlighted in yellow.

8.5 Limitations

Theoretical Soundness and Completeness. The CCT approach taken by SECT is theoretically sound and complete in that it can instantiate any sequentially consistent schedule that is feasible for the OS-scheduler and also never emits infeasible schedules. The SECT implementation itself, though, may miss some synchronization operations in the kernel due to inline assembly code or locations spuriously marked

as unsafe for preemption by the safety checks; both cases could result in false negative schedules which are feasible for the OS but cannot be produced by SECT. If the safety checks instead incorrectly report a context switch to be safe, it is also possible for SECT to report false positive bugs. However, when evaluating SECT on recent versions of the Linux kernel, we manually triaged more than 500 crash reports and did not see *any* reports that we could trace to false positives from unsafe preemptions.

Semantic Extensibility. We have enabled SECT to provide some semantic information to the scheduling algorithms (e.g. memory address accessed), however, if a new scheduling algorithm requires different semantic information, this would require changes to SECT beyond simply implementing the standalone algorithm in eBPF code.

8.6 Discussion

SECT is a practical and effective framework for detecting and triggering concurrency bugs in kernel code. We hope that our lightweight design for extensibility and forward compatibility will allow developers and researchers to leverage it to increase the robustness of operating system kernels. In particular, we hope to unify a fragmented research ecosystem, lowering the barrier to entry for new experimentation with different concurrency testing algorithms in real-world, large-scale concurrent code in the Linux kernel. More broadly, such a system enables a horizontal expansion of fuzzing to a new, critical domain.

Part III

Related Work and Conclusion

Chapter 9

Related Work

9.1 Fuzzing and Cyber Reasoning Systems

Fuzzing User Surveys Prior qualitative and quantitative studies have examined the fuzzing process from a user perspective. Challenges in fuzzing have previously been identified by expert researchers and users at a recent Shonan meeting [23]. Similarly, Nourry et al. [167] constructed an extensive taxonomy on the specific challenges faced by fuzzing experts. In Chapter 3, we identify some key challenges in fuzzing which largely agree with some findings in these prior publications. Namely, (1) that usability issues are the largest impediment to adoption among developers [167, 23] (2) that users of fuzzers prefer shorter time budgets than those typically seen in academic literature [167], and that (3) these users also want to be able to detect classes of bugs not easily detectable by existing fuzzers and sanitizers [23]. However, we also provide several novel results and findings not found in these prior studies; for example, that a majority of software professionals want to run fuzzers as part of their local development workflow, and that we establish novel, user-expectation based timing bounds for fuzzers in different workflows. More generally, the questions from our study are not focused on challenges in *existing* fuzzer workflows, but rather on desired use cases to bring fuzzing to *new* users. Nearly all participants from [167] and [23] are deeply ingrained in existing workflows, having either attended a small conference for fuzzing researchers or participated in online discussions about OSS-Fuzz, whereas our users bring a fresh perspective on adaptations needed to drive adoption in fuzzing. Also, unlike these two works, we conduct a follow up study based on key outcomes from our survey results (c.f. Chapter 4).

Empirical User Studies of Automated Test Generation Techniques Other researchers have conducted fuzzer usability studies with students [186, 187] and in industrial contexts [126]. We identified usability as the largest obstacle to fuzzer adoption. These studies are complementary to our survey in that they provide insights into the specific usability challenges that users encounter with fuzzers. Indeed, while a substantial portion of developers in our study (43%) indicated that they are willing to write a custom fuzzing harness, in practice, many users struggle with this task using existing fuzzing tools [186, 187, 126]. Future research in automatic fuzz-harness generation [107, 16] could mitigate these issues in practice. Likely a combination of improving these usability issues identified by other work and meeting expectations of software professionals will be needed to shift fuzzers left in development workflows.

[66] conducted a user study for the closely related technique of automated *unit test* generation with EvoSuite [65]. They found that bug detection was *not* easier for developers using automatically generated unit tests, despite a significant increase in code coverage from these tests. Our survey has a similar goal, in that we are attempting to identify how to make automated testing tools (in this case, fuzzers) a useful component of the software lifecycle. We conduct an empirical evaluation as a follow up (Section 4.1). However, we did not evaluate the usability of fuzzers for developers (as has been discussed in aforementioned prior work), but rather the efficacy of available fuzzers for a novel application, namely commit fuzzing in CI/CD systems.

Regression and Unit Fuzzing Dynamic analysis has emerged as one of the primary ways of combating regression bugs [29, 152], which may comprise as much as 77% of vulnerabilities in open source projects [246]. As a result, many researchers and industry leaders have begun to investigate and use fuzzers in local and CI/CD settings. Google has created CIFuzz[1] for continuous integration fuzzing for OSSFuzz and other projects. Other companies like Gitlab[2] and CodeIntelligence[3] have similar offerings. A case study of CI fuzzing in the Linux kernel has even been recently conducted [206]. From academic research, regression fuzzers target recently changed

[1] https://google.github.io/oss-fuzz/getting-started/continuous-integration/
[2] https://docs.gitlab.com/ee/user/application_security/coverage_fuzzing
[3] https://www.code-intelligence.com/cli-tool

code with modified basic-block distance-to-change based power schedules [246, 243] or in combination with shadow symbolic execution [164]. These tools are similar in concept and design to directed fuzzers [26, 138, 146], which target one or more code locations. Our follow-up study in Chapter 4 evaluates the current abilities of general purpose, directed and regression fuzzers, for commit-level fuzzing.

Fuzzer Evaluation Configurations Past recommendations for evaluation configurations have primarily been derived from observations of existing fuzzer capabilities [113, 82]. In contrast, we ask developers a priori how much time they would be willing to allocate to fuzzing tools in various contexts. We believe that these user expectations can be combined with practical limitations to provide more representative guidelines for evaluations in the future. Several other evaluations of fuzzers in the context of regression or directed fuzzing have been conducted in parallel with this work. [108] curated a benchmark of OSS Fuzz reported bugs for evaluating regression fuzzers. However, they omit bugs that are found in less than three minutes. This choice is appropriate for determining which fuzzers are best at finding challenging regression bugs, but diminishes the ability of fuzzers of finding regression bugs quickly in general. Additionally, Kim and Hong use the seed corpus present in the version control history for the bug-inducing commit of the given program. In Chapter 4, we use this setup as our baseline *static* corpus, but also compare results to a *saturated* corpus up to the bug-inducing commit. Indeed, our results indicate that the setup used by [109] is *not optimal* for change-based fuzzing. [114] also conduct an evaluation of fuzzers in the CI/CD context, similarly finding that many regression bugs can be found relatively quickly by existing fuzzers (< 15 minutes). However, unlike our study in Chapter 4, their research focuses only on the capabilities of general-purpose grey-box fuzzers, omitting regression and directed fuzzers. Instead of using a saturated corpus on a single commit, they simulate a sequence of commits starting from the corpus of the first bug-inducing commit and augmenting it cumulatively. As neither approach is an exact replication of long-term continuous fuzzing, we believe that results from both works can be informative and complementary. [109] assessed the state of directed fuzzer *evaluations*, excluding general purpose and regression fuzzers, and do so in a more conventional context with 24 hour timeouts rather than CI/CD or local development workflows.

Vulnerability Repair Code generation [14] and bug repairs [57] have been some of the applications of LLMs that can be utilized in a CRS. Early results indicate that LLM-based repair tools tend to outperform traditional repair tools on established datasets [233]. Similarly, LLMs have begun to be utilized for localizing the fault [104], with promising initial results. Agentic systems can automatically explore a project or invoke subtasks via repeated interactions with the LLMs [244, 195].

LLMs in Vulnerability Discovery Large Language models have also shown their capability in generating test harnesses [134] and unit tests [120]. Furthermore, they have been effective in fuzzing network protocols by providing the fuzzer a grammar for mutating the input messages [155]. In addition to dynamic analysis, LLMs are also beginning to see success in augmenting static analysis, particularly for bug discovery [124].

CRS Benchmarks SWEBench [101] is a widely used benchmark containing natural language description GitHub issues to be resolved by LLMs. These issues are mostly bug-fixes, but can span multiple files. Other benchmarks have even been developed to specifically evaluate LLM capabilities and risks [224]. To date, available benchmarks still only evaluate CRS sub-components, rather than a complete workflow; we hope that they can provide a starting point for a general CRS benchmark.

9.2 Concurrency

Controlled Scheduling Controlled concurrency testing explores the space of feasible interleavings by explicitly serializing the execution. This approach is closely related to *Deterministic Multi-Threading* (DMT), such as D-Threads [135], Kendo [174] and CoreDet [19]. To intercept possibly visible heap-memory operations, some prior approaches to controlled scheduling leverage dynamic binary instrumentation [239]. In all works in this thesis, we use more efficient static instrumentation, either at the binary [55] or LLVM level.

For kernel code, SKI [64] and Snowboard [75] control the schedule via hypercalls which suspend and resume CPUs of the entire virtualized system under test. However,

these hypercalls are extremely expensive and thus not scalable to large execution traces. SegFuzz [98] utilizes a combination of hardware breakpoints and hypercalls to achieve more efficient scheduling. While the hardware breakpoints are more generally more performant, their limited number means that instances of SegFuzz cannot share virtual machines, and thus each fuzzing instance incurs full virtualization overhead. We compare these approaches to our lightweight scheduling mechanism via EBPF in Chapter 8.

Stateless Randomized Algorithms for Concurrency Testing Many approaches employ a randomized thread scheduler to expose bugs, foregoing exhaustive enumeration for performance, and often probabilistic guarantees [32, 245, 241]. Randomized approaches recognize that while the interleaving space is vast, many interleavings exhibit similar concurrent behaviors. A prominent strategy to design such schedulers is to identify a reduced search space, often backed by a hypothesis about the existence of bug patterns. Randomized schedulers categorize interleavings into distinct classes and sample one execution per class. PCT[32, 162] exemplifies this, grouping interleavings with the same preemption behaviors and randomly selects executions across diverse preemption behaviors. PPCT [162] the multi-core counterpart of PCT, accelerates fuzzing campaigns without compromising exploration efficiency. Partial Order Sampling (POS) [241] improves on its predecessor RAPOS [201] by uniformly sampling executions across different partial orders. Recent extensions of randomized methodologies encompass weak memory programs [68, 139] and distributed systems [240, 176, 177], building on the foundation of PCT and POS. These approaches contrast with the approach taken in this thesis in that they are all *stateless*; they do not persist information across executions to bias the search, akin to black-box fuzzing.

Stateful Randomized Algorithms for Concurrency Testing Our work is not the first to use a stateful approach to concurrency testing. Maple [239] tracks the coverage of several interleaving "idioms" across different executions. These idioms represent concurrent memory access patterns of up to four events, and are derived from the small-scope hypothesis [189] and observations on a dataset of 17 concurrency bugs [239]. In algorithms developed in this thesis, the stateful coverage

metric is more complete, subsuming the idioms used in Maple as chains of reads-from or reads-from-value dependencies. Importantly, neither feedback function developed in this thesis is limited to only a small number of events or context switches. Indeed, as shown in Chapter 7, the main *benefit* of statefulness is to find concurrency bugs with *high* preemption depth. Thus using a coverage metric which is limited to fewer preemptions is likely not an optimal approach. Q-learning [160] has also been used as a form of stateful guidance in randomized concurrency testing. Like fuzzing, it supports a general feedback function, though prior work used application specific feedback functions such as the contents of a particular message queue or data-structure rather than a more general coverage metric such as reads-from or reads-from-value. We compare a fuzzing-style approach with Q-learning directly in Chapter 6, however these two broader frameworks are similar and we believe they could be potentially unified in the future. The primary novel contributions of this thesis over prior stateful approaches are in structured schedule representation (Chapter 6) and directedness (Chapter 7).

Systematic Algorithms for Concurrency Testing (Model Checking) In the realm of concurrency testing, model checking assumes a deterministic stance and meticulously searches all possible program executions. Approaches such as [161, 189] explore preemption or context-bounded program executions. PERIOD [226] partitions the program into M chunks, executing all N-orderings of those M in a fixed small number of N "periods" (where typically $M >> N$). We have presented experimental comparison of `RFF` with PERIOD in Chapter 6. Dynamic partial-order reduction (DPOR) help reduce the set of program executions by categorizing executions into equivalent classes and considering one interleaving from each equivalence class [5, 62]. DPOR techniques have further been improved to be *optimal* with respect to coarser equivalence classes [116, 11, 6, 7]. Our experiments show favorable comparison of a fuzzing-style randomized search vis-à-vis state-of-the-art (stateless) model checker GenMC [118, 117]. We attribute this to the large number of program executions any enumerative search approach needs to consider, even with a possibly optimal reduction [45]. By using a random search augmented with partial order information, approaches like `RFF` can explore the space of interleavings relatively evenly in practice with respect to witnessing different

partial-order equivalence classes.

Static and dynamic analyses Static analyses explore multiple program paths and interleavings at once, often through abstract interpretation [198] and pointer analysis [214, 213] directly on the source code. These analyses provide foundational information that enable static detection of concurrency bugs [21, 48, 36]. However, generally static race detection is extremely challenging in the general case. As a result such analyses often have scalability or precision issues, and are typically foregone in favor of more precise and efficient dynamic analyses.

These dynamic analysis tools, such as ThreadSanitizer [202, 203], aim to detect traditional concurrency bugs such as data races, and deadlocks by analyzing executions of concurrent software. While their detection ability may be limited to the observed interleaving, predictive methods for data race detection [111, 179, 149, 148, 193], deadlocks [218] and atomicity violations [150] have been developed. Such techniques generalize the execution observed during testing to a larger class of *correct or equivalent reorderings* [209, 58] and look for bugs in these, thereby enhancing the coverage of dynamic analysis without having to rerun the underlying program. Predictive analyses have also been developed for more properties beyond races, deadlocks and atomicity violations [87, 10]. We remark that despite the enhanced coverage, runtime predictive analyses do not consider executions that fall outside of the causal purview of an originally observed execution. In the setting of full system testing, we believe predictive analysis is *complementary* to concurrency testing tools, in that they provide additional traces for race-detection and checking other properties over each execution.

Concurrency Bugs. Prior work has categorized concurrency bug patterns in terms of their empirical occurrence in real-world software [137, 125]. Preemption depth has been previously studied as a contributor to bug difficulty in these works, and has also been leveraged to bound the search space for concurrency testing and model checking approaches [189, 161, 32]. We believe we are the first to draw the explicit connection between stateful randomized scheduling algorithms such as DCF, Maple [239], Q-Learning [160] and `RFF` and preemption depth. The related problem of state density arises from these stateful scheduling approaches. While

the state-explosion problem is commonly known for model-checking approaches [70], we highlight its importance for stochastic concurrency testing. Finally, aspects of the scheduling space has also been used as a measure of difficulty in [125]. However, there the focus is only on *tight* context switching windows, which are difficult for *uncontrolled* testing approaches. Our notion of scheduling breadth is more general; regardless of a large or small amount of noise in the scheduling space, a good scheduling algorithm should be robust to these kinds of programs.

Delay Injection and Concurrency-Aware Fuzzers Several works have attempted to explore the scheduling space opportunistically using primarily input-space fuzzers, enforcing only loose or intermittent control over the schedule. We dub this approach *concurrency-aware fuzzing*. Concurrency-aware fuzzers have been applied to user space programs [41], kernels [97, 234] and file systems [100]. AutoInter-fuzzing [115] employs static analysis to generate concurrent memory access pairs, controlling *only* operations from each pair in different orders to trigger concurrent bugs. MUZZ [41] emphasizes extremely low-overhead, exploring the interleaving space in an *uncontrolled* manner by randomly changing the OS thread priorities *only* at thread creation time. Other works focus on different modes of delay injection, such as happens-before inference [123] or exploring different thread execution speeds [40]. Conzzer [100] exercises *intermittent* control at runtime, using concurrent-function-call feedback to determine if it should trigger a delay in an adjacently executed function in a future execution. It only conducts a fine-grained, controlled analysis on demand for race detection between two functions on two threads. This means that it cannot detect bugs involving more than two threads or more than two functions, and that the fine grained analysis will not run for issues not involving data-races (or not detected as data races by their incomplete race detector).

Many kernel-oriented fuzzers are concurrency aware. The most prominent fuzzer used for detecting kernel bugs, Syzkaller [76], has two basic, ad-hoc tactics for finding concurrency bugs, namely (1) executing a program multiple times in parallel and (2) duplicating random system calls. However, Syzkaller imposes *no control* over the schedule during execution. Our work in Chapter 8 leverages these strategies, and combines them with controlled scheduling for improved concurrency fuzzing. Razzer [97], uses hardware breakpoints to attain *intermittent* control over only a

subset of events. Krace [234] enforces only *weak* control via delay injection.

In contrast to concurrency-aware fuzzing and delay injection, the approach taken in this thesis is to assume full, fine-grained control over the schedule. Full control enables (1) determinism and reproducibility and (2) a probabilistically complete exploration of the schedule space [245].

Chapter 10

Conclusion and Future Work

Software systems have grown increasingly complex, concurrent, and critical to modern society. Traditional approaches to ensuring software security and reliability of manual code audits, conventional testing, and even existing automated analysis techniques struggle to keep pace with this growth. This thesis has investigated how fuzzing, a highly successful technique for finding security bugs post-hoc in sequential programs, can be expanded both vertically and horizontally to address the challenges of our evolving software landscape. We identified these challenges through a comprehensive user study of software professionals, finding several avenues researchers can narrow the gap between research and practice. Our **vertical** expansion embeds fuzzing more deeply and naturally within the software development lifecycle. Rather than treating fuzzing as a post-development activity conducted by security specialists, we have explored how fuzzing can shift left into earlier development phases and integrate into automated, end-to-end security workflows. Our **horizontal** expansion of fuzzing extends fuzzing techniques beyond their traditional domain of sequential, stateless programs to handle the rich, concurrent, stateful behaviors that characterize modern software systems. By developing both new systems to enable fuzzing in these contexts, and applying novel, adaptive randomized search strategies, we have laid the foundation for a new generation of fuzzing tools that can be used by developers and security professionals alike.

10.1 Overview

User Survey First, through a comprehensive user survey of software professionals (Chapter 3), this thesis identified critical gaps between existing fuzzing tools and

the needs of practitioners who are not fuzzing experts. The survey revealed that developers strongly desire to use fuzzers in local development workflows (64% of participants) and CI/CD pipelines (75% of participants), with expected timeouts far shorter than the 24-hour campaigns common in research evaluations. Practitioners also emphasized the importance of diverse bug-finding beyond sequential memory safety issues, particularly for functional correctness and concurrency bugs. These findings motivated two complementary research directions explored in subsequent chapters: (1) making fuzzing practical for change-based development workflows, and (2) expanding fuzzing to detect *functional* concurrency bugs, which users identified as the second and third most important classes of bugs after memory safety.

Vertical Expansion: Integration and Automation This thesis has also advanced the vertical integration of fuzzing into software development and security workflows. We conducted a systematic evaluation of how existing general-purpose, directed, and regression fuzzers perform when testing individual code changes under the time constraints developers expect per our survey. Using a saturated corpus built from continuous fuzzing of earlier program versions, we found that state-of-the-art fuzzers can already find more than half of regression bugs within 5 minutes (the threshold for local development) and up to 60% within 1 hour (the threshold for CI/CD). However, our results also revealed that build-time overhead and initial corpus quality have dramatic impacts on effectiveness, suggesting important directions for future research. Zooming out, we also articulated a vision for Cyber Reasoning Systems (CRS) that autonomously discover, analyze, and repair vulnerabilities without human intervention. We presented a concrete instantiation of such a system and evaluated it on the available DARPA AIxCC competition benchmark, demonstrating that it can find multiple vulnerabilities and generate plausible patches within hours. Critically, we showed that LLM-integrated components were essential to the system's success: removing LLM-based input generation meant no bugs were found, and removing LLM-based repair meant no patches were generated. This highlights both the potential and the necessity of integrating modern AI techniques with traditional program analysis to achieve fully automated security workflows.

Horizontal Expansion: Fuzzing for Concurrent Systems A major contribution of this thesis is demonstrating that fuzzing-inspired biased random search can be effectively applied to the domain of concurrent program schedules. We developed a structured grey-box fuzzing approach that searches the space of "reads-from" relations rather than concrete thread interleavings. By representing executions as sets of abstract reads-from constraints, we dramatically reduce the search space of possible schedules. We also show we can obtain further reductions by leveraging *directedness*; by focusing the search on only a subset of concurrent behaviors of the program, we are able to avoid wasting our search budget on irrelevant behaviors and program states. To enable concurrency fuzzing, we developed multiple frameworks for concurrency testing of both user space programs (Section 6.3) and kernel code (Chapter 8). SECT, our EBPF scheduler, when combined with Syzkaller, achieved 38% higher branch coverage than the state-of-the-art concurrency for the Linux kernel fuzzer while introducing less overhead, and discovered eight previously unknown kernel bugs, five of which have been fixed by developers.

10.2 Perspectives

This thesis conceptually presents a unified view of fuzzing as a general-purpose framework for adaptive randomized search. In particular, we show that this framework is not just applicable to the traditional application of searching for program inputs, but also to the search over program schedules. Indeed, the success of our novel reads-from abstraction for schedule fuzzing mirrors the success of structured input fuzzing approaches that leverage grammars or input specifications. And our novel proposal for directedness in concurrency fuzzing translates many concepts from the related work in directed fuzzing to the domain of schedules. In both cases, the core principles remain consistent: reduce the search space by leveraging semantic structure, use lightweight feedback to bias the search toward novel or interesting behaviors, and avoiding exhaustive enumeration in favor of randomized methods. We hope that this formulation can serve as a basis for future research in concurrent system fuzzing, and as a template for applying fuzzing techniques to other new application domains.

This thesis also provides novel empirical insights, underlining the importance

of understanding and validating tools in realistic deployment contexts. Our user survey revealed substantial gaps between assumptions of the research community and concrete capabilities needed for broader adoption of fuzzing tools. Later, our evaluation of commit-level fuzzing showed that factors often treated as secondary in research (build times, initial corpus construction) can dominate practical effectiveness. These findings underscore the value of engaging with practitioners and evaluating tools under the constraints of real workflows. As the fuzzing research community continues to mature, we believe such engagement will become increasingly important to ensure that advances in the laboratory translate to impact in practice.

Lastly, this thesis presents several technical contributions. The vision of Cyber Reasoning Systems presented in Chapter 5 represents an ambitious goal: end-to-end automation of deep program analysis. While complete automation remains challenging, this thesis demonstrates that substantial progress is possible by carefully combining complementary techniques. Within our CRS, we saw that LLM-based analyses enable other stages of the vulnerability discovery and remediation process, such as program repair and static analysis, to match the level of automation achieved by fuzzing. Additionally, our work in enabling fuzzing for concurrent systems such as the Linux kernel, is a step towards applying automated testing to the vast quantity of real-world systems which are beyond the reach of existing tools. More concretely, this thesis contributes several fundamental artifacts to the open-source fuzzing community, including a user-space concurrency fuzzer[1], as well as a full-fledged, open-source Cyber Reasoning System prototype implementation[2].

10.3 Future Directions

Expanding the Concurrency Bug Remediation Pipeline In this thesis, we have presented multiple approaches to *finding* concurrency bugs (Chapter 6, 7) However, we have also emphasized the importance of integrating bug-finding with bug triage and repair workflows (Chapter 5). While we have made progress in this direction for sequential software in our development of a CRS, there is no such yet

[1] `https://github.com/dylanjwolff/RFF`
[2] `https://github.com/DARPA-AIxCC/asc-crs-healing-touch`

system for *concurrent* software. Can we leverage the kind of Controlled Concurrency Testing tools and techniques developed in this thesis for analyses that go beyond bug-finding? Can we construct a CRS framework to discover, triage and repair *concurrency* bugs? Will LLMs prove to be as effective in this domain as they have been for sequential software? So much software exists as reactive, concurrent systems in distributed deployments – developing a comprehensive remediation pipeline for these systems has the opportunity for an even broader impact than sequential CRSs.

Simultaneous Input and Schedule Exploration We have presented two approaches to schedule exploration in this thesis which require a *fixed* program input (c.f. Chapter 6, 7). Such approaches can readily be combined with input fuzzing via a naive alternating search between input and schedule space, as in Chapter 8. However, there is an opportunity to explore a more tightly integrated fuzzer which can explore both new schedules and new inputs *simultaneously*, raising many interesting research questions. What balance of concurrent and sequential feedback should be used to bias the search? How can we maintain an input-invariant schedule representation? By careful combination of these two approaches, we can achieve a more autonomous concurrency fuzzing framework which doesn't require a human to provide bespoke workloads for schedule exploration.

Revisiting Delay Injection This thesis has focused on *controlled* concurrency testing because it provides deterministic reproducibility of schedules and can guarantee that particular desired schedules are indeed explored. Delay injection, is a contrasting technique that does not have these benefits. However, it does allow for schedule exploration without full control of the system and at relatively low overhead. In removing the need for full control, delay injection is applicable across a wide variety of software ecosystems and environments without needing complex tooling. However, there have been many advances in controlled scheduling algorithms in recent years, both those presented in this thesis and beyond [241, 245], yet delay injection strategies remain relatively primitive. Can we approximate controlled concurrency testing algorithms effectiveness by only injecting delays without full control of the scheduler? Can we attain any probabilistic guarantees of completeness from delay injection alone? If we can achieve similar effectiveness without full control

of the schedule, this would facilitate a broader application of concurrency fuzzing in practice.

10.4 Concluding Remarks

Software complexity continues to grow, driven by increasing system scale, the proliferation of concurrent and distributed architectures, and now the advent of AI-generated code. Ensuring the security and reliability of our software infrastructure requires automated reasoning techniques that can keep pace with this growth. This thesis has examined this question of automation at scale through the lens of fuzzing approaches. We find that fuzzing can be a cornerstone of this automation if carefully expanded into new domains and workflows.

The work presented here represents steps toward a future in which vulnerability discovery and remediation are routine, largely automated activities integrated naturally into the software development lifecycle. While substantial challenges remain, the convergence of advances in program analysis, fuzzing techniques, and artificial intelligence suggests that this future is increasingly within reach. By continuing to engage with practitioners, the research community can build the foundation for holistic, automated validation infrastructure that our software ecosystem demands.

Bibliography

[1] Oct. 2016. [Online]. Available: https://resources.sei.cmu.edu/library/asset-view.cfm?assetID=507974.

[2] Aug. 2020. [Online]. Available: https://web.archive.org/web/20201001073424/community.peachfuzzer.com/.

[3] [Online]. Available: https://github.com/search?q=fuzzer.

[4] [Online]. Available: https://semgrep.dev/.

[5] P. Abdulla, S. Aronis, B. Jonsson, and K. Sagonas, "Optimal dynamic partial order reduction", *ACM SIGPLAN Notices*, vol. 49, no. 1, pp. 373–384, 2014.

[6] P. A. Abdulla, M. F. Atig, B. Jonsson, M. Lång, T. P. Ngo, and K. Sagonas, "Optimal stateless model checking for reads-from equivalence under sequential consistency", *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 1–29, 2019.

[7] P. Agarwal, K. Chatterjee, S. Pathak, A. Pavlogiannis, and V. Toman, "Stateless model checking under a reads-value-from equivalence", in *International Conference on Computer Aided Verification*, Springer, 2021, pp. 341–366.

[8] D. Aitel, "An introduction to spike, the fuzzer creation kit", *presentation slides), Aug*, vol. 1, 2002.

[9] "American fuzzy lop (AFL)", https://github.com/google/AFL.

[10] Z. Ang and U. Mathur, "Predictive monitoring against pattern regular languages", *Proc. ACM Program. Lang.*, vol. 8, no. POPL, Jan. 2024. [Online]. Available: https://doi.org/10.1145/3632915.

[11] S. Aronis, B. Jonsson, M. Lång, and K. Sagonas, "Optimal dynamic partial order reduction with observers", in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2018, pp. 229–248.

[12] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "Nautilus: Fishing for deep bugs with grammars." In *NDSS*, 2019.

[13] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Restler: Stateful rest api fuzzing", in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, IEEE, 2019, pp. 748–758.

[14] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models", 2021. arXiv: `2108.07732 [cs.PL]`. [Online]. Available: `https://arxiv.org/abs/2108.07732`.

[15] J. Ba, G. J. Duck, and A. Roychoudhury, "Efficient greybox fuzzing to detect memory errors", in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, Association for Computing Machinery, 2023. [Online]. Available: `https://doi.org/10.1145/3551349.3561161`.

[16] D. Babić, S. Bucur, Y. Chen, F. Ivančić, T. King, M. Kusano, C. Lemieux, L. Szekeres, and W. Wang, "Fudge: Fuzz driver generation at scale", in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019, Tallinn, Estonia: Association for Computing Machinery, 2019, pp. 975–985, ISBN: 9781450355728.

[17] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanović, T. King, A. Reynolds, and C. Tinelli, "Cvc4", in *International Conference on Computer Aided Verification*, Springer, 2011, pp. 171–177.

[18] Beck, "Test Driven Development: By Example", USA: Addison-Wesley Longman Publishing Co., Inc., 2002, ISBN: 0321146530.

[19] T. Bergan, O. Anderson, J. Devietti, L. Ceze, and D. Grossman, "Coredet: A compiler and runtime system for deterministic multithreaded execution", in *Proceedings of the Fifteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XV, Pittsburgh, Pennsylvania, USA: Association for Computing Machinery,

2010, pp. 53–64, ISBN: 9781605588391. [Online]. Available: https://doi.org/10.1145/1736020.1736029.

[20] A. Biere, A. Cimatti, E. Clarke, and Y. Zhu, "Symbolic model checking without bdds", in *International conference on tools and algorithms for the construction and analysis of systems*, Springer, 1999, pp. 193–207.

[21] S. Blackshear, N. Gorogiannis, P. W. O'Hearn, and I. Sergey, "RacerD: Compositional static race detection", *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–28, 2018.

[22] T. Blazytko, M. Bishop, C. Aschermann, J. Cappos, M. Schlögel, N. Korshun, A. Abbasi, M. Schweighauser, S. Schinzel, S. Schumilo, et al., "{Grimoire}: Synthesizing structure while fuzzing", in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1985–2002.

[23] M. Böhme, C. Cadar, and A. Roychoudhury, "Fuzzing: Challenges and reflections", *IEEE Software*, vol. 38, no. 3, 2021.

[24] M. Böhme and B. Falk, "Fuzzing: On the exponential cost of vulnerability discovery", in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 713–724.

[25] M. Böhme, V. J. Manès, and S. K. Cha, "Boosting fuzzer efficiency: An information theoretic perspective", in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 678–689.

[26] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing", in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17, Dallas, Texas, USA: Association for Computing Machinery, 2017, pp. 2329–2344, ISBN: 9781450349468.

[27] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain", in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[28] P. Boonstoppel, C. Cadar, and D. Engler, "Rwset: Attacking path explosion in constraint-based test generation", in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2008, pp. 351–366.

[29] L. Braz, E. Fregnan, V. Arora, and A. Bacchelli, "An exploratory study on regression vulnerabilities", in *Proceedings of the 16th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '22, Helsinki, Finland: Association for Computing Machinery, 2022, pp. 12–22, ISBN: 9781450394277.

[30] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners", *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[31] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L.-J. Hwang, "Symbolic model checking: 1020 states and beyond", *Information and computation*, vol. 98, no. 2, pp. 142–170, 1992.

[32] S. Burckhardt, P. Kothari, M. Musuvathi, and S. Nagarakatte, "A randomized scheduler with probabilistic guarantees of finding bugs", *SIGPLAN Not.*, vol. 45, no. 3, pp. 167–178, Mar. 2010, ISSN: 0362-1340. [Online]. Available: https://doi.org/10.1145/1735971.1736040.

[33] C. Cadar, D. Dunbar, D. R. Engler, et al., "Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs." In *OSDI*, vol. 8, 2008, pp. 209–224.

[34] C. Cadar and K. Sen, "Symbolic execution for software testing: Three decades later", *Communications of the ACM*, vol. 56, no. 2, pp. 82–90, 2013.

[35] Y. Cai, B. Zhu, R. Meng, H. Yun, L. He, P. Su, and B. Liang, "Detecting concurrency memory corruption vulnerabilities", in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 706–717.

[36] Y. Cai, P. Yao, and C. Zhang, "Canary: Practical static detection of inter-thread value-flow bugs", in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021, pp. 1126–1140.

[37] S. Canakci, N. Matyunin, K. Graffi, A. Joshi, and M. Egele, "Targetfuzz: Using darts to guide directed greybox fuzzers", in *Proceedings of the 2022 ACM on Asia conference on computer and communications security*, 2022, pp. 561–573.

[38] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramer, and C. Zhang, "Quantifying memorization across neural language models", in *The Eleventh International Conference on Learning Representations*, 2022.

[39] B. Caswell, C. Keita, and J. Ranweiler, "Project onefuzz", Oct. 2020. [Online]. Available: `https://www.microsoft.com/en-us/research/project/project-onefuzz/`.

[40] D. Chen, Y. Jiang, C. Xu, X. Ma, and J. Lu, "Testing multithreaded programs via thread speed control", in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 15–25.

[41] H. Chen, S. Guo, Y. Xue, Y. Sui, C. Zhang, Y. Li, H. Wang, and Y. Liu, "{Muzz}: Thread-aware grey-box fuzzing for effective bug hunting in multi-threaded programs", in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2325–2342.

[42] M. Cho, D. An, H. Jin, and T. Kwon, "{Bokasan}: Binary-only kernel address sanitizer for effective kernel fuzzing", in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4985–5002.

[43] N. Chomsky and M. P. Schützenberger, "The algebraic theory of context-free languages", in *Studies in Logic and the Foundations of Mathematics*, vol. 26, Elsevier, 1959, pp. 118–161.

[44] D. Churchill, "Keynotes", in *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 2018, pp. 23–25.

[45] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Progress on the state explosion problem in model checking", in *Informatics: 10 Years Back, 10 Years Ahead*, Springer, 2001, pp. 176–194.

[46] L. Cordeiro and B. Fischer, "Verifying multi-threaded software using smt-based context-bounded model checking", in *Proceedings of the 33rd International Conference on Software Engineering*, 2011, pp. 331–339.

[47] J. Corina, A. Machiry, C. Salls, Y. Shoshitaishvili, S. Hao, C. Kruegel, and G. Vigna, "Difuze: Interface aware fuzzing for kernel drivers", in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 2123–2138.

[48] A. Costea, A. Tiwari, S. Chianasta, K. R, A. Roychoudhury, and I. Sergey, "Hippodrome: Data race repair using static analysis summaries", *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 32, no. 2, pp. 1–33, 2023.

[49] E. Daka and G. Fraser, "A survey on unit testing practices and problems", in *2014 IEEE 25th International Symposium on Software Reliability Engineering*, IEEE, 2014, pp. 201–211.

[50] L. De Moura and N. Bjørner, "Z3: An efficient smt solver", in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2008, pp. 337–340.

[51] J. Dejaeghere, B. Gbadamosi, T. Pulls, and F. Rochet, "Comparing security in ebpf and webassembly", in *Proceedings of the 1st Workshop on EBPF and Kernel Extensions*, ser. eBPF '23, New York, NY, USA: Association for Computing Machinery, 2023, pp. 35–41, ISBN: 9798400702938. [Online]. Available: https://doi.org/10.1145/3609021.3609306.

[52] P. Deligiannis, A. Senthilnathan, F. Nayyar, C. Lovett, and A. Lal, "Industrial-strength controlled concurrency testing for c# programs with coyote", in *Tools and Algorithms for the Construction and Analysis of Systems: 29th International Conference, TACAS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Paris, France, April 22–27, 2023, Proceedings, Part II*, Paris, France: Springer-Verlag, 2023,

pp. 433–452, ISBN: 978-3-031-30819-2. [Online]. Available: https://doi.org/10.1007/978-3-031-30820-8_26.

[53] D. Distefano, M. Fähndrich, F. Logozzo, and P. W. O'Hearn, "Scaling static analyses at facebook", *Commun. ACM*, vol. 62, no. 8, pp. 62–70, Jul. 2019, ISSN: 0001-0782.

[54] Y. Dodge, "Binomial test", in *The Concise Encyclopedia of Statistics*. New York, NY: Springer New York, 2008, pp. 47–49, ISBN: 978-0-387-32833-1.

[55] G. J. Duck, X. Gao, and A. Roychoudhury, "Binary rewriting without control flow recovery", in *ACM SIGPLAN Sympsoium on Programming Language Design and Implementation (PLDI)*, 2020.

[56] T. Elmas, S. Qadeer, and S. Tasiran, "Goldilocks: Efficiently computing the happens-before relation using locksets", in *International Workshop on Formal Approaches to Software Testing*, Springer, 2006, pp. 193–208.

[57] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models", in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, 2023, pp. 1469–1481.

[58] A. Farzan and U. Mathur, "Coarser equivalences for causal concurrency", *Proceedings of the ACM on Programming Languages*, vol. 8, no. POPL, pp. 911–941, 2024.

[59] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "Afl++: Combining incremental steps of fuzzing research", in *Proceedings of the 14th USENIX Conference on Offensive Technologies*, ser. WOOT'20, USA: USENIX Association, 2020.

[60] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti, "Libafl: A framework to build modular and reusable fuzzers", in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1051–1065.

[61] C. Flanagan and S. N. Freund, "Fasttrack: Efficient and precise dynamic race detection", in *ACM-SIGPLAN Symposium on Programming Language Design and Implementation*, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:2696584.

[62] C. Flanagan and P. Godefroid, "Dynamic partial-order reduction for model checking software", *ACM Sigplan Notices*, vol. 40, no. 1, pp. 110–121, 2005.

[63] M. Fleischer, D. Das, P. Bose, W. Bai, K. Lu, M. Payer, C. Kruegel, and G. Vigna, "ACTOR: Action-Guided kernel fuzzing", in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 5003–5020, ISBN: 978-1-939133-37-3. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/fleischer.

[64] P. Fonseca, R. Rodrigues, and B. B. Brandenburg, "SKI: Exposing kernel concurrency bugs through systematic schedule exploration", in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, Broomfield, CO: USENIX Association, Oct. 2014, pp. 415–431, ISBN: 978-1-931971-16-4. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/fonseca.

[65] G. Fraser and A. Arcuri, "Evosuite: Automatic test suite generation for object-oriented software", in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 2011, pp. 416–419.

[66] G. Fraser, M. Staats, P. McMinn, A. Arcuri, and F. Padberg, "Does automated unit test generation really help software testers? a controlled empirical study", *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 24, no. 4, pp. 1–49, 2015.

[67] "Fuzzbench 2021-06-02-klee report", Jun. 2021. [Online]. Available: https://www.fuzzbench.com/reports/experimental/2021-06-02-klee/index.html.

[68] M. Gao, S. Chakraborty, and B. K. Ozkan, "Probabilistic concurrency testing for weak memory programs", in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 603–616.

[69] X. Gao, G. J. Duck, and A. Roychoudhury, "Scalable fuzzing of program binaries with e9afl", in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, IEEE Press, 2022, pp. 1247–

1251. [Online]. Available: https://doi.org/10.1109/ASE51524.2021.9678913.

[70] P. Godefroid, "Partial-order methods for the verification of concurrent systems: an approach to the state-explosion problem", Springer, 1996.

[71] P. Godefroid, "Software model checking: The verisoft approach", *Formal Methods in System Design*, vol. 26, pp. 77–101, 2005.

[72] P. Godefroid, "Random testing for security: Blackbox vs. whitebox fuzzing", in *Proceedings of the 2nd international workshop on Random testing: co-located with the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2007)*, 2007, pp. 1–1.

[73] P. Godefroid, "Fuzzing: Hack, art, and science", *Commun. ACM*, vol. 63, no. 2, pp. 70–76, Jan. 2020, ISSN: 0001-0782. [Online]. Available: https://doi.org/10.1145/3363824.

[74] P. Godefroid, B.-Y. Huang, and M. Polishchuk, "Intelligent rest api data fuzzing", in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 725–736.

[75] S. Gong, D. Altinbüken, P. Fonseca, and P. Maniatis, "Snowboard: Finding kernel concurrency bugs through systematic inter-thread communication analysis", in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP '21, Virtual Event, Germany: Association for Computing Machinery, 2021, pp. 66–83, ISBN: 9781450387095. [Online]. Available: https://doi.org/10.1145/3477132.3483549.

[76] Google, "Syzkaller", https://github.com/google/syzkaller, 2015.

[77] Google, "GitHub - LevelDB: a fast key-value storage library that provides an ordered mapping from string keys to string values", https://github.com/google/leveldb, Accessed 01-12-2025, 2025.

[78] P. L. Gorski, L. L. Iacono, D. Wermke, C. Stransky, S. Möller, Y. Acar, and S. Fahl, "Developers deserve security warnings, too: On the effect of integrated security advice on cryptographic API misuse", in *Fourteenth Symposium on Usable Privacy and Security (SOUPS 2018)*, Baltimore, MD: USENIX

Association, Aug. 2018, pp. 265–281, ISBN: 978-1-939133-10-6. [Online]. Available: https://www.usenix.org/conference/soups2018/presentation/gorski.

[79] C. L. Goues, M. Pradel, and A. Roychoudhury, "Automated program repair", *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[80] T. Han, X. Gong, and J. Liu, "CARDSHARK: Understanding and stablizing linux kernel concurrency bugs against the odds", in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, Aug. 2024, pp. 6203–6218, ISBN: 978-1-939133-44-1. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/han-tianshuo.

[81] X. Y. Han, "Custom kernel scheduler with ebpf for concurrency fuzzing", Master's Thesis, National University of Singapore, Apr. 2024.

[82] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, and A. L. Hosking, "Seed selection for successful fuzzing", in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2021, Virtual, Denmark: Association for Computing Machinery, 2021, pp. 230–243, ISBN: 9781450384599.

[83] M. Hilton, N. Nelson, T. Tunnell, D. Marinov, and D. Dig, "Trade-offs in continuous integration: Assurance, security, and flexibility", in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017, Paderborn, Germany: Association for Computing Machinery, 2017, pp. 197–207, ISBN: 9781450351058.

[84] S. Hocevar, "Samhocevar/zzuf: Application fuzzer", [Online]. Available: https://github.com/samhocevar/zzuf.

[85] C. Holloman, "AWS Outage: Billions Lost, Multi-Cloud Is Wall Street's Solution", *Forbes*, Oct. 2025. [Online]. Available: https://www.forbes.com/sites/christerholloman/2025/10/20/aws-outage-billions-lost-multi-cloud-is-wall-streets-solution/.

[86] M. Howard and S. Lipner, "The security development lifecycle", Microsoft Press Redmond, 2006, vol. 8.

[87] J. Huang, Q. Luo, and G. Rosu, "Gpredict: Generic predictive concurrency analysis", in *37th IEEE/ACM International Conference on Software Engineering, ICSE 2015, Florence, Italy, May 16-24, 2015, Volume 1*, A. Bertolino, G. Canfora, and S. G. Elbaum, Eds., IEEE Computer Society, 2015, pp. 847–857. [Online]. Available: https://doi.org/10.1109/ICSE.2015.96.

[88] Hwaci, "HC-Tree", https://sqlite.org/hctree/doc/hctree/doc/hctree/index.html, Accessed 01-12-2025, 2025.

[89] Hwaci, "SQLite", https://sqlite.org, Accessed 01-12-2025, 2025.

[90] "Infer static analyzer: Infer: Infer", [Online]. Available: https://fbinfer.com/.

[91] P. Institute, "Costs and consequences of gaps in vulnerability response", 2024. [Online]. Available: https://www.servicenow.com/lpayr/ponemon-vulnerability-survey.html.

[92] jacoco, "Jacoco/jacoco", Oct. 2019. [Online]. Available: https://github.com/jacoco/jacoco.

[93] J. Jaffar, R. Maghareh, S. Godboley, and X.-L. Ha, "Tracerx: Dynamic symbolic execution with interpolation (competition contribution)", *Fundamental Approaches to Software Engineering*, vol. 12076, p. 530, 2020.

[94] N. Jalbert, C. Pereira, G. Pokam, and K. Sen, "{Radbench}: A concurrency bug benchmark suite", in *3rd USENIX Workshop on Hot Topics in Parallelism (HotPar 11)*, 2011.

[95] Y. Javed, S. Sethi, and A. Jadoun, "Alexa's voice recording behavior: A survey of user understanding and awareness", in *Proceedings of the 14th International Conference on Availability, Reliability and Security*, ser. ARES '19, Canterbury, CA, United Kingdom: Association for Computing Machinery, 2019, ISBN: 9781450371643.

[96] Y. Jeon, W. Han, N. Burow, and M. Payer, "Fuzzan: Efficient sanitizer metadata design for fuzzing", in *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*, USENIX Association, 2020.

[97] D. R. Jeong, K. Kim, B. Shivakumar, B. Lee, and I. Shin, "Razzer: Finding kernel race bugs through fuzzing", in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 754–768.

[98] D. R. Jeong, B. Lee, I. Shin, and Y. Kwon, "Segfuzz: Segmentizing thread interleaving to discover kernel concurrency bugs through fuzzing", in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2104–2121.

[99] X. Jiacheng, Z. Xuhong, J. Shouling, T. Yuan, Z. Binbin, W. Qinying, C. Peng, and C. Jiming, "Mock: Optimizing kernel fuzzing mutation with context-aware dependency", in *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*, The Internet Society, 2024.

[100] Z.-M. Jiang, J.-J. Bai, K. Lu, and S.-M. Hu, "Context-sensitive and directional concurrency fuzzing for data-race detection",, Jan. 2022.

[101] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?", in *The Twelfth International Conference on Learning Representations*.

[102] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "Inferfix: End-to-end program repair with llms", in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023, San Francisco, CA, USA: Association for Computing Machinery, 2023, pp. 1646–1656, ISBN: 9798400703270.

[103] J. D. Kalbfleisch and R. L. Prentice, "The statistical analysis of failure time data", John Wiley & Sons, 2002.

[104] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization", *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1424–1446, Jul. 2024, ISSN: 2994-970X.

[105] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations", *Journal of the American statistical association*, vol. 53, no. 282, pp. 457–481, 1958.

[106] B. Kasikci, C. Zamfir, and G. Candea, "Data races vs. data race bugs: Telling the difference with portend", *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 185–198, 2012.

[107] M. Kelly, C. Treude, and A. Murray, "A case study on automated fuzz target generation for large codebases", in *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2019, pp. 1–6.

[108] J. Kim and S. Hong, "Poster: Bugoss: A regression bug benchmark for empirical study of regression fuzzing techniques", in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2023, pp. 470–473.

[109] T. E. Kim, J. Choi, S. Im, K. Heo, and S. K. Cha, "Evaluating directed fuzzers: Are we heading in the right direction?", *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024.

[110] J. C. King, "Symbolic execution and program testing", *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.

[111] D. Kini, U. Mathur, and M. Viswanathan, "Dynamic race prediction in linear time", in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2017, Barcelona, Spain: Association for Computing Machinery, 2017, pp. 157–170, ISBN: 9781450349888. [Online]. Available: https://doi.org/10.1145/3062341.3062374.

[112] B. A. Kitchenham and S. L. Pfleeger, "Personal opinion surveys", in *Guide to Advanced Empirical Software Engineering*. London: Springer London, 2008, pp. 63–92, ISBN: 978-1-84800-044-5.

[113] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing", in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18, Toronto, Canada: Association for Computing Machinery, 2018, pp. 2123–2138, ISBN: 9781450356930.

[114] T. Klooster, F. Turkmen, G. Broenink, R. T. Hove, and M. Böhme, "Continuous fuzzing: A study of the effectiveness and scalability of fuzzing in ci/cd pipelines", in *2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)*, 2023, pp. 25–32.

[115] Y. Ko, B. Zhu, and J. Kim, "Fuzzing with automatically controlled interleavings to detect concurrency bugs", *Journal of Systems and Software*, vol. 191, p. 111 379, 2022.

[116] M. Kokologiannakis, I. Marmanis, V. Gladstein, and V. Vafeiadis, "Truly stateless, optimal dynamic partial order reduction", *Proceedings of the ACM on Programming Languages*, vol. 6, no. POPL, pp. 1–28, 2022.

[117] M. Kokologiannakis, A. Raad, and V. Vafeiadis, "Model checking for weakly consistent libraries", in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019, pp. 96–110.

[118] M. Kokologiannakis and V. Vafeiadis, "Genmc: A model checker for weak memory models", in *International Conference on Computer Aided Verification*, Springer, 2021, pp. 427–440.

[119] O. Legunsen, W. U. Hassan, X. Xu, G. Roşu, and D. Marinov, "How good are the specs? a study of the bug-finding effectiveness of existing java api specifications", in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '16, Singapore, Singapore: Association for Computing Machinery, 2016, pp. 602–613, ISBN: 9781450338455.

[120] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models", in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23, Melbourne, Victoria, Australia: IEEE Press, 2023, pp. 919–931, ISBN: 9781665457019.

[121] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks", *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[122] A. Li, B. Kang, V. Vikram, I. Laybourn, S. Dharanikota, S. Tiwari, and R. Padhye, "Fray: An Efficient General-Purpose Concurrency Testing Platform for the JVM", *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 4035–4063, 2025.

[123] G. Li, S. Lu, M. Musuvathi, S. Nath, and R. Padhye, "Efficient scalable thread-safety-violation detection: Finding thousands of concurrency bugs during testing", in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 162–180.

[124] H. Li, Y. Hao, Y. Zhai, and Z. Qian, "Enhancing static analysis for practical bug detection: An llm-integrated approach", *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 474–499, 2024.

[125] J. Liang, M. Yuan, Z. Ding, S. Ma, X. Han, and C. Zhang, "RaceBench: A triggerable and observable concurrency bug benchmark", in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2023, pp. 415–428.

[126] J. Liang, M. Wang, Y. Chen, Y. Jiang, and R. Zhang, "Fuzz testing in practice: Obstacles and solutions", in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018, pp. 562–566.

[127] "Libfuzzer", https://llvm.org/docs/LibFuzzer.html.

[128] D. van der Linden, P. Anthonysamy, B. Nuseibeh, T. T. Tun, M. Petre, M. Levine, J. Towse, and A. Rashid, "Schrödinger's security: Opening the box on app developers' security rationale", in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ser. ICSE '20, Seoul, South Korea: Association for Computing Machinery, 2020, pp. 149–160, ISBN: 9781450371216. [Online]. Available: `https://doi.org/10.1145/3377811.3380394`.

[129] "Linux commit log 3b9bc84d", `https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=3b9bc84d311104906d2b4995a9a02d7b7ddab` 2022.

[130] "Linux commit log 61179292", `https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=611792920925fb088ddccbe2783c7f92fdfb0` 2023.

[131] "Linux commit log 88b1afbf", `https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=88b1afbf0f6b221f6c5bb66cc80cd3b38d696` 2022.

[132] "Linux commit log cb2239c1", https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=cb2239c198ad9fbd5aced22cf93e45562da78 2023.

[133] "Linux commit log d6c1b3", https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=d6c1b3599b2feb5c7291f5ac3a36e5fa7cedb 2024.

[134] D. Liu, J. Metzman, O. Chang, and G. O. S. S. Team, "Ai-powered fuzzing: Breaking the bug hunting barrier", [Online]. Available: https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html.

[135] T. Liu, C. Curtsinger, and E. D. Berger, "Dthreads: Efficient deterministic multithreading", in *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, Association for Computing Machinery, 2011, pp. 327–336. [Online]. Available: https://doi.org/10.1145/2043556.2043587.

[136] E. S. Lowry and C. W. Medlock, "Object code optimization", *Communications of the ACM*, vol. 12, no. 1, pp. 13–22, 1969.

[137] S. Lu, S. Park, E. Seo, and Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics", in *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XIII, Seattle, WA, USA: Association for Computing Machinery, 2008, pp. 329–339, ISBN: 9781595939586. [Online]. Available: https://doi.org/10.1145/1346281.1346323.

[138] C. Luo, W. Meng, and P. Li, "Selectfuzz: Efficient directed fuzzing with selective path exploration", in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2693–2707.

[139] W. Luo and B. Demsky, "C11tester: A race detector for c/c++ atomics", in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 630–646.

[140] C. Lyu, S. Ji, C. Zhang, Y. Li, W.-H. Lee, Y. Song, and R. Beyah, "{Mopt}: Optimized mutation scheduling for fuzzers", in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1949–1966.

[141] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey", *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.

[142] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other", *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947, ISSN: 00034851. Accessed: Feb. 12, 2025. [Online]. Available: `http://www.jstor.org/stable/2236101`.

[143] H. B. Mann and D. R. Whitney, "On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other", *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[144] M. N. Mansur, M. Christakis, V. Wüstholz, and F. Zhang, "Detecting critical bugs in smt solvers using blackbox mutational fuzzing", *arXiv preprint arXiv:2004.05934*, 2020.

[145] N. Mantel et al., "Evaluation of survival data and two new rank order statistics arising in its consideration", *Cancer Chemother Rep*, vol. 50, no. 3, pp. 163–170, 1966.

[146] P. D. Marinescu and C. Cadar, "Katch: High-coverage testing of software patches", in *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, 2013, pp. 235–245.

[147] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "Restest: Automated black-box testing of restful web apis", in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 682–685.

[148] U. Mathur, D. Kini, and M. Viswanathan, "What happens-after the first race? enhancing the predictive power of happens-before based dynamic race detection", *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–29, 2018.

[149] U. Mathur, A. Pavlogiannis, and M. Viswanathan, "Optimal prediction of synchronization-preserving races", *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–29, 2021.

[150] U. Mathur and M. Viswanathan, "Atomicity checking in linear time using vector clocks", in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20, Lausanne, Switzerland: Association for Computing Machinery, 2020, pp. 183–199, ISBN: 9781450371025. [Online]. Available: https://doi.org/10.1145/3373376.3378475.

[151] A. Mazurkiewicz, "Introduction to trace theory", in *The book of traces*, 1995, pp. 3–41.

[152] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing", in *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2017, pp. 233–242.

[153] H. D. Menendez and D. Clark, "Hashing fuzzing: Introducing input diversity to improve crash detection", *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3540–3553, 2022.

[154] R. Meng, Z. Dong, J. Li, I. Beschastnikh, and A. Roychoudhury, "Linear-time temporal logic guided greybox fuzzing", in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 1343–1355, ISBN: 9781450392211.

[155] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing", in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.

[156] J. Metzman, L. Szekeres, L. Simon, R. Sprabery, and A. Arya, "Fuzzbench: An open fuzzer benchmarking platform and service", in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021,

Athens, Greece: Association for Computing Machinery, 2021, pp. 1393–1403, ISBN: 9781450385626.

[157] J. Micco, "Advances in continuous integration testing at google", 2018. [Online]. Available: https://research.google/pubs/pub46593.

[158] B. P. Miller, L. Fredriksen, and B. So, "An empirical study of the reliability of unix utilities", *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.

[159] M. H. N. Mohamed, X. Wang, and B. Ravindran, "Understanding the security of linux ebpf subsystem", in *Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems*, ser. APSys '23, Seoul, Republic of Korea: Association for Computing Machinery, 2023, pp. 87–92, ISBN: 9798400703058. [Online]. Available: https://doi.org/10.1145/3609510.3609822.

[160] S. Mukherjee, P. Deligiannis, A. Biswas, and A. Lal, "Learning-based controlled concurrency testing", *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–31, 2020.

[161] M. Musuvathi, S. Qadeer, T. Ball, G. Basler, P. A. Nainar, and I. Neamtiu, "Finding and reproducing heisenbugs in concurrent programs", in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'08, San Diego, California: USENIX Association, 2008, pp. 267–280.

[162] S. Nagarakatte, S. Burckhardt, M. M. Martin, and M. Musuvathi, "Multicore acceleration of priority-based schedulers for concurrency bug detection", in *Proceedings of the 33rd ACM SIGPLAN conference on Programming Language Design and Implementation*, 2012, pp. 543–554.

[163] S. Narayanasamy, Z. Wang, J. Tigani, A. Edwards, and B. Calder, "Automatically classifying benign and harmful data races using replay analysis", in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2007, pp. 22–31.

[164] Y. Noller, C. Pasareanu, M. Böhme, Y. Sun, H. L. Nguyen, and L. Grunske, "Hydiff: Hybrid differential software analysis", in *Proceedings of the 42nd*

ACM/IEEE International Conference on Software Engineering, ser. ICSE 2020, Seoul, South Korea, 2020, pp. 1273–1285.

[165] Y. Noller, R. Shariffdeen, X. Gao, and A. Roychoudhury, "Trust enhancement issues in program repair", in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 2228–2240, ISBN: 9781450392211.

[166] J. R. Norris, "Markov chains", Cambridge university press, 1998.

[167] O. Nourry, Y. Kashiwa, B. Lin, G. Bavota, M. Lanza, and Y. Kamei, "The human side of fuzzing: Challenges faced by developers during fuzzing activities", *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 1, Nov. 2023, ISSN: 1049-331X.

[168] NVD, "CVE-2023-31083", `https://nvd.nist.gov/vuln/detail/cve-2023-31083`, 2023.

[169] NVD, "CVE-2024-42111", `https://nvd.nist.gov/vuln/detail/cve-2023-42111`, 2024.

[170] NVD, "CVE-2024-44941", `https://nvd.nist.gov/vuln/detail/cve-2024-44941`, 2024.

[171] NVD, "CVE-2024-49903", `https://nvd.nist.gov/vuln/detail/cve-2023-49903`, 2024.

[172] NVD, "CVE-2024-50125", `https://nvd.nist.gov/vuln/detail/cve-2023-50125`, 2024.

[173] NVD, "CVE-2024-57900", `https://nvd.nist.gov/vuln/detail/cve-2023-57900`, 2024.

[174] M. Olszewski, J. Ansel, and S. Amarasinghe, "Kendo: Efficient deterministic multithreading in software", in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XIV, Washington, DC, USA: Association for Computing Machinery, 2009, pp. 97–108, ISBN: 9781605584065. [Online]. Available: `https://doi.org/10.1145/1508244.1508256`.

[175] P. B. Oswal, "Improving linux kernel fuzzing", English, Ph.D. dissertation, 2023, p. 43, ISBN: 9798379515645. [Online]. Available: https://www.proquest.com/dissertations-theses/improving-linux-kernel-fuzzing/docview/2812311865/se-2.

[176] B. K. Ozkan, R. Majumdar, F. Niksic, M. T. Befrouei, and G. Weissenbacher, "Randomized testing of distributed systems with probabilistic guarantees", *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–28, 2018.

[177] B. K. Ozkan, R. Majumdar, and S. Oraee, "Trace aware random testing for distributed systems", *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 1–29, 2019.

[178] R. Padhye, C. Lemieux, K. Sen, M. Papadakis, and Y. Le Traon, "Semantic fuzzing with zest", in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 329–340.

[179] A. Pavlogiannis, "Fast, sound, and effectively complete dynamic race prediction", *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–29, 2019.

[180] K. Pearson, "X. on the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling", *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 50, no. 302, pp. 157–175, Jul. 1900.

[181] D. Peled, "All from one, one for all: On model checking using representatives", in *5th International Conference on Computer Aided Verification (CAV)*, 1993.

[182] J. Peng and R. J. Williams, "Incremental multi-step q-learning", in *Machine Learning Proceedings 1994*, Elsevier, 1994, pp. 226–232.

[183] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: A greybox fuzzer for network protocols", in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 460–465.

[184] V.-T. Pham, M. Böhme, A. E. Santosa, A. R. Caciulescu, and A. Roychoudhury, "Smart greybox fuzzing", *IEEE Transactions on Software Engineering*, 2019.

[185] Q.-S. Phan, K.-H. Nguyen, and T. Nguyen, "The challenges of shift left static analysis", in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2023, pp. 340–342.

[186] S. Plöger, M. Meier, and M. Smith, "A qualitative usability evaluation of the clang static analyzer and libFuzzer with CS students and CTF players", in *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*, USENIX Association, Aug. 2021, pp. 553–572, ISBN: 978-1-939133-25-0. [Online]. Available: https://www.usenix.org/conference/soups2021/presentation/ploger.

[187] S. Plöger, M. Meier, and M. Smith, "A usability evaluation of afl and libfuzzer with cs students", in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ser. CHI '23, Hamburg, Germany: Association for Computing Machinery, 2023, ISBN: 9781450394215.

[188] S. Poeplau and A. Francillon, "Symbolic execution with SymCC: Don't interpret, compile!", in *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, Aug. 2020, pp. 181–198, ISBN: 978-1-939133-17-5. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/poeplau.

[189] S. Qadeer and J. Rehof, "Context-bounded model checking of concurrent software", in *Tools and Algorithms for the Construction and Analysis of Systems, 11th International Conference, TACAS 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005, Proceedings*, N. Halbwachs and L. D. Zuck, Eds., ser. Lecture Notes in Computer Science, vol. 3440, Springer, 2005, pp. 93–107. [Online]. Available: https://doi.org/10.1007/978-3-540-31980-1%5C_7.

[190] "Quickcheck", [Online]. Available: `https://hackage.haskell.org/package/QuickCheck`.

[191] RaceBenchData, "RaceBenchData: A synthetic concurrency bug dataset", `https://github.com/rb130/RaceBenchData`, Accessed 20-04-2024, 2023.

[192] N. Risse and M. Böhme, "Uncovering the limits of machine learning for automatic vulnerability detection", in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 4247–4264.

[193] J. Roemer, K. Genç, and M. D. Bond, "Smarttrack: Efficient predictive race detection", in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2020, pp. 747–762.

[194] P. Royston and M. K. Parmar, "Restricted mean survival time: An alternative to the hazard ratio for the design and analysis of randomized trials with a time-to-event outcome", *BMC medical research methodology*, vol. 13, pp. 1–15, 2013.

[195] H. Ruan, Y. Zhang, and A. Roychoudhury, "Specrover: Code intent extraction via llms", in *International Conference on Software Engineering (ICSE)*, 2025.

[196] P. Runeson, "A survey of unit testing practices", *IEEE Software*, vol. 23, no. 4, pp. 22–29, 2006.

[197] M. Schreier, "Qualitative content analysis in practice", Sage publications, 2012.

[198] M. Schwarz, S. Saan, H. Seidl, K. Apinis, J. Erhard, and V. Vojdani, "Improving thread-modular abstract interpretation", in *International Static Analysis Symposium*, Springer, 2021, pp. 359–383.

[199] "Sctbench: A set of c/c++ pthread benchmarks for evaluating concurrency testing techniques", https://github.com/mc-imperial/sctbench, Accessed: 2023-07-01, 2016.

[200] K. Sen, "Concolic testing", in *Proceedings of the twenty-second IEEE/ACM international conference on Automated software engineering*, 2007, pp. 571–572.

[201] K. Sen, "Effective random testing of concurrent programs", in *Proceedings of the 22nd IEEE/ACM international conference on Automated software engineering*, 2007, pp. 323–332.

[202] K. Serebryany and T. Iskhodzhanov, "Threadsanitizer: Data race detection in practice", in *Proceedings of the Workshop on Binary Instrumentation and Applications*, ser. WBIA '09, New York, New York, USA: Association for Computing Machinery, 2009, pp. 62–71, ISBN: 9781605587936. [Online]. Available: https://doi.org/10.1145/1791194.1791203.

[203] K. Serebryany, A. Potapenko, T. Iskhodzhanov, and D. Vyukov, "Dynamic race detection with llvm compiler", in *Runtime Verification*, S. Khurshid and K. Sen, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 110–114, ISBN: 978-3-642-29860-8.

[204] "OSS-Fuzz: Google's Continuous Fuzzing Service for Open-Source Software", Vancouver, BC: USENIX Association, Aug. 2017.

[205] A. Sharma, C. Cadar, and J. Metzman, "Effective fuzzing within ci/cd pipelines (registered report)", in *Proceedings of the 3rd ACM International Fuzzing Workshop*, 2024, pp. 52–60.

[206] H. Shi, R. Wang, Y. Fu, M. Wang, X. Shi, X. Jiao, H. Song, Y. Jiang, and J. Sun, "Industry practice of coverage-guided enterprise linux kernel fuzzing", in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2019, Tallinn, Estonia: Association for Computing Machinery, 2019, pp. 986–995, ISBN: 9781450355728.

[207] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis", in *IEEE Symposium on Security and Privacy*, 2016.

[208] Shuttle, "GitHub - awslabs/shuttle: A library for testing concurrent Rust code", https://github.com/awslabs/shuttle, Accessed 20-04-2024, 2021.

[209] Y. Smaragdakis, J. Evans, C. Sadowski, J. Yi, and C. Flanagan, "Sound predictive race detection in polynomial time", in *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL '12, Philadelphia, PA, USA: Association for Computing Machinery, 2012, pp. 387–400, ISBN: 9781450310833. [Online]. Available: https://doi.org/10.1145/2103656.2103702.

[210] L. Smith, "Shift-Left Testing — drdobbs.com", https://www.drdobbs.com/shift-left-testing/184404768, [Accessed 25-10-2023], 2001.

[211] J. Spadaro and L. Wyatt, "The mutiny fuzzing framework and decept proxy", Feb. 2021. [Online]. Available: https://blogs.cisco.com/security/talos/mutiny-decept.

[212] D. Steinhöfel and A. Zeller, "Input invariants", in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 583–594.

[213] Y. Sui, P. Di, and J. Xue, "Sparse flow-sensitive pointer analysis for multithreaded programs", in *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 2016, pp. 160–170.

[214] Y. Sui and J. Xue, "Svf: Interprocedural static value-flow analysis in llvm", in *Proceedings of the 25th international conference on compiler construction*, ACM, 2016, pp. 265–266.

[215] "Summary of the Amazon DynamoDB Service Disruption in the Northern Virginia (US-EAST-1) Region", en-US. [Online]. Available: https://aws.amazon.com/message/101925/.

[216] "Syzbot dashboard", [Online]. Available: https://syzkaller.appspot.com/upstream/fixed.

[217] "This Week's AWS Crash Made Smart Beds Overheat, Get Stuck In Wrong Position", en, Oct. 2025. [Online]. Available: https://www.pcmag.com/news/this-weeks-aws-crash-made-smart-beds-overheat-get-stuck-in-wrong-position.

[218] H. C. Tunç, U. Mathur, A. Pavlogiannis, and M. Viswanathan, "Sound dynamic deadlock prediction in linear time", *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, Jun. 2023. [Online]. Available: https://doi.org/10.1145/3591291.

[219] A. Vargha and H. D. Delaney, "Comparing several robust tests of stochastic equality.", 2000.

[220] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong", *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[221] J. S. Vitter, "Random sampling with a reservoir", *ACM Transactions on Mathematical Software (TOMS)*, vol. 11, no. 1, pp. 37–57, 1985.

[222] D. Votipka, K. R. Fulton, J. Parker, M. Hou, M. L. Mazurek, and M. Hicks, "Understanding security mistakes developers make: Qualitative analysis from build it, break it, fix it", in *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, Aug. 2020, pp. 109–126, ISBN: 978-1-939133-17-5. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/votipka-understanding.

[223] A. Wald, "Tests of statistical hypotheses concerning several parameters when the number of observations is large", *Transactions of the American Mathematical society*, vol. 54, no. 3, pp. 426–482, 1943.

[224] S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, S. Whitman, S. Ding, V. Ionescu, Y. Li, and J. Saxe, "Cyberseceval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models", 2024. arXiv: 2408.01605 [cs.CR]. [Online]. Available: https://arxiv.org/abs/2408.01605.

[225] "Welcome to hypothesis!", [Online]. Available: https://hypothesis.readthedocs.io/en/latest/.

[226] C. Wen, M. He, B. Wu, Z. Xu, and S. Qin, "Controlled concurrency testing via periodical scheduling", in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association

for Computing Machinery, 2022, pp. 474–486, ISBN: 9781450392211. [Online]. Available: https://doi.org/10.1145/3510003.3510178.

[227] F. Wilcoxon, "Individual comparisons by ranking methods", *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[228] T. Winters, T. Manshreck, and H. Wright, "Software engineering at google: Lessons learned from programming over time", O'Reilly Media, 2020, ch. 1, p. 17.

[229] D. Wolff, Z. Shi, G. J. Duck, U. Mathur, and A. Roychoudhury, "Greybox fuzzing for concurrency testing", in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24, La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 482–498, ISBN: 9798400703850. [Online]. Available: https://doi.org/10.1145/3620665.3640389.

[230] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization", *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[231] S. C. Woo, M. Ohara, E. Torrie, J. P. Singh, and A. Gupta, "The splash-2 programs: Characterization and methodological considerations", *ACM SIGARCH computer architecture news*, vol. 23, no. 2, pp. 24–36, 1995.

[232] "Working with the built-in fuzz test for device farm", [Online]. Available: https://docs.aws.amazon.com/devicefarm/latest/developerguide/test-types-built-in-fuzz.html.

[233] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt", in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 819–831.

[234] M. Xu, S. Kashyap, H. Zhao, and T. Kim, "Krace: Data race fuzzing for kernel file systems", in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1643–1660.

[235] C. Yang, Z. Zhao, and L. Zhang, "Kernelgpt: Enhanced kernel fuzzing via large language models", in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25, Rotterdam, Netherlands: Association for Computing Machinery, 2025, pp. 560–573, ISBN: 9798400710797. [Online]. Available: https://doi.org/10.1145/3676641.3716022.

[236] Y. Yang, X. Chen, and G. Gopalakrishnan, "Inspect: A runtime model checker for multithreaded c programs", Technical Report UUCS-08-004, University of Utah, Tech. Rep., 2008.

[237] K. Yasukata and K. Ishiguro, "Developing process scheduling policies in user space with common os features", in *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems*, ser. APSys '24, Kyoto, Japan: Association for Computing Machinery, 2024, pp. 38–44, ISBN: 9798400711053. [Online]. Available: https://doi.org/10.1145/3678015.3680481.

[238] J. Yu and S. Narayanasamy, "A case for an interleaving constrained shared-memory multi-processor", *ACM SIGARCH Computer Architecture News*, vol. 37, no. 3, pp. 325–336, 2009.

[239] J. Yu, S. Narayanasamy, C. Pereira, and G. Pokam, "Maple: A coverage-driven testing tool for multithreaded programs", in *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*, 2012, pp. 485–502.

[240] X. Yuan and J. Yang, "Effective concurrency testing for distributed systems", in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 1141–1156.

[241] X. Yuan, J. Yang, and R. Gu, "Partial order aware concurrency sampling", in *Computer Aided Verification: 30th International Conference, CAV 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings, Part II 30*, Springer, 2018, pp. 317–335.

[242] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input", *IEEE Transactions on software engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[243] J. Zhang, Z. Cui, X. Chen, H. Yang, L. Zheng, and J. Liu, "Cidfuzz: Fuzz testing for continuous integration", *IET Software*, vol. 17, no. 3, pp. 301–315, Apr. 2023.

[244] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement", in *International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[245] H. Zhao, D. Wolff, U. Mathur, and A. Roychoudhury, "Selectively uniform concurrency testing", in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 1003–1019.

[246] X. Zhu and M. Böhme, "Regression greybox fuzzing", in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21, Virtual Event, Republic of Korea: Association for Computing Machinery, 2021, pp. 2169–2182, ISBN: 9781450384544. [Online]. Available: https://doi.org/10.1145/3460120.3484596.

Publications during PhD Study

[1] D. Wolff, Z. Shi, G. J. Duck, U. Mathur, and A. Roychoudhury, "Greybox fuzzing for concurrency testing", in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24, La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 482–498, ISBN: 9798400703850. [Online]. Available: https://doi.org/10.1145/3620665.3640389.

[2] D. Wolff, M. Mirchev, and A. Roychoudhury, "Large language models in software security analysis", *Communications of the ACM*, vol. (to appear), 2026.

[3] D. Wolff, R. Shariffdeen, Y. Noller, and A. Roychoudhury, "Shifting fuzzing left in software workflows", *Empirical Software Engineering*, vol. 30, no. 5, p. 146, 2025.

[4] D. Wolff, M. Böhme, and A. Roychoudhury, "Fuzzing: On benchmarking outcome as a function of benchmark properties", *ACM Transactions on Software Engineering and Methodology*, 2025.

[5] H. Zhao, D. Wolff, U. Mathur, and A. Roychoudhury, "Selectively uniform concurrency testing", in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 1003–1019.

[6] Z. Luo, H. Zhao, D. Wolff, C. Cadar, and A. Roychoudhury, "Agentic concolic execution", in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2026, pp. 1–19.